

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS



SI807 SISTEMAS DE INTELIGENCIA DE NEGOCIOS

**“Arquitectura de Inteligencia de Negocios en Microsoft Azure para la evaluación
de Nivel de Madurez de servicios en BBVA”**

INFORME TÉCNICO FINAL

GRUPO N°4:

Código	Apellidos y Nombres	Correo Electrónico
20222029J	Cárdenas Palacios Leonardo Gustavo	leogcardenas@gmail.com
20222048D	Espinoza Cerna, Alex	espinozacernaalex@gmail.com
20222144C	Inocente Caro, Miguel Anderson	miguelander30@gmail.com

Profesores: Dr. Ing. Aradiel Castaneda, Hilario y García Atuncar, Fernando

Diciembre, 2025

Indice

1. IMPLEMENTACIÓN DE LA SOLUCIÓN EN LA NUBE

- 1.1. Justificación Nube
 - 1.1.1. Contexto General
 - 1.1.2. Evolución del Proyecto
- 1.2. Selección de la Nube
 - 1.2.1. Análisis por Criterio
 - 1.2.2. Decisión Final
- 1.3. Arquitectura Nube
 - 1.3.1. Diagrama General de la Arquitectura
 - 1.3.2. Capas Funcionales de la Solución
 - 1.3.3. Servicios Azure Utilizados
 - 1.3.4. Flujo de Datos end-to-end
- 1.4. Matriz de Costos Nube
 - 1.4.1. Metodología de Estimación
 - 1.4.2. Costos reales observados en Azure Portal
 - 1.4.3. Costos proyectados mensuales y anuales

2. SERVICIOS DE NUBE DE LA SOLUCIÓN PROPUESTA

- 2.1. Asignar Roles AIM
 - 2.1.1. Explicación del Uso
 - 2.1.2. Comandos CLI
- 2.2. Crear los Resource Group
 - 2.2.1. Explicación del Uso
 - 2.2.2. Comandos CLI
- 2.3. Crear Key Vault
 - 2.3.1. Explicación del Uso
 - 2.3.2. Comandos CLI
- 2.4. Crear el Storage Account
 - 2.4.1. Explicación del Uso
 - 2.4.2. Comandos CLI
- 2.5. Crear PostgreSQL Flexible Server
 - 2.5.1. Explicación del Uso
 - 2.5.2. Comandos CLI
- 2.6. Crear Databricks
 - 2.6.1. Explicación del Uso
 - 2.6.2. Comandos CLI
- 2.7. Crear Container Registry
 - 2.7.1. Explicación del Uso
 - 2.7.2. Comandos CLI
- 2.8. Crear Container Apps Environment
 - 2.8.1. Explicación del Uso
 - 2.8.2. Comandos CLI
- 2.9. Crear el Container App
 - 2.9.1. Explicación del Uso
 - 2.9.2. Comandos CLI
- 2.10. Crear el Static Web App
 - 2.10.1. Explicación del Uso
 - 2.10.2. Comandos CLI

2.11. Crear el Log Analytics workspace

2.11.1. Explicación del Uso

2.11.2. Comandos CLI

2.12. Crear las Alertas

2.12.1. Explicación del Uso

2.12.2. Comandos CLI

2.13. Audit Logs

2.13.1. Explicación del Uso

2.14. Crear el Backup Vault

2.14.1. Explicación del Uso

2.14.2 Comandos CLI

2.15. Crear la Red

2.15.1. Explicación del Uso

2.15.2. Comandos CLI

2.16. Servicios Adicionales

3. ARCHIVOS UTILIZADOS

3.1. Notebooks

3.1.1. Practitioner Notebooks

3.1.2. Continuous Integration Notebooks

3.2. Querys

3.2.1. Practitioner Querys

3.2.2. Continuous Integration Querys

3.3. Data Cruda

3.3.1. Data Cruda Practitioner

3.3.2. Data Cruda Continuous Integration

3.4. Stack Tecnológico

3.4.1. Backend

3.4.2. Frontend

3.5. Kqls

4. VISUALIZACIÓN DEL DASHBOARD

4.1. Interfaz Subir Datos

4.2. Interfaz Ver Dashboard Global

4.3. Interfaz Ver Dashboard por Servicio

5. DISTRIBUCIÓN DE LOS ROLES GITHUB

6. REFERENCIAS BIBLIOGRAFIA

Indice de Figuras

- Figura 1. Diagrama de Flujo de proceso de Certificación Actual BBVA
- Figura 2. Diagrama Arquitectónico en la Nube Azur
- Figura 3. Carpeta de la Data Cruda
- Figura 4. Dashboard Funcional BBVA
- Figura 5. Servicios en el Portal de Azure
- Figura 6. Costos acumulados
- Figura 7. Costos por Recursos (Donut)
- Figura 8. Costos por Grupo de Recurso y Locación (Donut)
- Figura 9. Recursos asignados a la Persona 1
- Figura 10. Recursos asignados a la Persona 2
- Figura 11. Recursos asignados a la Persona 3
- Figura 12. Estructura de los recursos creados
- Figura 13. Vista general del Azure Key Vault kv-bbva-dashboard en Azure Portal
- Figura 14. Vista del recurso Storage Account
- Figura 15. Creación de los contenedores en Storage Account
- Figura 16. Estructura de la capa de bronce
- Figura 17. Estructura de la capa de plata
- Figura 18. Vista del servicio creado en PostgreSQL
- Figura 19. Estructura de la capa de Oro
- Figura 20. Diagrama relacional de base datos practitioner
- Figura 21. Diagrama relacional de base datos continuous integration
- Figura 22. Vista general del servicio Databricks
- Figura 23. Vista general de dentro del workspace
- Figura 24. Vista General de los Jobs
- Figura 25. Tareas del Job del Practitioner
- Figura 26. Tareas del Job del Continuous Integration
- Figura 27. Vista General de la vista los notebooks
- Figura 28. Vista general de Azure Container
- Figura 29. Vista general del servicio Registry
- Figura 30. Vista general del servicio Azure Container Apps
- Figura 31. Vista general del servicio Container App
- Figura 32. Vista general del servicio Static Web App
- Figura 33. Vista general de la sección Environments
- Figura 34. Vista General del Servicio Log Analytics Workspace
- Figura 35. Vista General de la Tabla StorageBlobLogs
- Figura 36. Estructura de las reglas de Alerta
- Figura 37. Vista General del Servicio Audit Logs
- Figura 38. Vista General del Servicio Backup Vault
- Figura 39: Vista General de las Propiedades del Servicio Backup Vault
- Figura 40. Vista General del Servicio Vnet
- Figura 41. Vista General de las Subnets Asignadas
- Figura 42. Vista General del Servicio Network Security Groups (NSG)
- Figura 43. Vista general de los Private DNS creados
- Figura 44. Vista General de la Configuración del Servicio Vnet
- Figura 45. Vista General de los servicios Principales
- Figura 46. Vista General de los Private Endpoints definidos
- Figura 47. Vista General de Network Interfaces (NIC) para cada servicio

- Figura 48. Vista General de los recursos asignados con el firewall respectivo
- Figura 49. Topología de la red
- Figura 50. Data Cruda Practitioner
- Figura 51. Data Cruda Continuous Integration
- Figura 52. Fragmento de estructura del Backend
- Figura 53. Fragmento de estructura del Frontend
- Figura 54. Botones de Navegación Superior
- Figura 55. Interfaz de carga inicial del Dashboard Niveles de Madurez
- Figura 56. Validación de estructura de archivo CSV con formato incorrecto
- Figura 57. Confirmación de carga exitosa del archivo CSV al Datalake
- Figura 58. Ejecución del proceso ETL en Databricks desde el Dashboard
- Figura 59. Mensaje de finalización del proceso de transformación exitoso
- Figura 60. Vista general del Dashboard Global
- Figura 61. Controles de selección entre niveles de madurez
- Figura 62. Tarjetas resumen con datos clave
- Figura 63. Filtros por Fecha y Geografía
- Figura 64. Distribución de niveles de certificación por porcentaje de adopción
- Figura 65. Niveles de certificación distribuidos por geografía
- Figura 66. Tabla detallada de cálculo de los KPI's por servicio y geografía
- Figura 67. Vista inicial del Dashboard por Servicio
- Figura 68. Ejemplo del Dashboard por Servicio
- Figura 69. Filtros en cascada para Nivel de Madurez, Geografía y Servicio N1
- Figura 70. Evolución mensual de la adopción total (modo mensual)
- Figura 71. Evolución acumulada de la adopción total (modo acumulado)
- Figura 72. Selector de KPI's para actualizar el gráfico evolutivo
- Figura 73. Indicador de Adopción Total mediante Gauge Chart (velocímetro)
- Figura 74. Indicador numérico del KPI RFO OK del servicio seleccionado
- Figura 75. Gráfico radar de KPIs del servicio
- Figura 76. Comparativo radar de KPIs
- Figura 77. Github Privado del Grupo 4
- Figura 78. Contribuciones por Miembro
- Figura 79. Merge por Miembro en la rama feature/grupo04-alex-pc4

Indice de Tablas

Tabla 1. Comparación de Seguridad y Cumplimiento Normativo
Tabla 2. Comparación de Escalabilidad y Disponibilidad
Tabla 3. Comparación de Pricing y Modelo de Costo
Tabla 4. Comparación de Servicios Big Data y Procesamiento Distribuido
Tabla 5. Comparación de Facilidad de Migración y Compatibilidad
Tabla 6. Evaluación ponderada por criterio
Tabla 7. Resumen de Resultados
Tabla 8. Servicios Azure por Capa / Dominio
Tabla 9. Costo Real por Servicios Azure
Tabla 10. Costos Mensuales y Anuales del Azure Data Lake Storage Gen 2
Tabla 11. Costos Mensuales y Anuales del Azure Databricks
Tabla 12. Costos Mensuales y Anuales del Azure Database for PostgreSQL
Tabla 13. Costos Mensuales y Anuales del Azure Container Apps
Tabla 14. Costos Mensuales y Anuales del Azure Static Web Apps
Tabla 15. Costos Mensuales y Anuales del Azure Key Vault
Tabla 16. Costos Mensuales y Anuales del Azure Monitor
Tabla 17. Costo total estimado Azure Nube
Tabla 18. Roles asignados a Persona 1
Tabla 19. Roles asignados a Personas 2
Tabla 20. Roles asignados a Persona 3

1. IMPLEMENTACIÓN DE LA SOLUCIÓN EN LA NUBE

1.1. Justificación Nube

1.1.1. Contexto General

A) Proceso de certificación actual (As-Is)

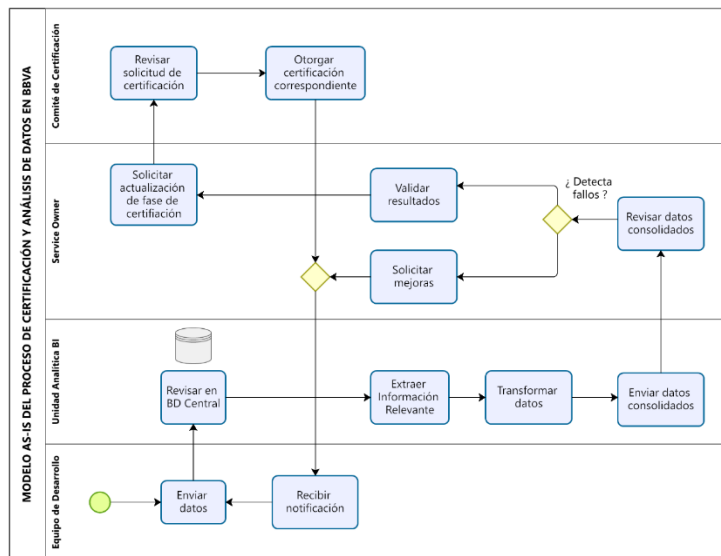
El proceso actual de certificación en BBVA se caracteriza por la ausencia de un modelo estandarizado de integración de información, lo que genera un flujo extenso y con múltiples puntos de fricción. Los datos provienen de diversas aplicaciones corporativas del área de Engineering (Jira, Bitbucket, GIAM, entre otras), con estructuras heterogéneas y sin mecanismos unificados que aseguren consistencia.

En la etapa inicial, los equipos registran información sin un formato común, lo que produce duplicidades, inconsistencias de nomenclatura y dificultades de consolidación. Posteriormente, la información se almacena en una base central sin criterios uniformes de calidad o categorización. La Unidad de Analítica y BI realiza entonces un esfuerzo intensivo de extracción, depuración y validación (usuarios, commits, trazabilidad e indicadores), incrementando el tiempo del proceso y el riesgo de inconsistencias.

Con base en los reportes, el Service Owner valida resultados y, ante hallazgos, devuelve observaciones al equipo, generando ciclos sucesivos de retrabajo. Finalmente, el Comité de Certificación/Gobernanza contrasta evidencias con políticas internas, pudiendo solicitar aclaraciones adicionales antes de emitir la aprobación. En conjunto, la fragmentación de fuentes y la dependencia de tareas manuales reducen la visibilidad del avance y prolongan significativamente el tiempo hasta la certificación final.

Este escenario muestra que el proceso actual de certificación dentro del área de Engineering depende de consolidaciones manuales, múltiples revisiones y ciclos sucesivos de validación, lo que lo vuelve lento, complejo y poco eficiente.

Figura 1. Diagrama de Flujo de proceso de Certificación Actual BBVA



Fuente: Elaboración propia.

B) Problema Central

El problema central es la fragmentación de la información y la baja automatización del proceso, lo que obliga a consolidaciones manuales, revisiones repetitivas y validaciones sucesivas. Esto impacta en la eficiencia operativa, la trazabilidad, la calidad, la seguridad y la capacidad de monitorear oportunamente el avance en niveles de madurez, limitando la toma de decisiones basada en evidencia.

1.1.2. Evolución del Proyecto

El proyecto implementa un flujo integral de Inteligencia de Negocios para monitorear los niveles Practitioner y Continuous Integration (CI). En su versión local, se basa en ingesta manual de CSV del Marco Playbook, ETL en PySpark sobre Hadoop/Hortonworks con Zeppelin, un Data Warehouse en PostgreSQL (modelo estrella) y una API Flask con dashboard React para visualizar KPIs.

La migración a Azure busca escalar el procesamiento distribuido sin administrar infraestructura física, simplificar la gestión con Data Lake y PostgreSQL administrado, y fortalecer seguridad, segmentación, cumplimiento y auditoría mediante servicios como VNet, NSG, Azure Firewall, Private Endpoints, Private DNS Zone, Key Vault, Azure Monitor, Log Analytics, Recovery Services Vault y Audit Logs. Asimismo, se mantiene una capa de exposición moderna con Azure Container Apps y Static Web Apps, reduciendo riesgos y costos asociados al on-premise y asegurando trazabilidad de consumo mediante monitoreo centralizado. Finalmente, la selección del proveedor cloud considera AWS, Azure y GCP, evaluados con criterios críticos alineados a soluciones de BI y Big Data.

1.2. Selección de la Nube

Los 7 criterios aplicados en la comparativa de selección fueron:

- Seguridad y cumplimiento normativo
- Escalabilidad y disponibilidad
- Pricing y modelo de costos
- Ecosistema de BI y análisis de datos
- Servicios Big Data y procesamiento distribuido
- Soporte y madurez de servicios
- Facilidad de migración y compatibilidad

1.2.1. Análisis por Criterio

A) Criterio 1: Seguridad y cumplimiento normativo

Los tres proveedores cuentan con catálogos amplios de certificaciones y servicios de seguridad. Sin embargo, en este proyecto el énfasis está en:

- **Aislamiento de red** (segmentación por VNet, subredes y reglas de tráfico).
- **Acceso privado a servicios PaaS** (Private Endpoints + Private DNS Zone).
- **Gestión segura de secretos** (Key Vault).
- **Supervisión centralizada** (Azure Monitor + Log Analytics + Audit Logs).
- **Respaldo y recuperación ante desastres** (Recovery Services Vault).

Tabla 1. Comparación de Seguridad y Cumplimiento Normativo

Aspecto clave	AWS	Azure	GCP
Enfoque de seguridad	Amplio portafolio de servicios de seguridad y cumplimiento normativo.	Enfoque de plataforma integrada: red privada (VNet), segmentación (NSG), Firewall, endpoints privados, Key Vault y monitoreo unificado.	Fuerte enfoque en protección de datos y seguridad gestionada para workloads.
Segmentación de red	VPC, Security Groups y Network ACLs (NACLs).	Virtual Network, subredes, Network Security Groups y Azure Firewall para definir perímetros y políticas de tráfico.	VPC, reglas de firewall, IAM y políticas a nivel de organización.
Acceso privado a servicios gestionados	Endpoints privados disponibles para ciertos servicios PaaS.	Private Endpoints junto con Private DNS Zone para exponer servicios únicamente dentro de la VNet.	Private Service Connect para acceso privado a servicios gestionados.
Gestión de secretos	AWS KMS y Secrets Manager.	Azure Key Vault como almacén central cifrado de secretos, claves y certificados, integrado nativamente con los servicios de la solución.	Cloud KMS y Secret Manager.
Monitoreo y auditoría	CloudTrail y CloudWatch para auditoría y métricas.	Azure Monitor, Log Analytics Workspace y Audit Logs para métricas, logging centralizado y trazabilidad de acciones.	Cloud Logging y Cloud Monitoring.
Respaldo y recuperación	Servicios de backup y DR específicos por tipo de recurso.	Recovery Services Vault para copias de seguridad y recuperación ante desastres, con políticas de retención configurables.	Respaldo y DR gestionados a nivel de servicios individuales.
Alineamiento con entorno bancario	Alto.	Muy alto: diseñado para entornos enterprise regulados, con integración nativa de red privada, gestión de claves, monitoreo, auditoría y respaldo centralizado.	Medio–alto.

Fuente: Microsoft Azure. (2024). Azure security documentation.

En un contexto similar al de BBVA, donde la seguridad, el aislamiento de red y la trazabilidad son críticos, **Azure** ofrece una combinación sólida de:

- **Segmentación de red** (VNet, NSG, Firewall).
- **Acceso privado a datos y servicios** (Private Endpoints, Private DNS Zone).
- **Protección de secretos** (Key Vault).
- **Monitoreo y auditoría centralizados** (Azure Monitor, Log Analytics, Audit Logs).
- **Respaldo y recuperación** (Recovery Services Vault).

Esto permite construir un flujo BI de uso interno con un nivel de gobierno y cumplimiento alineado a entornos financieros.

B) Criterio 2: Escalabilidad y disponibilidad

Los tres proveedores dominan el mercado de nube y ofrecen altos niveles de disponibilidad. Para este proyecto, los aspectos más relevantes son:

- Escalar el procesamiento distribuido (PySpark en Databricks).
- Escalar el Data Warehouse relacional (PostgreSQL administrado).
- Publicar el backend Flask y el frontend React en servicios gestionados, sin administrar servidores físicos.
- Proteger todo dentro de una VNet con controles de tráfico y monitoreo.

Tabla 2. Comparación de Escalabilidad y Disponibilidad

Aspecto clave	AWS	Azure	GCP
Cobertura global	Líder histórico, con gran número de regiones y zonas de disponibilidad.	Número elevado de regiones, con fuerte presencia en entornos enterprise y soporte nativo para escenarios híbridos.	Menor número de regiones, con una red global altamente optimizada.
Escalabilidad de cómputo	EC2 con Auto Scaling, contenedores en ECS/EKS y funciones serverless (Lambda).	Servicios de cómputo y contenedores desplegados dentro de una VNet, protegidos con NSG y Azure Firewall, con escalado horizontal según la demanda.	GKE, Cloud Run, App Engine y Cloud Functions.
Escalabilidad de datos	Amazon S3 junto con Redshift, Aurora y RDS para procesamiento y analítica.	Azure Data Lake y PostgreSQL administrado, expuestos mediante Private Endpoints dentro de la VNet.	Cloud Storage junto con BigQuery y Cloud SQL.
Disponibilidad gestionada	Alta disponibilidad, con	Alta disponibilidad con SLAs competitivos, complementable con	Muy alta, especialmente en servicios

	SLAs definidos por cada servicio.	Recovery Services Vault para fortalecer la continuidad del servicio.	analíticos serverless.
--	-----------------------------------	--	------------------------

Fuente: Gartner. (2024). *Quadrant for Cloud Infrastructure and Platform Services*.

Para un flujo ETL + Data Lake + DW + API + frontend, Azure permite:

- Desplegar los componentes críticos dentro de una red privada (VNet).
- Proteger el acceso con NSG + Azure Firewall.
- Exponer servicios PaaS mediante Private Endpoints.
- Monitorizar el comportamiento con Azure Monitor + Log Analytics.

Todo ello sin necesidad de administrar hardware propio y manteniendo la posibilidad de escalar los recursos a medida que crecen los volúmenes de datos o usuarios.

C) Criterio 3: Pricing y modelo de costos

AWS, Azure y GCP utilizan un modelo de pago por uso, con la opción de descuentos por compromisos de uso y herramientas de control de costos. Para este proyecto interesa:

- Poder empezar pequeño y crecer con el tiempo.
- Pagar principalmente por ejecuciones de Databricks y capacidad del DW PostgreSQL.
- Tener trazabilidad de los recursos que más consumen mediante métricas y logs.

Tabla 3. Comparación de Pricing y Modelo de Costos

Aspecto clave	AWS	Azure	GCP
Modelo base de costos	Esquema <i>pay-as-you-go</i> complementado con Reserved Instances y Savings Plans.	Modelo <i>pay-as-you-go</i> con reservas por capacidad, apoyado en métricas unificadas mediante Azure Monitor y Log Analytics.	Esquema <i>pay-as-you-go</i> con descuentos automáticos por uso sostenido.
Visibilidad y control de costos	Cost Explorer y Budgets para seguimiento y control del gasto.	Portal unificado con vistas de consumo por recurso, grupo de recursos y etiquetas, complementado con métricas y logs de Azure Monitor.	Herramientas integradas de presupuesto, alertas y análisis de uso.
Ajuste a cargas BI tipo batch	Viable, aunque requiere controlar recursos que permanecen encendidos.	Viable y optimizado mediante combinación de SKUs pequeños, escalado por demanda y políticas de apagado en entornos no productivos.	Muy competitivo en escenarios completamente <i>serverless</i> .

Fuente: Flexera. (2024). *State of the Cloud Report: Cloud Pricing Analysis*.

Aunque GCP resulta muy atractivo en escenarios totalmente serverless, en un proyecto que combina Spark + DW relacional + aplicaciones web, Azure ofrece un equilibrio razonable entre:

- Flexibilidad para crecer de forma gradual.
- Herramientas nativas para entender dónde se está gastando (métricas y logs centralizados).
- Opciones de reservas en aquellas capas que luego se estabilicen.

D) Criterio 4: Ecosistema de BI y análisis de datos

El proyecto actual construye dashboards y tablas interactivas mediante un frontend React publicado en Static Web Apps, consumiendo datos procesados por Databricks y almacenados en PostgreSQL.

Aun así, se consideró el ecosistema de BI de cada nube a nivel estratégico (posible evolución futura):

- **AWS:** QuickSight como solución BI principal.
- **Azure:** ecosistema BI de Microsoft, integrado con plataformas de datos y servicios de Azure.
- **GCP:** Looker y Looker Studio, muy orientados a entornos data/analytics.

Aunque la versión actual del proyecto utiliza un dashboard propio (React), elegir Azure mantiene abierta la posibilidad de integrar, a futuro, herramientas BI del ecosistema Microsoft de forma natural, sin cambiar de proveedor de nube.

E) Criterio 5: Servicios Big Data y procesamiento distribuido

La arquitectura local está basada en Spark sobre Hadoop. En la nube se busca:

- Mantener la lógica en PySpark.
- Dejar de manejar clústeres manualmente.
- Aprovechar un Data Lake con capas Bronze/Silver/Gold.

En Azure, esto se resuelve con:

- Azure Databricks como entorno gestionado para notebooks PySpark.
- Azure Data Lake para el almacenamiento en crudo y transformado.

Tabla 4. Comparación de Servicios Big Data y Procesamiento Distribuido

Aspecto	AWS	Azure	GCP
Spark gestionado	Amazon EMR y AWS Glue para procesamiento distribuido con Spark.	Azure Databricks, servicio de primer nivel totalmente integrado a la plataforma Azure.	Dataproc para ejecución de cargas Spark gestionadas.
Integración con Data Lake	Amazon S3 como capa de almacenamiento para datos analíticos.	Azure Data Lake con soporte nativo para arquitectura	Cloud Storage como repositorio

		Medallion (Bronze, Silver, Gold).	principal de datos.
Continuidad con arquitectura local	Requiere adaptación del entorno y de los flujos existentes.	Permite migrar notebooks PySpark y sustituir HDFS por Data Lake con cambios mínimos.	Requiere adaptación de workloads al entorno Dataproc.

Fuente: *Databricks. (2024). The Databricks Lakehouse Platform Architecture Guide.*

F) Criterio 6: Soporte y madurez de servicios

En cuota de mercado y madurez:

- **AWS** sigue liderando en participación.
- **Azure** se sitúa en segundo lugar con fuerte foco enterprise.
- **GCP** ocupa el tercer lugar, con foco en datos e IA.

Los tres son proveedores “tier 1”, pero Azure está especialmente bien posicionado en entornos corporativos que ya utilizan tecnologías Microsoft.

Para un proyecto que desea parecerse a cómo trabajaría una empresa grande (como un banco) que ya usa herramientas Microsoft, Azure ofrece una alineación más directa en:

- Prácticas recomendadas.
- Documentación y ejemplos.
- Ecosistema de partners y soporte corporativo.

G) Criterio 7: Facilidad de migración y compatibilidad

El stack actual está conformado por:

- **PySpark + Zeppelin** (ETL).
- **HDFS** (staging / data lake).
- **PostgreSQL** (Data Warehouse relacional con modelo estrella).
- **Flask** (backend).
- **React** (frontend).

Tabla 5. Comparación de Facilidad de Migración y Compatibilidad

Componente actual	AWS	Azure	GCP
PySpark + Zeppelin	EMR Notebooks y AWS Glue para ejecución y gestión de notebooks Spark.	Azure Databricks , con notebooks PySpark totalmente gestionados e integrados a la plataforma.	Dataproc con soporte de notebooks gestionados.
HDFS (Hortonworks)	Amazon S3 como reemplazo del sistema de archivos distribuido.	Azure Data Lake como capa de almacenamiento analítico escalable.	Cloud Storage como almacenamiento distribuido.

PostgreSQL (DW)	RDS PostgreSQL o Amazon Redshift para analítica.	PostgreSQL administrado en Azure (servicio PaaS).	Cloud SQL o BigQuery para analítica y reporting.
Flask + React	ECS/EKS, Elastic Beanstalk y AWS Amplify para backend y frontend.	Azure Container Apps para el backend y Azure Static Web Apps para el frontend.	Cloud Run o GKE para despliegue de aplicaciones.

Fuente: Microsoft Azure. (2024). *Azure Migration and Modernization Program Guide*.

La migración “uno a uno” del stack actual es especialmente directa hacia Azure:

- ETL PySpark → notebooks en Azure Databricks.
- HDFS → Azure Data Lake.
- PostgreSQL local → PostgreSQL administrado en Azure.
- Flask + React → Container Apps + Static Web Apps, con tráfico protegido dentro de una VNet y expuesto sólo a través de los puntos que se definan.

Esto reduce retrabajo y permite concentrarse en mejorar el flujo BI.

1.2.2. Decisión Final

Se asignaron pesos porcentuales a cada uno de los 7 criterios según su relevancia para el proyecto y se calificó a cada proveedor en una escala 1–4: 1 = Regular, 2 = Bueno, 3 = Muy bueno y 4 = Excelente.

Tabla 6. Evaluación ponderada por criterio

Característica	Peso	GCP	AWS	Azure
Seguridad y Cumplimiento Normativo	20 %	3	4	4
Escalabilidad y Disponibilidad	18 %	4	4	4
Pricing y Modelo de Costos	16 %	3	3	4
Ecosistema de BI y Análisis de Datos	15 %	3	3	4
Servicios Big Data y Procesamiento Distribuido	15 %	4	3	4
Soporte y Madurez de Servicios	10 %	3	4	4
Facilidad de Migración y Compatibilidad	6 %	3	4	4

Fuente: *Elaboración propia*.

Tabla 7. Resumen de Resultados

Proveedor	Puntuación ponderada (1–4)
GCP	3.33
AWS	3.54
Azure	4.00

Fuente: : Elaboración basada en Microsoft Azure. (2024). Cloud Adoption Framework.

Azure obtiene la puntuación máxima posible (4.00), superando a AWS (3.54) y GCP (3.33) en la evaluación global, de acuerdo con los pesos definidos para los criterios del proyecto.

1.3. Arquitectura Nube

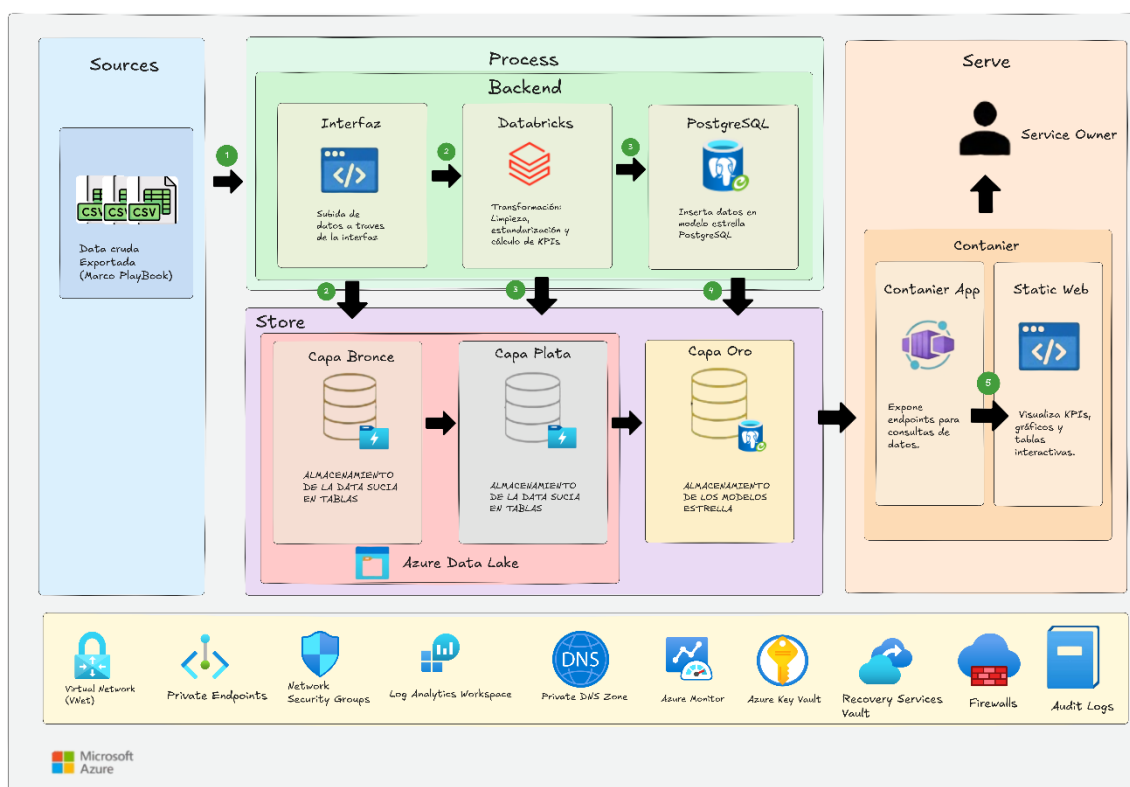
La arquitectura en Azure despliega un flujo completo de Business Intelligence para medir los niveles de madurez *Practitioner* y *Continuous Integration (CI)* a partir de archivos CSV generados por el Marco Playbook.

La solución cloud mantiene la lógica funcional de la arquitectura on-premise (ETL en Spark + modelo estrella + dashboard web), pero la mejora en:

- Escalabilidad y elasticidad (autoscaling de cómputo y servicios gestionados).
- Alta disponibilidad y recuperación ante desastres.
- Seguridad, redes privadas y gobierno de datos.
- Observabilidad centralizada (métricas, logs y alertas).

1.3.1. Diagrama General de la Arquitectura

Figura 2. Diagrama Arquitectónico en la Nube Azure



Fuente: Elaboración propia.

El siguiente diagrama de arquitectura muestra lo siguiente:

- **Sources:** archivos CSV del Marco Playbook.

- **Process / Backend:** interfaz de carga, Databricks y base de datos PostgreSQL gestionada.
- **Store:** Data Lake con capas Bronze, Silver y Gold.
- **Serve:** backend en Azure App Service y frontend en Azure Static Web Apps.
- **Servicios transversales:** VNet, Private Endpoints, NSG, Azure Firewall, Key Vault, Azure Monitor, Log Analytics, Private DNS Zone, Recovery Services Vault y Audit Logs.

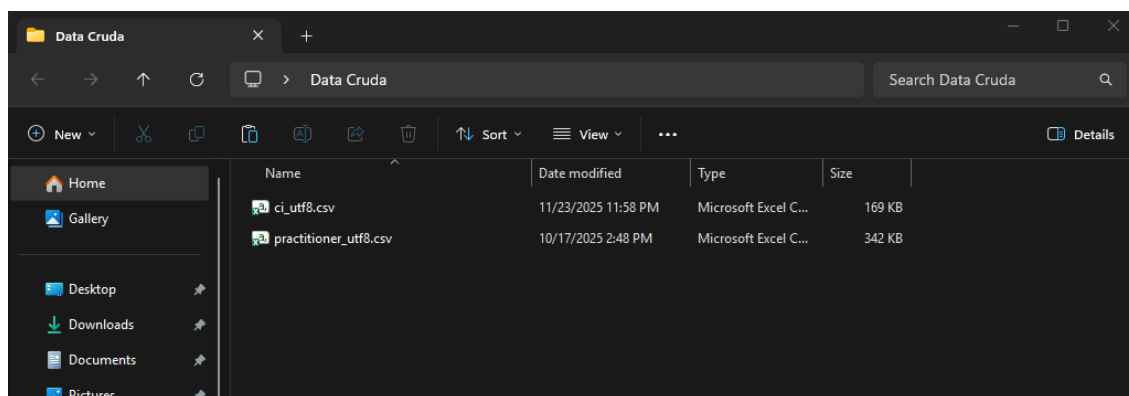
1.3.2. Capas Funcionales de la Solución

La arquitectura se organiza en capas lógicas, siguiendo el flujo de datos de extremo a extremo.

A) Fuentes de datos (Sources)

- **Origen principal:** archivos CSV exportados manualmente desde el Marco Playbook.
- Cada mes se cargan dos archivos:
 - CSV Practitioner.
 - CSV Continuous Integration.

Figura 3. Carpeta de la Data Cruda



Fuente: Carpetas de PC.

B) Capa de proceso (Backend)

Componentes principales:

- **Interfaz de carga**
Formulario web que permite al *Service Owner* subir los CSV al entorno cloud.
- **Azure Databricks**
Ejecuta los notebooks PySpark que realizan:
 - Limpieza y validación de datos.
 - Estandarización de tipos y formatos.
 - Cálculo de KPIs de Practitioner y CI.
 - Generación de las tablas finales para el modelo estrella.
- **PostgreSQL gestionado en Azure**

Base de datos analítica donde se implementa el modelo dimensional tipo estrella (tablas de hechos y dimensiones) que será consumido por el dashboard y por futuros casos de uso.

C) Capa de almacenamiento (Store)

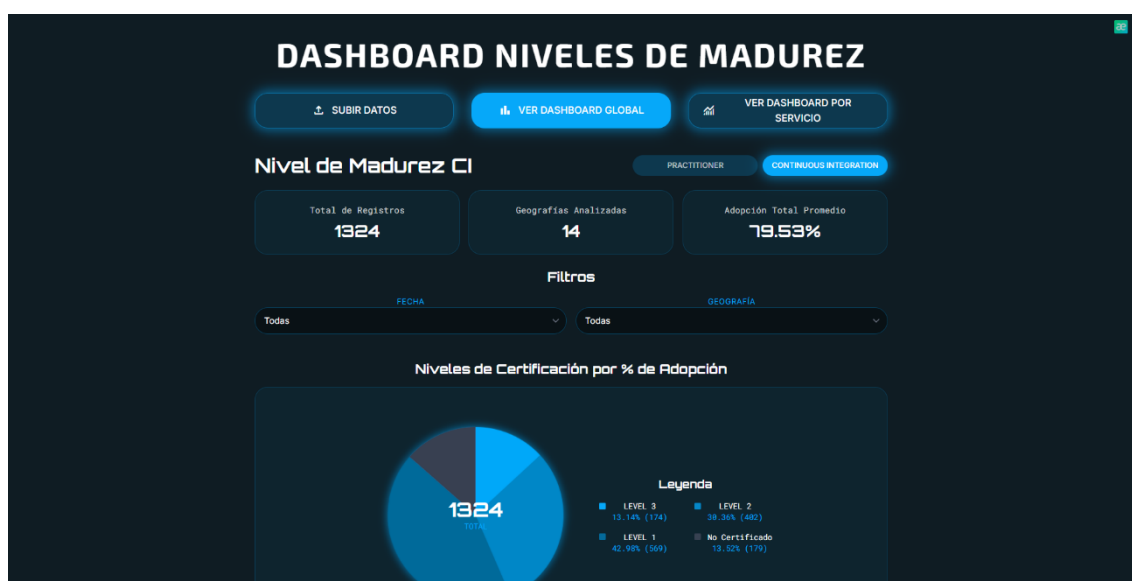
La solución utiliza un Data Lake en Azure con una arquitectura tipo Medallion:

- **Capa Bronze (raw)**
Almacena los CSV tal como llegan desde el Marco Playbook, sin transformar. Es la “fuente de la verdad” histórica.
- **Capa Silver (trusted)**
Contiene los datos limpios y tipificados en formato optimizado (por ejemplo, Parquet). Aquí ya se han aplicado reglas de calidad (nulos, duplicados, tipos).
- **Capa Gold (refined)**
Concentra los datos agregados y listos para consumo analítico y carga al modelo estrella en PostgreSQL.

D) Capa de presentación (Serve)

- **Azure App Service (backend Flask)**
Expone una API REST que consulta el modelo estrella y sirve los KPIs y métricas necesarios para el dashboard.
- **Azure Static Web Apps (frontend React)**
Hospeda una Single Page Application (SPA) desarrollada en React que:
 - Consume las APIs del backend.
 - Muestra tablas, gráficos y KPIs de madurez Practitioner/CI.
 - Actúa como “visor BI” personalizado, en lugar de utilizar un servicio de BI cloud externo (Power BI, Looker, etc.).

Figura 4. Dashboard Funcional BBVA



Fuente: Dashboard BI BBVA.

1.3.3. Servicios Azure Utilizados

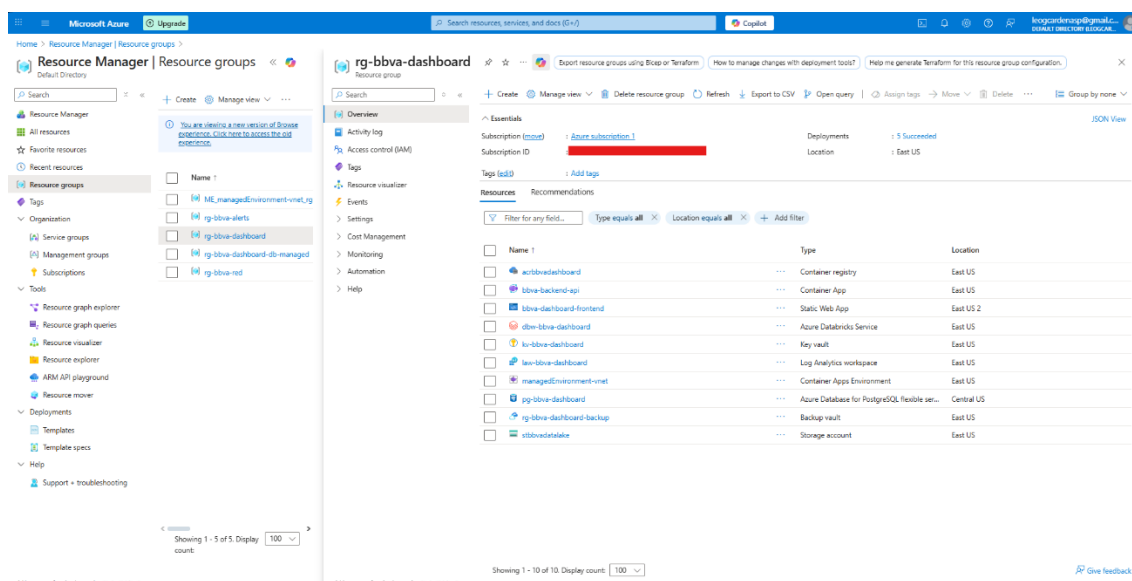
La siguiente tabla resume los servicios clave de la arquitectura:

Tabla 8. Servicios Azure por Capa / Dominio

Capa / Dominio	Servicio Azure	Rol principal en la solución
Red y seguridad	Virtual Network (VNet)	Red privada que aísla los recursos de la solución.
	Subredes + Network Security Groups	Segmentación de servicios (datos, backend, seguridad) y filtrado de tráfico a nivel L4.
	Private Endpoints	Acceso privado y seguro a cuentas de almacenamiento y bases de datos.
	Azure Firewall	Firewall centralizado para control de tráfico entrante y saliente.
	Private DNS Zone	Resolución DNS interna para recursos con endpoints privados.
	Audit Logs	Auditoría de acciones administrativas y eventos de seguridad.
Procesamiento	Azure Databricks	Ejecución de notebooks PySpark para el proceso ETL completo.
Almacenamiento	Azure Data Lake (Bronze / Silver / Gold)	Almacenamiento estructurado bajo arquitectura Medallion.
	Blob Storage	Zona de aterrizaje inicial para archivos CSV del Marco Playbook.
Datos relacionales	PostgreSQL gestionado en Azure	Data Warehouse con modelo dimensional (hechos y dimensiones).
Aplicación / API	Azure App Service	Backend Flask que expone KPIs y métricas mediante API REST.
Presentación	Azure Static Web Apps	Aplicación SPA en React que actúa como visor BI en la nube.
Seguridad de secretos	Azure Key Vault	Almacenamiento seguro de credenciales y cadenas de conexión.
Monitoreo y logging	Azure Monitor + Log Analytics	Centralización de métricas, logs y configuración de alertas.
Backup / DR	Recovery Services Vault	Gestión de copias de seguridad y recuperación ante desastres.

Fuente: *Elaboración propia.*

Figura 5. Servicios en el Portal de Azure



Fuente: *Portal de Azure.*

1.3.4. Flujo de Datos end-to-end

A) Carga de archivos

El *Service Owner* descarga los CSV del Marco Playbook y los sube a través de la interfaz web. La aplicación los envía a Blob Storage (zona de staging).

B) Ingesta al Data Lake (Bronze)

Un proceso controlado carga los CSV desde Blob a la capa Bronze del Data Lake, preservando la estructura original.

C) Procesamiento en Databricks (Silver)

Azure Databricks:

- Lee los archivos de Bronze.
- Aplica reglas de limpieza y tipificación.
- Escribe los resultados limpios en la capa Silver.

D) Enriquecimiento y agregación (Gold)

Nuevos notebooks en Databricks:

- Realizan joins, agregaciones y cálculo de KPIs.
- Escriben las tablas refinadas en la capa Gold.

E) Carga al Data Warehouse (PostgreSQL)

Databricks inserta/actualiza las tablas de hechos y dimensiones en PostgreSQL utilizando los datos refinados de Gold.

F) Exposición vía API (App Service)

El backend Flask, desplegado en App Service, expone endpoints REST que consultan el modelo estrella.

G) Visualización (Static Web Apps)

La SPA React, hospedada en Azure Static Web Apps:

- Consume las APIs del backend.
- Presenta KPIs, gráficos y tablas al *Service Owner* y al resto de usuarios internos.

1.4. Matriz de Costos Nube

1.4.1. Metodología de Estimación

Para la estimación de costos se utilizó la herramienta oficial Azure Pricing Calculator (Microsoft Azure, 2024), la cual permite calcular los costos proyectados de los servicios cloud seleccionados en función de la configuración y volumetría esperada.

Enfoque aplicado (estimación + validación real):

La matriz combina (1) estimación teórica con Pricing Calculator y (2) validación con costos reales obtenidos desde Azure Portal → Cost Management, exportados a Excel (Cost Management export). Esto permite que el capítulo no se quede en supuestos: los montos reales son los que facturó la propia nube según el consumo del entorno desplegado.

A) Supuestos de uso y volumetría (ajustados)

a) Frecuencia de procesamiento (Databricks)

- **24 ejecuciones/año** (2 jobs mensuales):
 - **12 jobs/ año** para data cruda *Practitioner*
 - **12 jobs/ año** para data cruda *Continuous Integration*

Nota: El clúster se utiliza bajo enfoque *batch/job* (no 24/7), lo que favorece costos controlables.

b) Volumen de datos

- CSV Practitioner: 50 MB/mes (600 MB/año)
- CSV Continuous Integration: 80 MB/mes (960 MB/año)
- **Total ingesta:** 130 MB/mes $\approx$ 1.56 GB/año

c) Azure Data Lake Storage Gen2

- Bronze: 5 GB (crudos históricos)
- Silver: 8 GB (Parquet)
- Gold: 3 GB (Delta/curado)
- **Total estimado:** 16 GB

d) Base de datos gestionada en Azure (PostgreSQL)

- Servicio: Azure Database for PostgreSQL (General Purpose $\approx$ 4 vCores)
- Tamaño: 10 GB inicial $\rightarrow$ 25 GB a 12 meses
- Carga: $\approx$ 50,000 queries/mes
- Concurrencia: $\sim$ 10 usuarios

e) Azure Databricks

- Clúster: Standard_DS3_v2 (4 vCores, 14 GB RAM)
- Autoscaling: 2–8 workers (promedio 4 durante ejecución)
- Horas estimadas: 4 h/mes (consideradas como el total mensual de ambos jobs)

f) Backend (API Flask)

- Despliegue: Azure Container Apps
- Consumo equivalente: 1–2 vCPU, $\sim$ 2 GB RAM
- Autoscaling: 1–3 réplicas (promedio 1.5)

g) Frontend (React)

- Servicio: **Azure Static Web Apps – Free**

h) Seguridad, monitoreo y gobierno

- Azure Key Vault, Azure Monitor + Log Analytics
- Componentes de red: VNet, NSG, Private Endpoints, Private DNS Zone
- Azure Firewall (evaluado): se incluyó en el piloto, pero se concluye no recomendado por el costo real observado (ver 1.4.2 y 1.4.3).

B) Consideraciones

- Región: **East US** (referencia)
- Redundancia: **LRS**
- Reservas: no consideradas (pay-as-you-go)
- Descuentos: se mantiene el supuesto de $\approx$ **40%** en base de datos solo como escenario empresarial (cuando aplique).

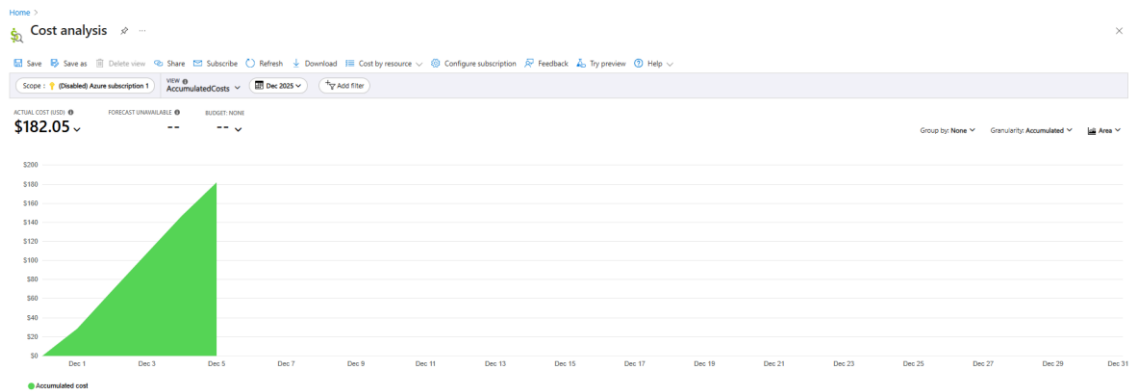
1.4.2. Costos reales observados en Azure Portal (Cost Management)

En esta sección se reportan costos reales obtenidos del Azure Portal, exportados a Excel (Cost Management). Es decir, son montos facturados por Azure según consumo, no simulaciones.

A) Evidencias

En este apartado se muestran las evidencias de los costos reales que se encuentran en el Portal de Azure.

Figura 6. Costos acumulados



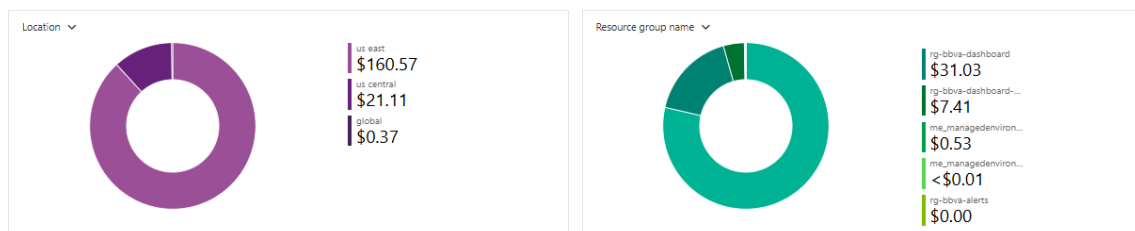
Fuente: Portal de Azure - Cost Management.

Figura 7. Costos por Recursos (Donut)



Fuente: Portal de Azure - Cost Management.

Figura 8. Costos por Grupo de Recurso y Locación (Donut)



Fuente: Portal de Azure - Cost Management.

B) Resumen real por servicio (USD)

Tabla 9. Costo Real por Servicios Azure

Servicio (Azure)	Costo real (USD)
Azure Firewall	136.67
Azure Database for PostgreSQL	21.11
Virtual Network	7.16
Container Registry	6.63
NAT Gateway	5.45
Azure Databricks	3.29
Virtual Machines	0.82
Storage	0.54
Azure DNS	0.37
Bandwidth	0.01
Key Vault	~0.00
TOTAL	182.05

Fuente: Portal de Azure - Cost Management.

C) Lectura clave del resultado real

- El costo no estuvo dominado por Data Lake o Databricks.
- El principal driver real fue Azure Firewall, que consumió ≈ 136 USD en muy poco tiempo, concentrando la mayor parte del total.

D) Conclusión del piloto

Aunque el Firewall aporta control perimetral, para este escenario (BI batch, bajo tráfico y entorno académico/prototipo) no es costo-eficiente porque:

- La arquitectura ya se apoya en VNet + NSG + Private Endpoints + Private DNS, que cubren segmentación y acceso privado.
- El Firewall elevó el gasto de forma desproporcionada frente al valor agregado real en el piloto.

Resultado práctico: se recomienda excluir Azure Firewall en el diseño final y mantener seguridad costo-eficiente con VNet/NSG/Private Endpoints + Key Vault + Monitor/Logs.

1.4.3. Costos proyectados mensuales y anuales (Pricing Calculator)

A continuación se presentan los **costos proyectados** por servicio (mensual y anual), manteniendo tu estructura original, pero con dos mejoras:

- Se declara explícitamente que la proyección se estima con Pricing Calculator.
- Se agrega un escenario recomendado sin Azure Firewall, sustentado por la evidencia real (1.4.2).

A) Costos por servicio (proyección teórica – Pricing Calculator)

a) Servicio 1: Azure Data Lake Storage Gen2

Tabla 10. Costos Mensuales y Anuales del Azure Data Lake Storage Gen 2

Componente	SKU/Tier	Volumen mensual	Costo mensual (USD)	Costo anual (USD)	Justificación
Almacenamiento Hot Tier	Standard LRS	16 GB	0.37	4.44	Capas Bronze/Silver/Gold con acceso mensual
Operaciones Lectura (Class 2)	—	10,000 ops/mes	0.01	0.12	Lecturas en Bronze/Silver
Operaciones Escritura (Class 1)	—	5,000 ops/mes	0.10	1.20	Escrituras en Bronze/Silver/Gold
Total Data Lake Gen2			0.48	5.76	

Fuente: Pricing Calculator.

b) Servicio 2: Azure Databricks (Premium Tier)

Se considera uso por jobs: 2 ejecuciones mensuales (Practitioner + CI) = 24 ejecuciones/año. La estimación mantiene 1 h/mes totales (por ambos jobs), evitando clúster permanente.

Tabla 11. Costos Mensuales y Anuales del Azure Databricks

Componente	SKU/Tier	Volumen mensual	Costo mensual (USD)	Costo anual (USD)	Justificación
Databricks Units	Premium Jobs	16 DBU/mes	0.90	10.80	4 h/mes × 4 DBU/h
Compute VM (driver)	Standard_DS3_v2	4 h/mes	0.68	8.16	Driver activo en ejecución
Compute VM (workers)	Standard_DS3_v2	16 worker-hours/mes	2.72	32.64	4 workers promedio × 4 h
Total Databricks			4.30	51.60	

Fuente: Pricing Calculator.

c) Servicio 3: Azure Database for PostgreSQL

Tabla 12. Costos Mensuales y Anuales del Azure Database for PostgreSQL

Componente	SKU/Tier	Volumen mensual	Costo mensual (USD)	Costo anual (USD)	Justificación
Compute (vCore)	General Purpose, 4 vCores	730 h/mes	423.60	5,083.20	Carga dashboard/API
Storage	Premium SSD	25 GB	6.25	75.00	Crecimiento 12 meses
Backup Storage (LRS)	Backups automatizados	25 GB	2.50	30.00	Retención típica
Subtotal			432.35	5,188.20	
Descuento corporativo ($\approx 40\%$)	—	—	-172.94	-2,075.28	Escenario enterprise
Total Base de Datos			259.41	3,112.92	

Fuente: *Pricing Calculator*.

d) Servicio 4: Azure Container Apps (backend Flask)

Tabla 13. Costos Mensuales y Anuales del Azure Container Apps

Componente	SKU/Tier	Volumen mensual	Costo mensual (USD)	Costo anual (USD)	Justificación
Cómputo base (réplica 1)	Consumo equivalente	730 h/mes	69.35	832.20	1–2 vCPU, 2 GB RAM
Autoscaling (extra)	Consumo equivalente	365 h/mes	34.68	416.10	Promedio 1.5 réplicas
Total Container Apps			104.03	1,248.30	

Fuente: *Pricing Calculator*.

e) Servicio 5: Azure Static Web Apps (frontend React)

Tabla 14. Costos Mensuales y Anuales del Azure Static Web Apps

Componente	SKU/Tier	Volumen mensual	Costo mensual (USD)	Costo anual (USD)	Justificación
Static Web Apps Plan	Free	Dentro del plan	0.00	0.00	SPA React sin costo

Fuente: *Pricing Calculator*.

f) Servicio 6: Azure Key Vault

Tabla 15. Costos Mensuales y Anuales del Azure Key Vault

Componente	SKU/Tier	Volumen mensual	Costo mensual (USD)	Costo anual (USD)	Justificación
Secret Operations	Standard Tier	10,000 ops/mes	0.03	0.36	Lectura de secretos
Total Key Vault			0.03	0.36	

Fuente: Pricing Calculator.

g) Servicio 7: Azure Monitor + Log Analytics Workspace

Tabla 16. Costos Mensuales y Anuales del Azure Monitor + Log Analytics Workspace

Componente	SKU/Tier	Volumen mensual	Costo mensual (USD)	Costo anual (USD)	Justificación
Log Analytics Ingestion	Pay-as-you-go	5 GB/mes	11.50	138.00	Logs plataforma
Retención	90 días	5 GB	0.50	6.00	Retención extendida
Alertas	Standard Tier	10/mes	0.20	2.40	Alertas consumo/fallas
Total Monitor/Logs			12.20	146.40	

Fuente: Pricing Calculator.

B) Costo total estimado (escenario recomendado SIN Azure Firewall)

Tabla 17. Costo total estimado Azure Nube

Servicio Azure	Costo mensual (USD)	Costo anual (USD)	% aprox.
Azure Database for PostgreSQL	259.41	3,112.92	68.2%
Azure Container Apps	104.03	1,248.30	27.3%
Azure Monitor + Log Analytics	12.20	146.40	3.2%
Azure Databricks	4.30	51.60	1.1%
Azure Data Lake Storage Gen2	0.48	5.76	0.1%
Azure Key Vault	0.03	0.36	<0.1%
Azure Static Web Apps	0.00	0.00	0.0%
TOTAL	380.45	4,565.40	100%

Fuente: Pricing Calculator.

C) Escenario con Azure Firewall (NO recomendado)

Evidencia real: Azure Firewall consumió ≈ 136.67 USD en el piloto (Cost Management). Esto demuestra que, de mantenerse activo, distorsiona por completo el costo mensual/anual del entorno frente al beneficio real para un caso de BI batch con bajo tráfico.

Decisión recomendada:

- Excluir Azure Firewall del diseño final.
- Mantener seguridad mediante:
 - VNet + NSG + Private Endpoints + Private DNS
 - Key Vault
 - Monitor/Log Analytics + Audit logs

D) Observaciones clave

- La nube ya entregó costos reales: los valores presentados en 1.4.2 provienen del Azure Portal (Cost Management) y su exportación a Excel; por tanto, son montos facturados por consumo real.
- Firewall fue el principal driver real de costos: en poco tiempo generó ≈ 136 USD, por lo que no es recomendable en este proyecto.
- Databricks es viable por patrón batch: al ejecutarse 24 veces al año (2 jobs/mes) y no como clúster permanente, el procesamiento se mantiene controlable.
- Viabilidad mensual y anual: bajo el escenario recomendado (sin Firewall), la proyección queda en ≈ 380.45 USD/mes y $\approx 4,565.40$ USD/año, con el mayor peso concentrado en base de datos y backend (PaaS), lo cual es coherente con una solución de BI con API y dashboard.

2. SERVICIOS DE NUBE DE LA SOLUCIÓN PROPUESTA

La solución propuesta se sustenta en una arquitectura nativa en la nube construida íntegramente sobre servicios de Microsoft Azure, diseñada para garantizar escalabilidad, seguridad, gobernanza y observabilidad.

Cada componente de la solución desde la ingesta de datos hasta la presentación del dashboard se implementa utilizando servicios PaaS y contenedores gestionados, lo que permite abstraer la complejidad de la infraestructura subyacente y enfocarse en el valor del negocio.

La arquitectura sigue un enfoque modular basado en capas (Bronze, Silver, Oro) y principios de seguridad como la red privada obligatoria, el principio de mínimo privilegio y la segmentación de redes. Asimismo, se integran mecanismos de monitoreo, respaldo y alertas proactivas para asegurar la continuidad operativa y el cumplimiento de los requisitos de robustez exigidos en entornos académicos y, potencialmente, productivos.

A continuación, se detalla el flujo de implementación paso a paso, organizado en torno a los servicios clave de Azure que conforman esta solución integral.

2.1. Asignar Roles AIM

2.1.1. Explicación del Uso

Antes de crear recursos en Azure, se definen claramente las responsabilidades de cada miembro del equipo y se asignan los roles mínimos necesarios (principio de mínimo privilegio) usando Azure RBAC.

En este entorno se trabaja con tres personas: Persona 1 (Leonardo), Persona 2 (Miguel) y Persona 3 (Alex), cada una con un ámbito de administración bien delimitado.

A) **Persona 1:** Propietario del despliegue de aplicaciones

Responsabilidades principales: Se encarga de todo lo relacionado con el despliegue y operación de las aplicaciones:

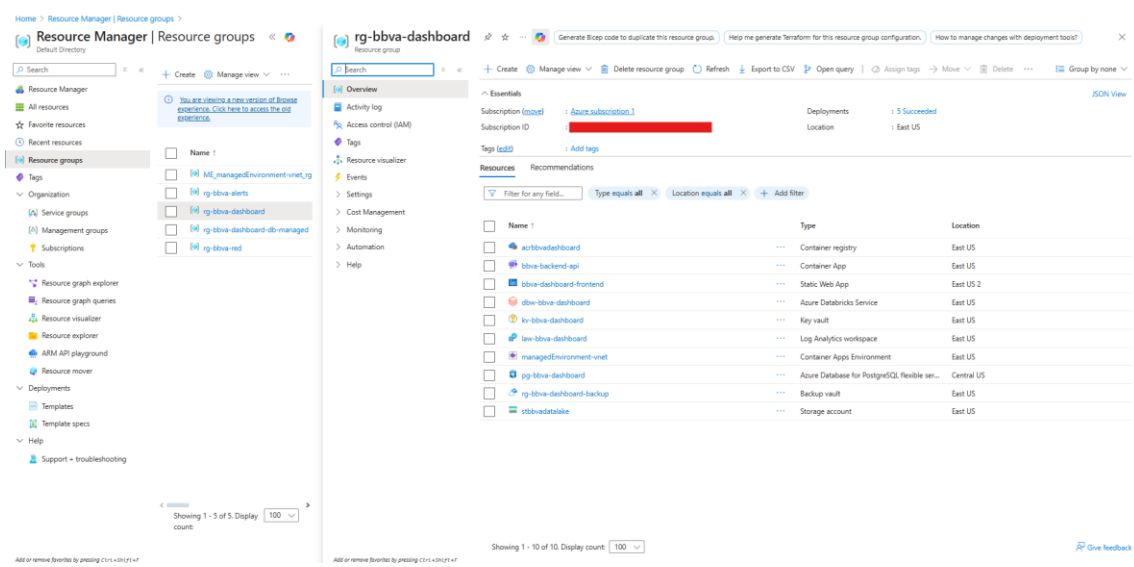
- Azure Container Registry: acrbbvadaashboard
- Backend: Container App bbva-backend-api
- Frontend: Static Web App bbva-dashboard-frontend
- Container Apps Environment: managedEnvironment-vnet
- Log Analytics Workspace: law-bbva-dashboard
- Objetivo: poder construir, versionar y desplegar el backend y frontend, además de revisar la observabilidad básica de la solución.

Tabla 18. Roles asignados a Persona 1

Recurso	Rol
Resource Group rg-bbva-dashboard	Contributor (base)
ACR acrbbvadaashboard	AcrPush, AcrPull
Container App bbva-backend-api	Contributor
Static Web App bbva-dashboard-frontend	Contributor
Container Apps Environment managedEnvironment-vnet	Container Apps Contributor
Log Analytics law-bbva-dashboard	Log Analytics Contributor

Fuente: *Elaboración propia.*

Figura 9. Recursos asignados a la Persona 1



Fuente: Portal de Azure.

B) Persona 2: Dueño de datos y analytics (Data & Security Engineer)

Responsabilidades principales: Administra todo lo que tiene que ver con datos, gobernanza y respaldo:

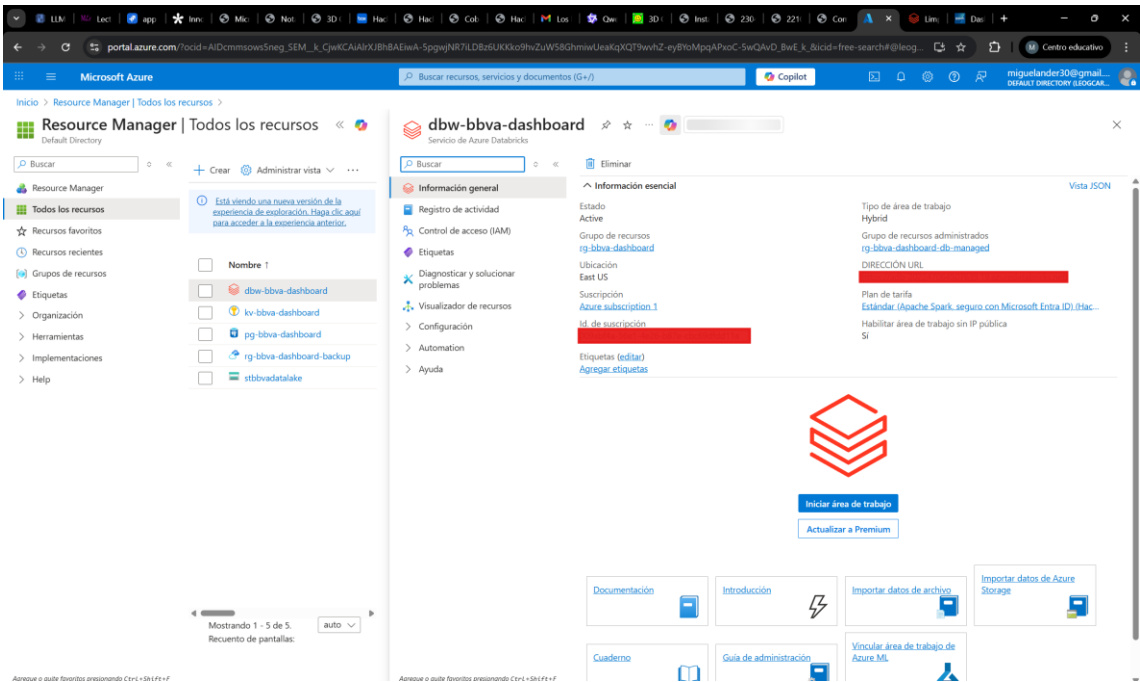
- Databricks Workspace: dbw-bbva-dashboard
- Key Vault: kv-bbva-dashboard
- PostgreSQL Flexible Server: pg-bbva-dashboard
- Backup Vault: rg-bbva-dashboard-backup
- Storage Account / Data Lake: stbbvadatalake (incluye niveles Bronze / Silver / Gold)

Tabla 19. Roles asignados a Personas 2

Recurso	Rol
Databricks Workspace	Owner
Key Vault kv-bbva-dashboard	Key Vault Secrets Officer
PostgreSQL Flexible Server	Contributor
Backup Vault	Backup Contributor
Storage Account stbbvadatalake	Storage Account Contributor
Blobs / Data Lake del Storage	Storage Blob Data Contributor

Fuente: Elaboración propia.

Figura 10. Recursos asignados a la Persona 2



Fuente: Portal de Azure.

C) Persona 3: Seguridad, redes y conectividad (Network & Firewall Admin)
Responsabilidades principales: Persona 3 es responsable de toda la capa de red y perímetro de seguridad:

- Virtual Network: vnet-bbva-dashboard
- Network Security Groups (NSGs) de todas las subnets
- Private Endpoints
- Private DNS Zones
- Firewall: fw-bbva-dashboard
- Firewall Policy: policy-bbva-firewall
- Route Table: rt-firewall
- Public IPs asociadas al firewall y otros componentes de red

Tabla 20. Roles asignados a Persona 3

Recurso	Rol
VNet completa	Network Contributor
NSGs	Network Contributor
Private Endpoints	Network Contributor
Private DNS Zones	Private DNS Zone Contributor
Public IPs	Network Contributor
Firewall	Network Contributor
Firewall Policy	Network Contributor
Route Table	Network Contributor

Fuente: Elaboración Propia.

Figura 11. Recursos asignados a la Persona 3

Nombre	Tipo	Grupo de recursos	Ubicación	Suscripción
fw-bbva-dashboard	Firewall	rg-bbva-red	East US	Azure subscription 1
nsg-containerapp	Grupo de seguridad de red	rg-bbva-red	East US	Azure subscription 1
nsg-databricks	Grupo de seguridad de red	rg-bbva-red	East US	Azure subscription 1
nsg-keyvault	Grupo de seguridad de red	rg-bbva-red	East US	Azure subscription 1
nsg-postgresql	Grupo de seguridad de red	rg-bbva-red	East US	Azure subscription 1
nsg-storage	Grupo de seguridad de red	rg-bbva-red	East US	Azure subscription 1
pe-acr	Punto de conexión privado	rg-bbva-red	East US	Azure subscription 1
pe-nic.6d2eb155-47cf-42d6-81a1-c410338567d8	Interfaz de red	rg-bbva-red	East US	Azure subscription 1
pe-keyvault	Punto de conexión privado	rg-bbva-red	East US	Azure subscription 1
pe-keyvault.nic.b5962fc2-fae-45a5-817e-361e905d86cf	Interfaz de red	rg-bbva-red	East US	Azure subscription 1

Fuente: *Portal de Azure.*

2.1.2. Comandos CLI

La implementación del modelo de gobernanza basado en roles descrito en la sección anterior se llevó a cabo mediante la Azure CLI, utilizando una secuencia automatizada de comandos que garantiza la reproducibilidad, trazabilidad y coherencia en la asignación de permisos. A continuación, se detalla el flujo de comandos ejecutados, organizado en tres fases: (1) obtención de identificadores de usuarios, (2) recuperación de identificadores de recursos, y (3) asignación de roles mediante Azure RBAC.

Primero, se listan los usuarios del directorio y se obtienen sus User Principal Names:

Paso 1: Identificación de usuarios

```
az ad user list --query "[].{name:displayName, upn:userPrincipalName}" -o table
```

Luego, se resuelven los Object IDs correspondientes:

```
$PERSONA1 = (az ad user show --id "UPN_DE_LA_PERSONA1" --query id -o tsv)
$PERSONA2 = (az ad user show --id "UPN_DE_LA_PERSONA2" --query id -o tsv)
$PERSONA3 = (az ad user show --id "UPN_DE_LA_PERSONA3" --query id -o tsv)
```

Paso 2: Obtención de identificadores de recursos

Para cada persona, se recuperan los identificadores únicos (IDs) de los recursos sobre los que se asignarán permisos.

Para la Persona 1:

```
$RG_ID = (az group show -n rg-bbva-dashboard --query id -o tsv)
```

```

$ACR_ID = (az acr show -n acrbbvadashboard -g rg-bbva-dashboard --query id -o tsv)
$BACKEND_ID = (az containerapp show -n bbva-backend-api -g rg-bbva-dashboard -
-query id -o tsv)
$FRONTEND_ID = (az staticwebapp show -n bbva-dashboard-frontend -g rg-bbva-
dashboard --query id -o tsv)
$ENV_ID = (az containerapp env show -n managedEnvironment-vnet -g rg-bbva-
dashboard --query id -o tsv)
$LAW_ID = (az monitor log-analytics workspace show -n law-bbva-dashboard -g
rg-bbva-dashboard --query id -o tsv)

```

Para la Persona 2:

```

$DBW_ID = (az databricks workspace show -n dbw-bbva-dashboard -g rg-bbva-
dashboard --query id -o tsv)
$KV_ID = (az keyvault show -n kv-bbva-dashboard --query id -o tsv)
$PG_ID = (az postgres flexible-server show -n pg-bbva-dashboard -g rg-bbva-
dashboard --query id -o tsv)
$BKP_ID = (az dataprotection backup-vault show -n rg-bbva-dashboard-backup -g
rg-bbva-dashboard --query id -o tsv)
$ST_ID = (az storage account show -n stbbvadalake -g rg-bbva-dashboard --
query id -o tsv)

```

Para la Persona 3:

```

$VNET_ID = (az network vnet show -n vnet-bbva-dashboard -g rg-bbva-red --
query id -o tsv)
$FW_ID = (az network firewall show -n fw-bbva-dashboard -g rg-bbva-red --
query id -o tsv)
$FW_PIP_ID = (az network public-ip show -n pip-firewall -g rg-bbva-red --
query id -o tsv)
$RT_ID = (az network route-table show -n rt-firewall -g rg-bbva-red --query
id -o tsv)
$FWPOL_ID = (az network firewall policy show -n policy-bbva-firewall -g rg-
bbva-red --query id -o tsv)
$DNS_ACR = (az network private-dns zone show -n privatelink.azurecr.io -g rg-
bbva-red --query id -o tsv)
$DNS_BLOB = (az network private-dns zone show -n
privatelink.blob.core.windows.net -g rg-bbva-red --query id -o tsv)
$DNS_DFS = (az network private-dns zone show -n
privatelink.dfs.core.windows.net -g rg-bbva-red --query id -o tsv)
$DNS_PG = (az network private-dns zone show -n
privatelink.postgres.database.azure.com -g rg-bbva-red --query id -o tsv)
$DNS_KV = (az network private-dns zone show -n
privatelink.vaultcore.azure.net -g rg-bbva-red --query id -o tsv)
$PE_ACR = (az network private-endpoint show -n pe-acr -g rg-bbva-red --query
id -o tsv)
$PE_KV = (az network private-endpoint show -n pe-keyvault -g rg-bbva-red --
query id -o tsv)
$PE_PG = (az network private-endpoint show -n pe-postgresql -g rg-bbva-red --
query id -o tsv)
$PE_BLOB = (az network private-endpoint show -n pe-storage-blob -g rg-bbva-
red --query id -o tsv)
$PE_DFS = (az network private-endpoint show -n pe-storage-dfs -g rg-bbva-red
--query id -o tsv)
$NIC_ACR = (az network nic show -n pe-acr.nic.6d2eb155-47cf-42d6-81a1-
c410338567d8 -g rg-bbva-red --query id -o tsv)
$NIC_KV = (az network nic show -n pe-keyvault.nic.b5962fc2-ffae-45a5-817e-
361e905d86cf -g rg-bbva-red --query id -o tsv)

```

```

$NIC_PG = (az network nic show -n pe-postgresql.nic.98fda535-3b45-4cbd-aa88-
bce0137ebc0c -g rg-bbva-red --query id -o tsv)
$NIC_BLOB = (az network nic show -n pe-storage-blob.nic.fe524dcc-3035-4f13-
a043-962b5e72edbd -g rg-bbva-red --query id -o tsv)
$NIC_DFS = (az network nic show -n pe-storage-dfs.nic.dff6e04d-160d-4a36-
8273-4fc845f54b7a -g rg-bbva-red --query id -o tsv)

$NSG_CA = (az network nsg show -n nsg-containerapp -g rg-bbva-red --query id
-o tsv)
$NSG_DB = (az network nsg show -n nsg-databricks -g rg-bbva-red --query id -o
tsv)
$NSG_KV = (az network nsg show -n nsg-keyvault -g rg-bbva-red --query id -o
tsv)
$NSG_PG = (az network nsg show -n nsg-postgresql -g rg-bbva-red --query id -o
tsv)
$NSG_ST = (az network nsg show -n nsg-storage -g rg-bbva-red --query id -o
tsv)

```

Paso 3: Asignación de roles RBAC

Finalmente, se ejecutan los comandos de asignación de roles, respetando los principios de mínimo privilegio y separación de responsabilidades.

Para Persona 1:

```

az role assignment create --assignee $PERSONA1 --role "Contributor"
--scope $RG_ID
az role assignment create --assignee $PERSONA1 --role "AcrPull"
--scope $ACR_ID
az role assignment create --assignee $PERSONA1 --role "AcrPush"
--scope $ACR_ID
az role assignment create --assignee $PERSONA1 --role "Container Apps
Contributor" --scope $ENV_ID
az role assignment create --assignee $PERSONA1 --role "Contributor"
--scope $BACKEND_ID
az role assignment create --assignee $PERSONA1 --role "Contributor"
--scope $FRONTEND_ID
az role assignment create --assignee $PERSONA1 --role "Log Analytics
Contributor" --scope $LAW_ID

```

Para Persona 2:

```

az role assignment create --assignee $PERSONA2 --role "Owner"
--scope $DBW_ID
az role assignment create --assignee $PERSONA2 --role "Key Vault Secrets
Officer" --scope $KV_ID
az role assignment create --assignee $PERSONA2 --role "Contributor"
--scope $PG_ID
az role assignment create --assignee $PERSONA2 --role "Backup Contributor"
--scope $BKP_ID
az role assignment create --assignee $PERSONA2 --role "Storage Account
Contributor" --scope $ST_ID
az role assignment create --assignee $PERSONA2 --role "Storage Blob Data
Contributor" --scope $ST_ID

```

Para Persona 3:

```

az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $VNET_ID
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $FW_ID
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $RT_ID
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $FWPOL_ID
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $FW_PIP_ID

az role assignment create --assignee $PERSONA3 --role "Private DNS Zone
Contributor" --scope $DNS_ACR
az role assignment create --assignee $PERSONA3 --role "Private DNS Zone
Contributor" --scope $DNS_BLOB
az role assignment create --assignee $PERSONA3 --role "Private DNS Zone
Contributor" --scope $DNS_DFS
az role assignment create --assignee $PERSONA3 --role "Private DNS Zone
Contributor" --scope $DNS_PG
az role assignment create --assignee $PERSONA3 --role "Private DNS Zone
Contributor" --scope $DNS_KV

az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $PE_ACR
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $PE_KV
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $PE_PG
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $PE_BLOB
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $PE_DFS

az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NIC_ACR
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NIC_KV
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NIC_PG
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NIC_BLOB
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NIC_DFS

az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NSG_CA
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NSG_DB
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NSG_KV
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NSG_PG
az role assignment create --assignee $PERSONA3 --role "Network Contributor" -
-scope $NSG_ST

```

2.2. Crear los Resource Group

2.2.1. Explicación del Uso

Como primer paso se definieron tres grupos de recursos para separar responsabilidades y facilitar la gobernanza:

A) rg-bbva-alerts

Propósito: Concentrar recursos de monitoreo y alertas. Recursos incluidos:

- Log Analytics
- Azure Monitor
- Action Groups

B) rg-bbva-dashboard

Propósito: Agrupar servicios directamente relacionados al dashboard. Recursos incluidos:

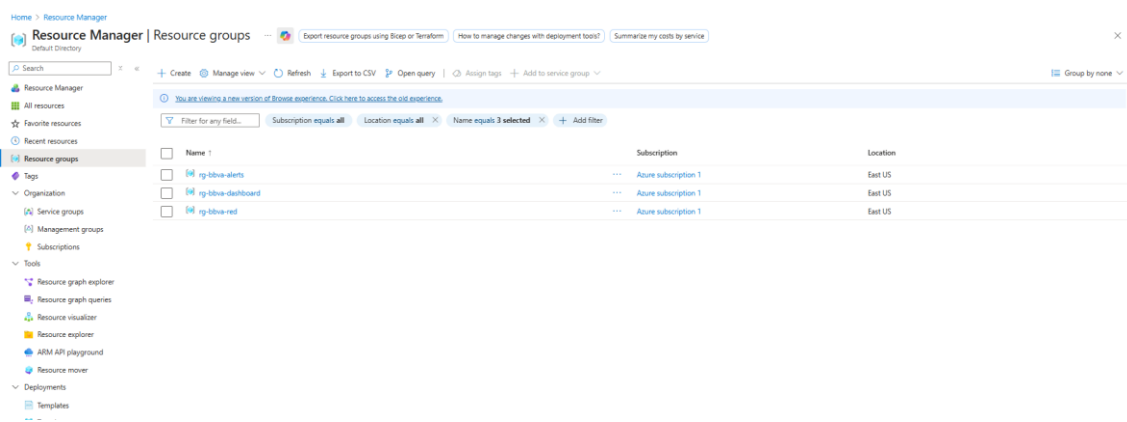
- Storage Account del Data Lake
- Servidor PostgreSQL flexible
- Workspace de Databricks

C) rg-bbva-red

Propósito: Reservado para componentes de red y seguridad. Recursos incluidos:

- Virtual Network y subredes privadas
- Network Security Groups (NSG)
- Private Endpoints
- Azure Firewall
- Recursos de gobernanza (Private DNS Zone, Recovery Services Vault)

Figura 12. Estructura de los recursos creados



Fuente: Elaboración Propia.

2.2.2. Comandos CLI

La implementación de la infraestructura es la creación del grupo de recursos que contendrá todos los servicios del entorno:

```
az group create --name rg-bbva-dashboard --location eastus
az group create --name rg-bbva-red --location eastus
az group create --name rg-bbva-alerts --location eastus
```

2.3. Crear Key Vault

2.3.1. Explicación del Uso

Se creó el recurso kv-bbva-dashboard dentro del grupo de recursos rg-bbva-dashboard. Propósito: Centralizar todos los secretos sensibles utilizados por la solución.

Secretos almacenados:

- Cadenas de conexión hacia PostgreSQL Flexible Server
- Claves de acceso del Storage Account
- Credenciales de service principal o identidades administradas que consumen el Data Lake

Configuración destacada

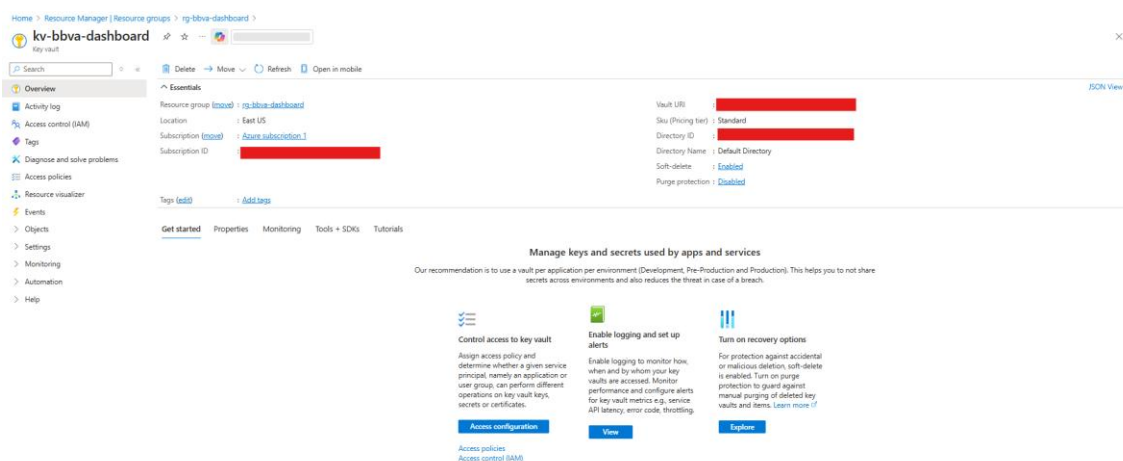
- SKU: Standard (suficiente para escenarios internos y académicos)
- Soft delete: Habilitado (protege contra eliminaciones accidentales)
- Control de acceso: Integración con Access Policies / IAM

Políticas de acceso

El acceso está limitado únicamente a:

- Identidades de Databricks
- Identidades del backend / scripts de administración
- Usuarios administradores del entorno

Figura 13. Vista general del Azure Key Vault kv-bbva-dashboard en Azure Portal



Fuente: Portal de Azure.

2.3.2. Comandos CLI

- Registrar el provider

```
az provider register --namespace Microsoft.KeyVault
```

- Crear el vault

```
az keyvault create --name kv-bbva-dashboard --resource-group rg-bbva-dashboard --location eastus
```

- Configurar la policy

```
$USER_OBJECT_ID = az ad signed-in-user show --query id -o tsv
```

```
$KV_ID = az keyvault show --name kv-bbva-dashboard --query id -o tsv
```

```
az role assignment create --assignee $USER_OBJECT_ID --role "Key Vault Secrets Officer" --scope $KV_ID
```

2.4. Crear el Storage Account

2.4.1. Explicación del Uso

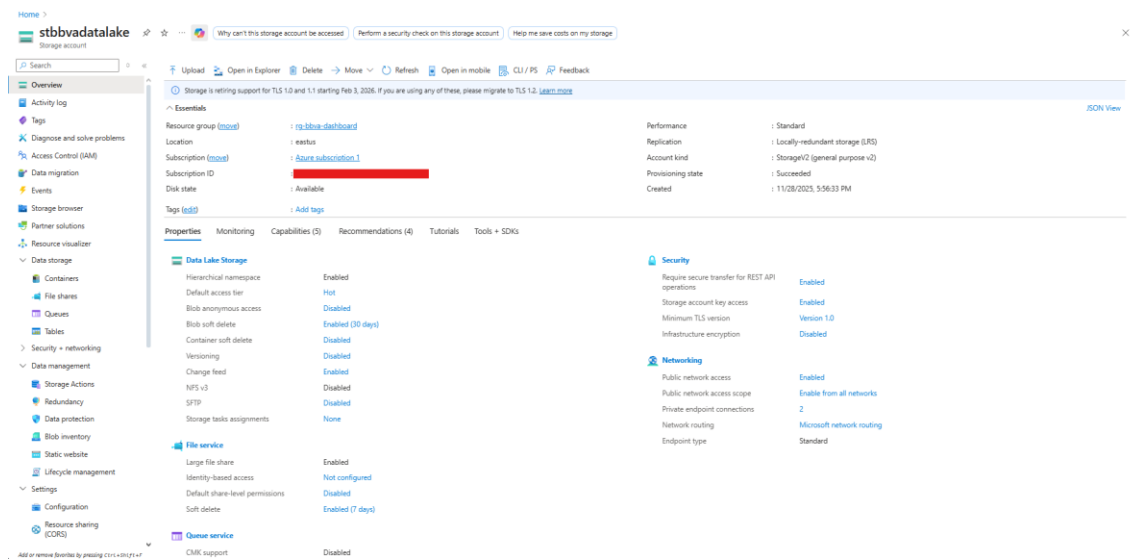
Para implementar el Data Lake se creó el Storage Account stbbvadalake con las siguientes características:

- Tipo: StorageV2 (general purpose v2)
- Hierarchical namespace: Enabled, para habilitar Azure Data Lake Storage Gen2
- Replicación: LRS (Locally Redundant Storage), suficiente para el ámbito académico del proyecto
- Access tier: Hot, ya que los datos se leen y escriben en cada ciclo de carga mensual

Acceso seguro mediante:

- Require secure transfer for REST API operations = Enabled
- Integración con Private Endpoints (se observan conexiones privadas configuradas en el panel de Networking)

Figura 14. Vista del recurso Storage Account



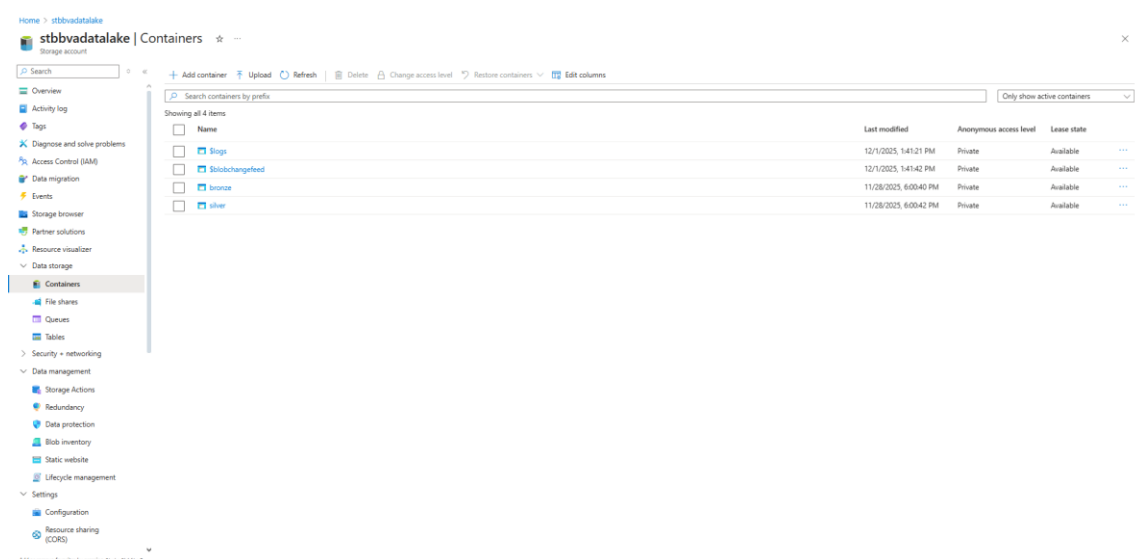
Fuente: *Portal de Azure.*

Dentro del Storage Account se definieron varios contenedores:

- bronze: capa de data sucia / raw, donde se almacenan los CSV tal como llegan
- silver: capa de data limpia / transformada, en formato parquet u otro formato optimizado tras el ETL
- slogs y \$blobchangefeed: contenedores técnicos para logs/cambios de blobs (propios del servicio)

Esta estructura respeta la separación raw/trusted y facilita el control de acceso por capa.

Figura 15. Creación de los contenedores en Storage Account



Fuente: *Portal de Azure.*

A) Estructura Bronze – Data Sucia

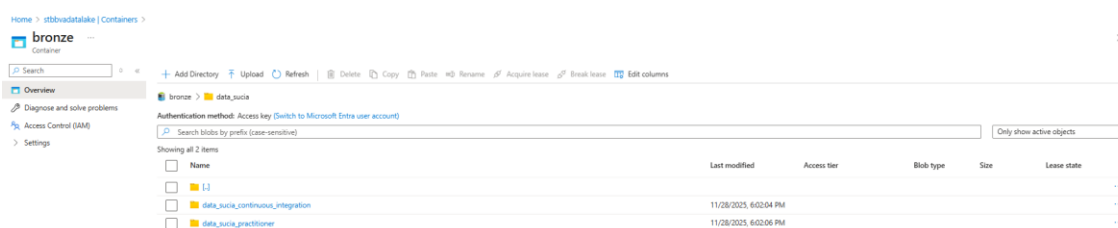
En el contenedor bronze se creó una carpeta lógica data_sucia que separa los archivos según el dominio funcional:

- data_sucia_practitioner
- data_sucia_continuous_integration

En estas ubicaciones se cargan los archivos CSV exportados desde el Marco Playbook (o sus equivalentes de prueba en este entorno).

Cada subcarpeta representa el “raw zone” de su dominio, sin transformaciones ni limpieza aplicada.

Figura 16. Estructura de la capa de bronce



Fuente: *Portal de Azure.*

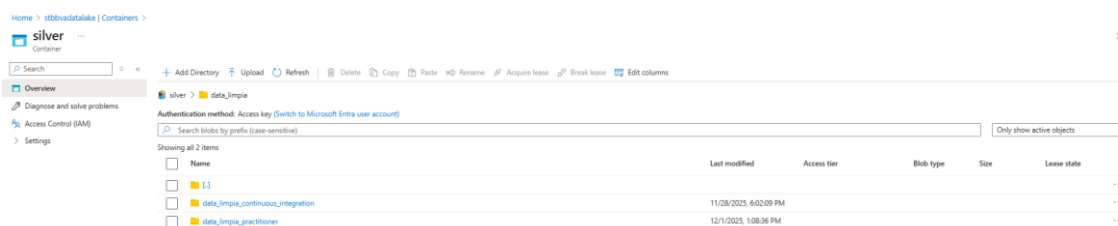
B) Estructura Silver – Data Limpia

En el contenedor silver se creó la carpeta data_limpia, también separada por dominio:

- data_limpia_practitioner
- data_limpia_continuous_integration

Aquí Databricks escribe los resultados de las transformaciones: datos limpios, tipificados y listos para ser cargados en la capa Oro (PostgreSQL).

Esta separación permite trazar claramente qué dataset se encuentra en qué nivel de calidad.

Figura 17. Estructura de la capa de plata

Fuente: *Portal de Azure.*

2.4.2. Comandos CLI

```
az storage account create --name stbbvadalake --resource-group rg-bbva-
dashboard --location eastus --sku Standard_LRS `
  --kind StorageV2 --hns true
```

Se crean los contenedores Bronze y Silver

```
az storage container create --account-name stbbvadalake --name bronze --
auth-mode login
az storage container create --account-name stbbvadalake --name silver --
auth-mode login
```

Se crean las carpetas Data Sucia y Data Limpia

```
az storage fs directory create --account-name stbbvadalake --file-system
bronze `
  --name data_sucia --auth-mode login
az storage fs directory create --account-name stbbvadalake --file-system
bronze `
  --name data_sucia/data_sucia_continuous_integration --auth-mode login
az storage fs directory create --account-name stbbvadalake --file-system
bronze `
  --name data_sucia/data_sucia_practitioner --auth-mode login
az storage fs directory create --account-name stbbvadalake --file-system
silver `
  --name data_limpiar --auth-mode login
az storage fs directory create --account-name stbbvadalake --file-system
silver `
  --name data_limpiar/data_limpiar_continuous_integration --auth-mode login
az storage fs directory create --account-name stbbvadalake --file-system
silver `
  --name data_limpiar/data_limpiar_practitioner --auth-mode login
```

Guardar Connection String en Key Vault

```
$AZURE_STORAGE_CONNECTION_STRING = az storage account show-connection-string
--name stbbvadalake `
  --resource-group rg-bbva-dashboard --query connectionString -o tsv

az keyvault secret set --vault-name kv-bbva-dashboard --name "azure-storage-
connection-string" `
  --value "$AZURE_STORAGE_CONNECTION_STRING"
```

2.5. Crear PostgreSQL Flexible Server

2.5.1. Explicación del Uso

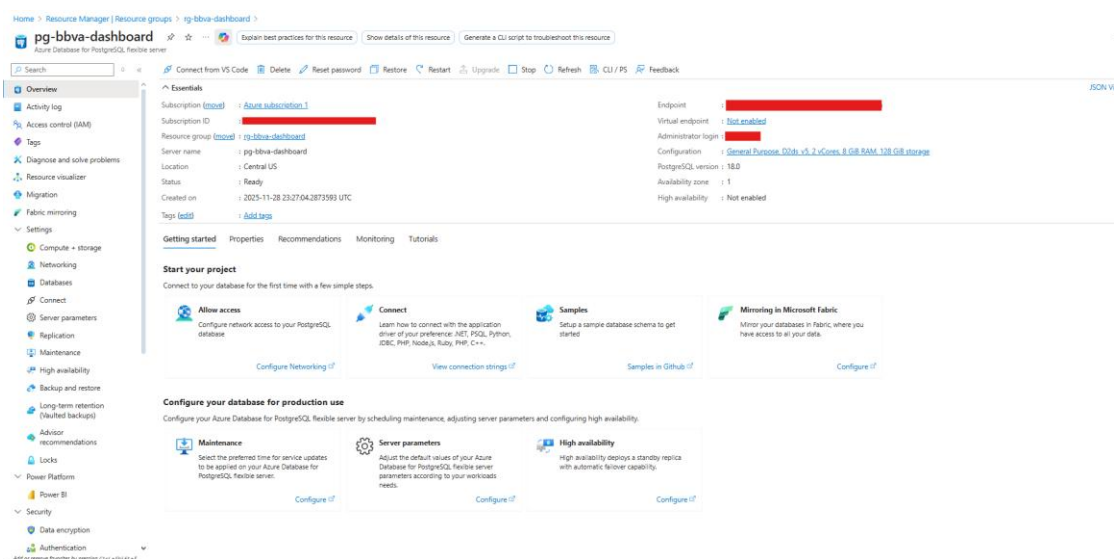
Para la capa de datos Oro se optó por Azure Database for PostgreSQL – Flexible Server, alineado con el motor usado en el entorno local.

El servidor creado es:

- Nombre: pg-bbva-dashboard
- Ubicación: Central US
- Tipo: General Purpose, tamaño D2ds v5 (2 vCores, 8 GB RAM, 128 GB storage)
- Acceso restringido mediante reglas de red y Private Endpoints (conectado a la VNet de rg-bbva-red)
- Alta disponibilidad deshabilitada en este entorno académico (pero documentada como recomendación para entornos productivos)

Este servidor se utiliza como Data Warehouse relacional donde se modela la capa Oro con esquemas en estrella para Practitioner y CI.

Figura 18. Vista del servicio creado en PostgreSQL



Fuente: Portal de Azure.

Estructura Gold

Dentro del servidor pg-bbva-dashboard se crearon las bases de datos de la capa Oro:

- data_oro_practitioner
- data_oro_ci

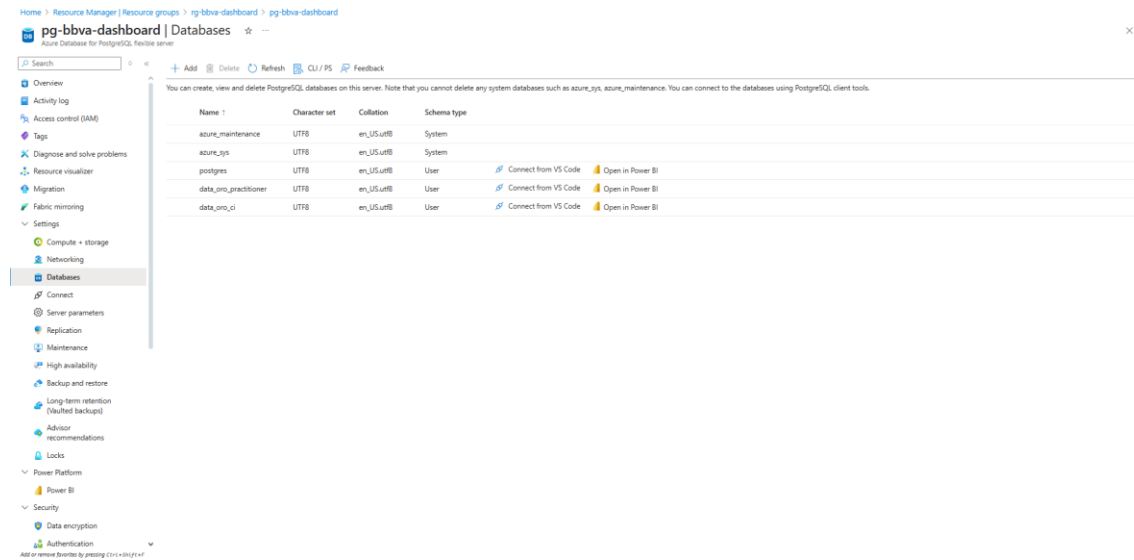
Cada una contiene:

- Tablas de hechos y dimensiones necesarias para los KPIs del dominio
- Índices y claves foráneas que soportan las consultas del dashboard

- Esquemas diseñados para facilitar las consultas analíticas desde el backend y, si se necesitara, desde herramientas externas de BI

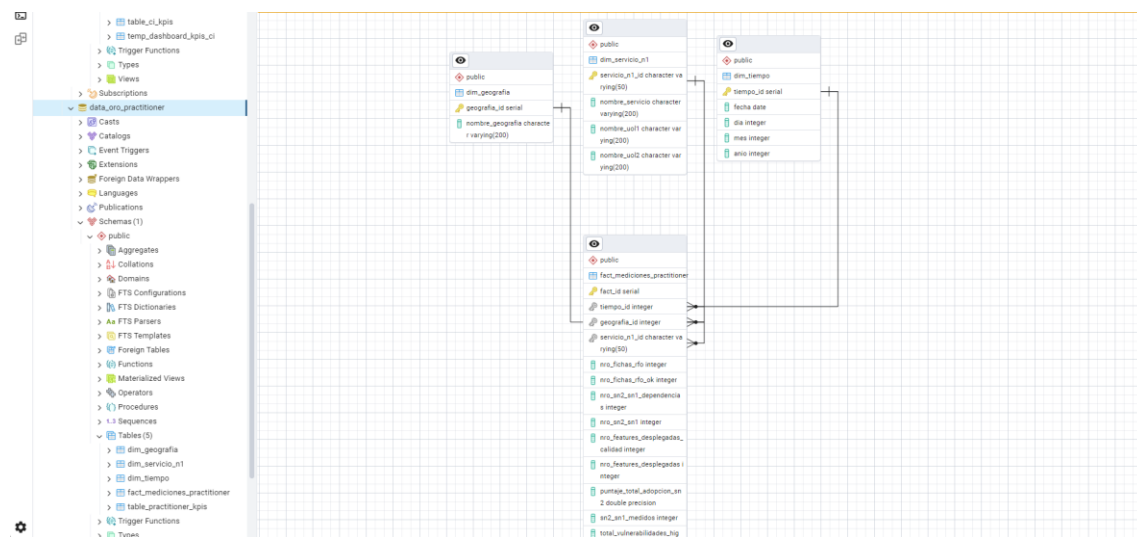
Estas bases de datos son el destino final del ETL ejecutado en Databricks.

Figura 19. Estructura de la capa de Oro



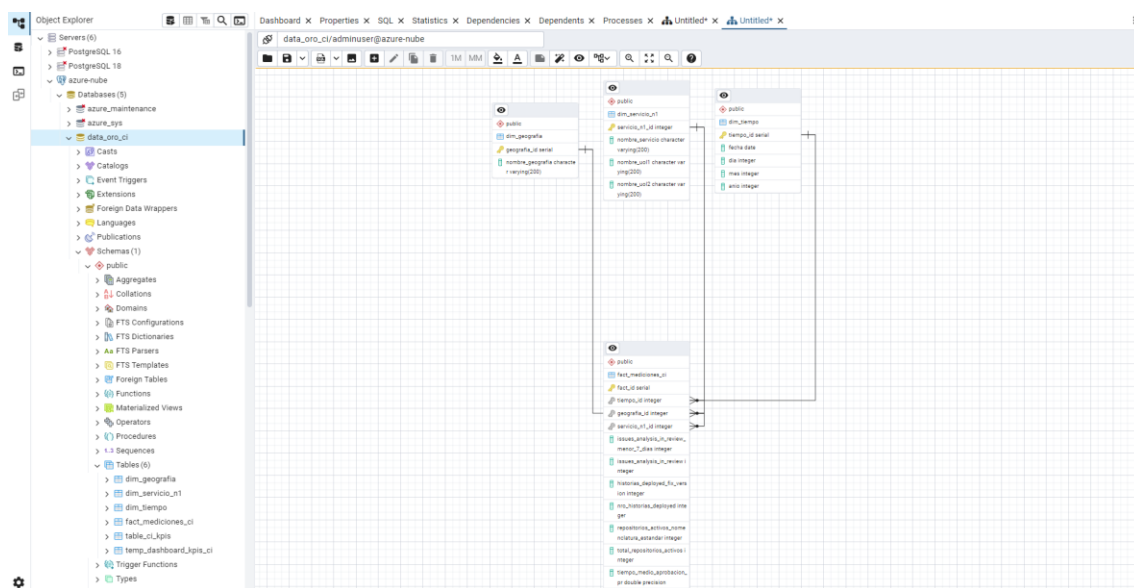
Fuente: *Portal de Azure.*

Figura 20. Diagrama relacional de base datos practitioner



Fuente: *PostgreSQL.*

Figura 21. Diagrama relacional de base datos continuous integration



Fuente: PostgreSQL.

2.5.2. Comandos CLI

```
az postgres flexible-server create --name pg-bbva-dashboard --resource-group
rg-bbva-dashboard --location centralus `
    --admin-user adminuser --admin-password "SecurePass123!" --sku-name
Standard_D2ds_v5 --tier GeneralPurpose `
    --storage-size 128 --version 18 --public-access all
```

Guardar credenciales de PostgreSQL en Key Vault

```
az keyvault secret set --vault-name kv-bbva-dashboard --name "db-host" --
value "pg-bbva-dashboard.postgres.database.azure.com"
az keyvault secret set --vault-name kv-bbva-dashboard --name "db-port" --
value "5432"
az keyvault secret set --vault-name kv-bbva-dashboard --name "db-user" --
value "adminuser"
az keyvault secret set --vault-name kv-bbva-dashboard --name "db-password" --
value "SecurePass123!"
az keyvault secret set --vault-name kv-bbva-dashboard --name "db-name-
practitioner" --value "data_oro_practitioner"
az keyvault secret set --vault-name kv-bbva-dashboard --name "db-name-ci" --
value "data_oro_ci"
```

Crear la bases de datos data_oro_practitioner y data_oro_ci

```
az postgres flexible-server db create --resource-group rg-bbva-dashboard --
server-name pg-bbva-dashboard `
    --database-name data_oro_practitioner
az postgres flexible-server db create --resource-group rg-bbva-dashboard --
server-name pg-bbva-dashboard `
    --database-name data_oro_ci
```

Crear las tablas practitioner (Ingresar la contraseña: SecurePass123!)

```
psql `
-h pg-bbva-dashboard.postgres.database.azure.com `
-p 5432 `
-U adminuser `
-d data_oro_practitioner `
-f "$env:USERPROFILE\Desktop\Querys\query_practitioner.sql" `
--set=sslmode=require
```

Crear las tablas continuous integration (Ingresar la contraseña: SecurePass123!)

```
psql `
-h pg-bbva-dashboard.postgres.database.azure.com `
-p 5432 `
-U adminuser `
-d data_oro_ci `
-f "$env:USERPROFILE\Desktop\Querys\query_ci.sql" `
--set=sslmode=require
```

2.6. Crear Databricks

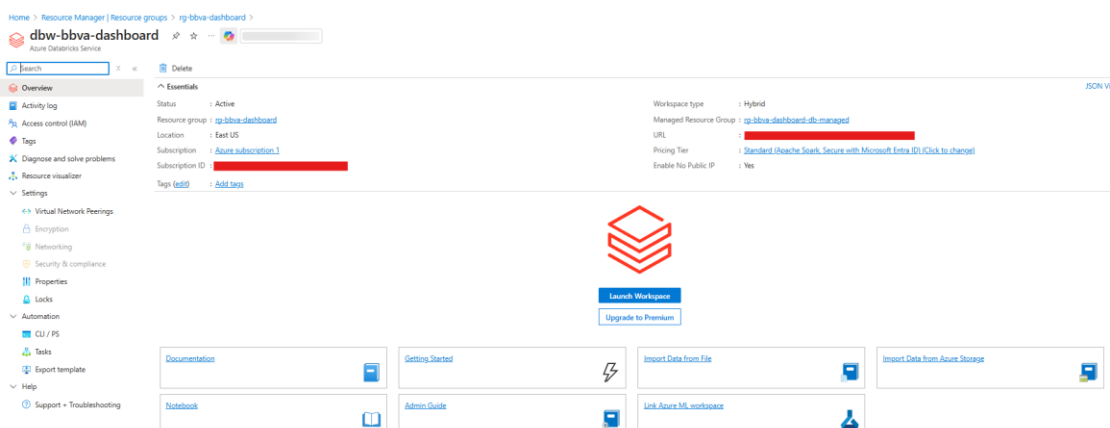
2.6.1. Explicación del Uso

Para el procesamiento distribuido se aprovisionó un workspace de Azure Databricks:

- Nombre: dbw-bbva-dashboard
- Resource group: rg-bbva-dashboard
- Tipo de workspace: Hybrid
- Enable No Public IP: Yes, lo que obliga a consumir Databricks a través de la red privada y endpoints seguros, alineado con los requisitos de seguridad del proyecto

Este workspace es el punto central para desarrollar notebooks PySpark, ejecutar el ETL y orquestar los jobs.

Figura 22. Vista general del servicio Databricks



Fuente: Portal de Azure.

A) Creación del clúster de Databricks

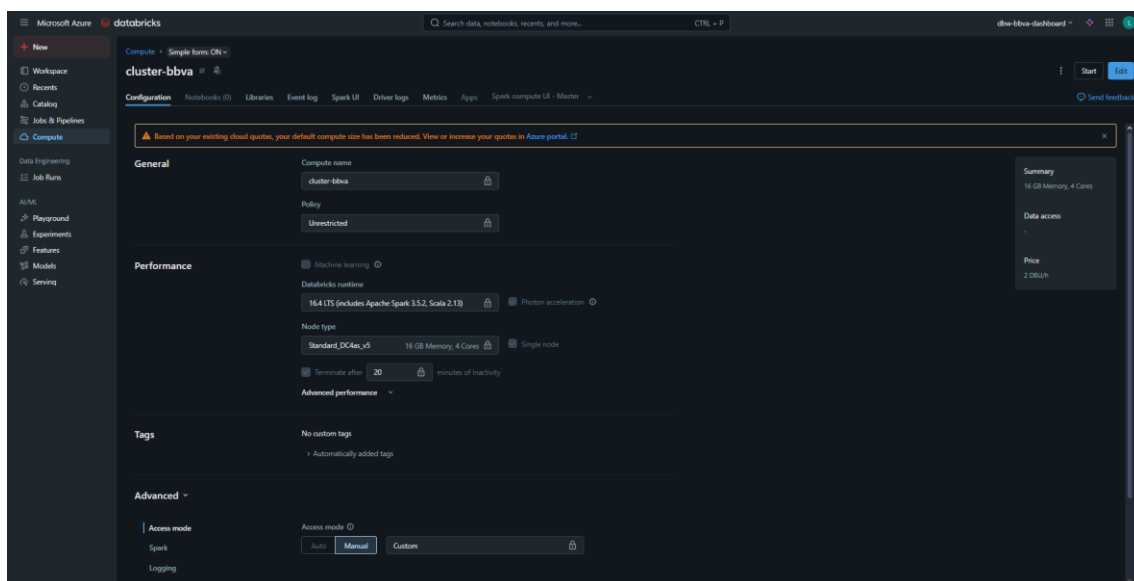
Dentro del workspace se configuró el clúster cluster-bbva, con las siguientes características:

- Runtime: Databricks 16.4 LTS (incluye Apache Spark 3.5.2).
- Tipo de nodo: Standard_D4as_v5 (16 GB Memory, 4 Cores).
- Modo: Single node (suficiente para los volúmenes actuales).
- Auto-termination: 20 minutos de inactividad, para evitar costos innecesarios.
- Access mode: Custom, permitiendo ajustar permisos de acceso a datos según las necesidades del ETL.

Este clúster es el que ejecuta los notebooks de limpieza y transformación que mueven la información desde:

- Bronze → Silver en el Data Lake.
- Silver → Oro en PostgreSQL Flexible Server.

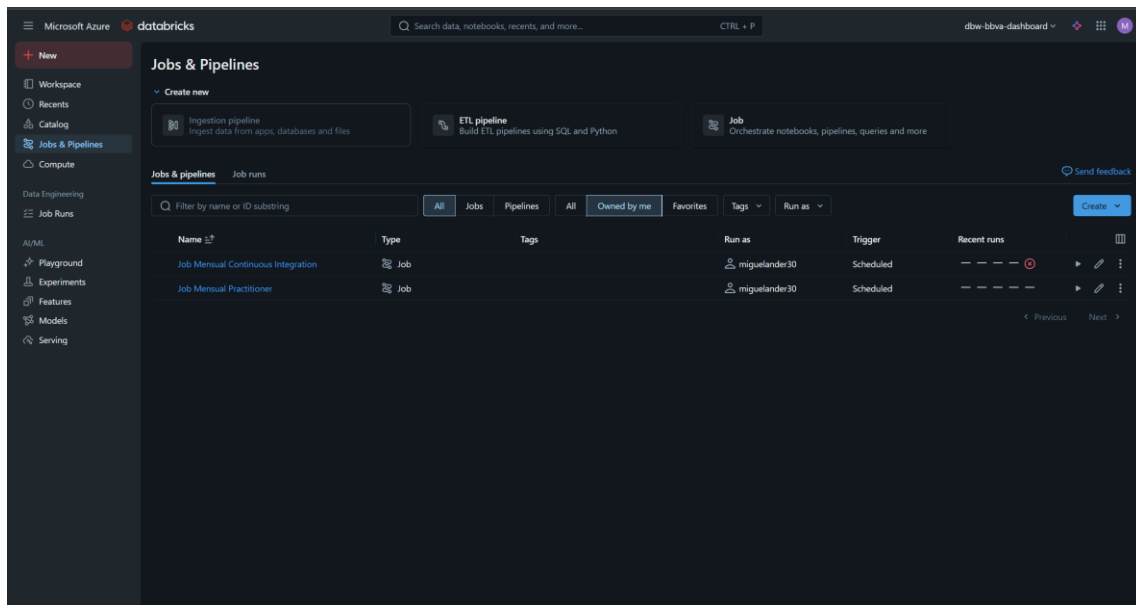
Figura 23. Vista general de dentro del workspace



Fuente: Databricks.

B) Creación de Jobs de ETL

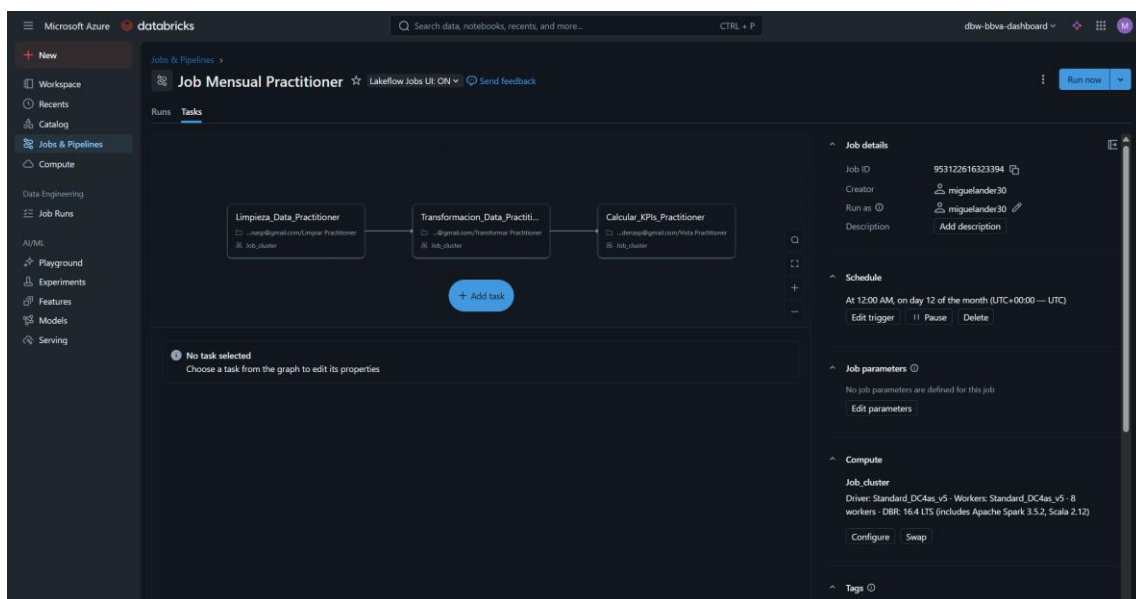
Sobre el clúster cluster-bbva se diseñaron dos Jobs principales (ilustrativos en este entorno académico):

Figura 24. Vista General de los Jobs

Fuente: *Databricks.*

a) Job ETL Practitioner

- Lee los CSV crudos desde bronze/data_sucia_practitioner.
- Aplica limpieza, tipificación y joins necesarios.
- Escribe los datos limpios en silver/data_limpia_practitioner.
- Finalmente carga la capa Oro en la base de datos data_oro_practitioner (tablas de hechos y dimensiones).
- Ejecución automática una vez al mes.

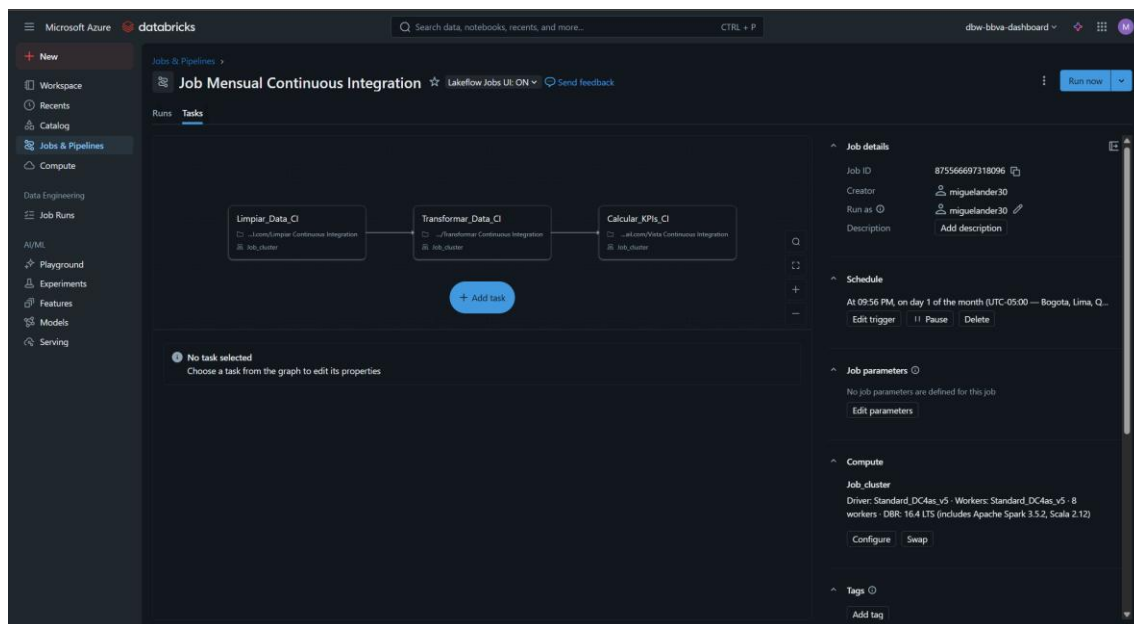
Figura 25. Tareas del Job del Practitioner

Fuente: *Databricks.*

b) Job ETL Continuous Integration (CI)

- Mismo patrón que el anterior, pero usando:
 - bronze/data_sucia_continuous_integration
 - silver/data_limpiar_continuous_integration
 - Base de datos data_oro_ci como destino final
- Ejecución automática una vez al mes.

Figura 26. Tareas del Job del Continuous Integration



Fuente: Databricks.

C) Subir los notebooks a Databricks:

Una vez creado el workspace y el clúster de Databricks, se suben los notebooks que implementan el ETL completo sobre las tablas de Practitioner y Continuous Integration. En la siguiente captura se observa la carpeta de trabajo del usuario con los seis notebooks creados:

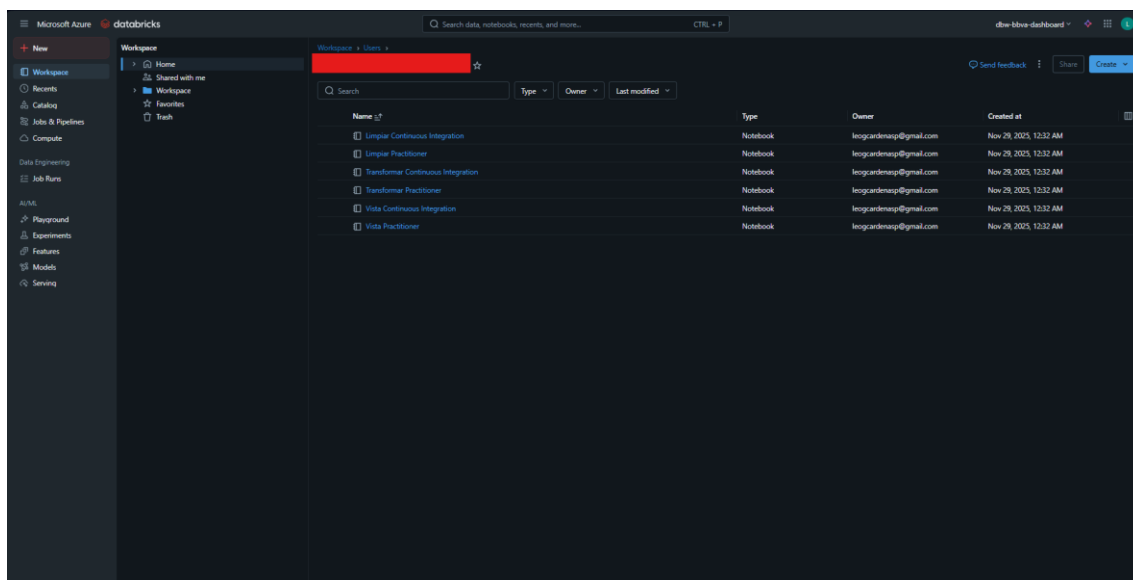
- Limpiar Practitioner
- Limpiar Continuous Integration
- Transformar Practitioner
- Transformar Continuous Integration
- Vista Practitioner
- Vista Continuous Integration

Los notebooks siguen el patrón Medallion:

- Los notebooks de Limpieza leen los CSV ubicados en la capa Bronze y escriben datos tipificados y depurados en la capa Silver.

- Los notebooks de **Transformación** enriquecen la información Silver (joins, derivación de campos, normalización) y preparan las tablas de Gold que luego se cargan en PostgreSQL.
- Los notebooks de **Vista** generan las vistas de negocio (KPIs) que se consultan desde el backend.

Figura 27. Vista General de la vista los notebooks



Fuente: *Databricks*.

2.6.2. Comandos CLI

```
az databricks workspace create --name dbw-bbva-dashboard --resource-group rg-bbva-dashboard \
  --location eastus --sku standard --managed-resource-group rg-bbva-dashboard-db-managed --public-network-access Enabled
```

Obtener acceso admin (Borrar Cache si es necesario)

```
az role assignment create \
  --role "Owner" \
  --assignee $(az ad signed-in-user show --query id -o tsv) \
  --scope $(az databricks workspace show --name dbw-bbva-dashboard --resource-group rg-bbva-dashboard --query id -o tsv)
```

Obtener la Key del Datalake

```
$DL_KEY = az storage account keys list --account-name stbbvadatalake --resource-group rg-bbva-dashboard --query "[0].value" \
  -o tsv
```

Guardar la Key del Datalake en Key Vault

```
az keyvault secret set --vault-name kv-bbva-dashboard --name "datalake-key" -value "$DL_KEY"
```

Crear el cluster - Obtener el Token (User Settings → Generate Token)

```
$DATABRICKS_TOKEN = "PONER AQUI EL TOKEN"

$WORKSPACE_URL = az databricks workspace show --name dbw-bbva-dashboard --
resource-group rg-bbva-dashboard `
    --query workspaceUrl -o tsv

$headers = @{
    "Authorization" = "Bearer $DATABRICKS_TOKEN"
    "Content-Type" = "application/json"
}

$body = @{
    cluster_name = "cluster-bbva"
    spark_version = "15.4.x-scala2.12"
    node_type_id = "Standard_DS3_v2"
    num_workers = 0
    autotermination_minutes = 20
    custom_tags = @{
        "ResourceClass" = "SingleNode"
    }
    enable_photon = $true
    runtime_engine = "PHOTON"
    spark_conf = @{
        "fs.azure.account.key.stbbvadalake.dfs.core.windows.net" = $DL_KEY
    }
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
    -Uri "https://$WORKSPACE_URL/api/2.1/clusters/create" `
    -Method Post `
    -Headers $headers `
    -Body $body
```

Guardar credenciales de Databricks en Key Vault

```
az keyvault secret set --vault-name kv-bbva-dashboard --name "databricks-
token" --value "$DATABRICKS_TOKEN"
az keyvault secret set --vault-name kv-bbva-dashboard --name "databricks-
workspace-url" --value "https://$WORKSPACE_URL"
```

Configurar Databricks CLI

```
$configContent = @"
[DEFAULT]
host = https://$WORKSPACE_URL
token = $DATABRICKS_TOKEN
"@

[System.IO.File]::WriteAllText("$env:USERPROFILE\.databrickscfg",
$configContent, [System.Text.Encoding]::ASCII)
```

Subir los Notebooks (En este caso estan en el escritorio en una carpeta Notebooks)

Publicar la imagen del backend en el Registry

A partir del código del backend Flask se construye una imagen Docker y se publica en el ACR.

En la captura del repositorio bbva-backend se ve la etiqueta v1, que corresponde a la primera versión estable del servicio:

La imagen incluye:

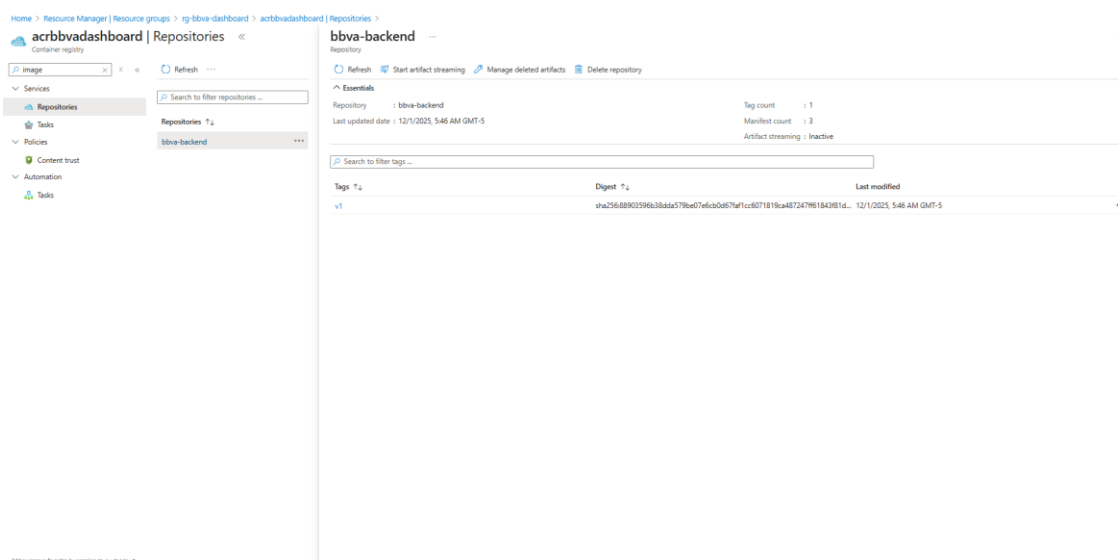
- Código Flask de la API
- Dependencias Python (requirements.txt)
- Configuración de Gunicorn para servir la aplicación

El tag v1 permite gestionar versiones posteriores (v2, v3, etc.) y facilitar rollbacks en caso de errores de despliegue.

El flujo típico es:

- docker build de la imagen del backend
- docker tag apuntando al login server del ACR
- docker push para publicar la versión en el registro

Figura 29. Vista general del servicio Registry



Fuente: *Portal de Azure.*

2.7.2. Comandos CLI

```
az acr create --name acrbbvadaashboard --resource-group rg-bbva-dashboards --location eastus --sku Standard --admin-enabled true
```

Guardar credenciales de ACR en Key Vault

```
$ACR_PASSWORD = az acr credential show --name acrbbvadashboard --query
"passwords[0].value" -o tsv
```

```
az keyvault secret set --vault-name kv-bbva-dashboard --name "acr-password" -
-value "$ACR_PASSWORD"
az keyvault secret set --vault-name kv-bbva-dashboard --name "acr-username" -
-value "acrbbvadashboard"
az keyvault secret set --vault-name kv-bbva-dashboard --name "acr-server" --
value "acrbbvadashboard.azurecr.io"
```

Abrir una terminal en la carpeta del backend para:

- Construir la imagen desde tu Dockerfile

```
docker build -t bbva-backend:v1 .
```

- Loguearse al ACR

```
az acr login --name acrbbvadashboard
```

- Taguear la imagen con el ACR

```
docker tag bbva-backend:v1 acrbbvadashboard.azurecr.io/bbva-backend:v1
```

- Subirla al ACR

```
docker push acrbbvadashboard.azurecr.io/bbva-backend:v1
```

2.8. Crear Container Apps Environment

2.8.1. Explicación del Uso

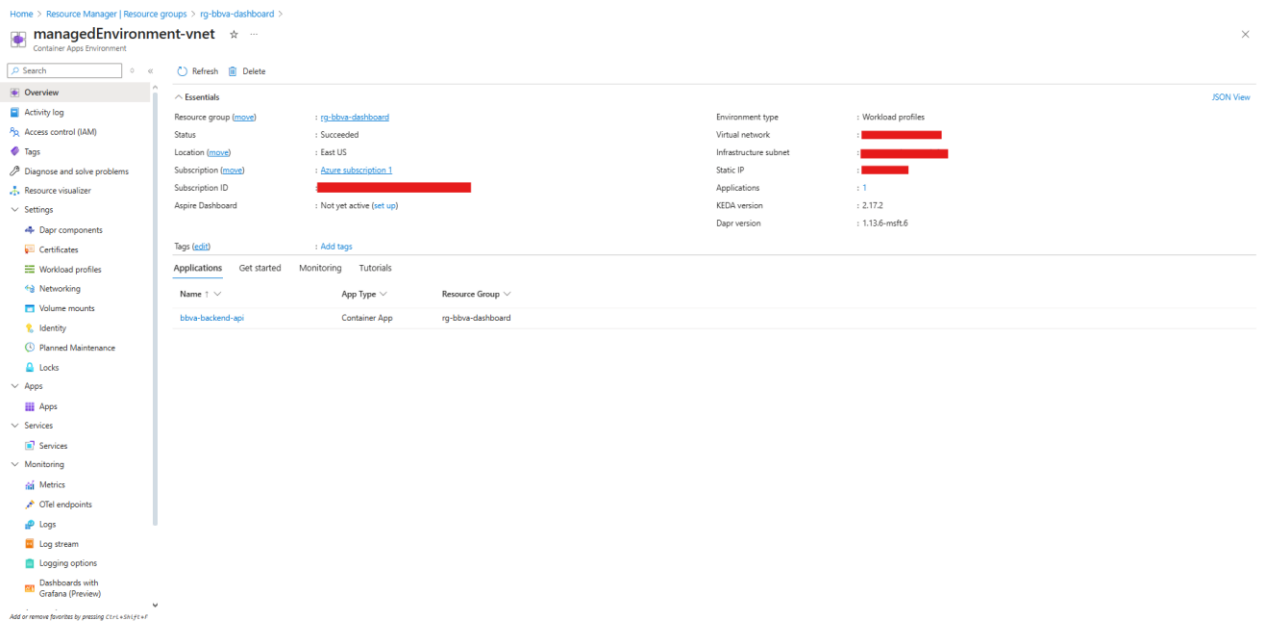
Como capa de ejecución para el backend se despliega un Azure Container Apps Environment asociado a una VNet (managedEnvironment-vnet).

En la captura se observa:

- El entorno ejecutándose en la región East US
- La integración con la red virtual y la subred de infraestructura, lo que permite:
 - Exponer el backend de forma controlada
 - Conectarlo de manera segura a PostgreSQL y al Storage Account
- Un contenedor de aplicaciones donde, para este laboratorio, se despliega una sola API: bbva-backend-api

Este entorno funcionará como “cluster lógico” para todas las Container Apps relacionadas con el dashboard.

Figura 30. Vista general del servicio Azure Container Apps



Fuente: *Portal de Azure.*

2.8.2. Comandos CLI

Obtener el id del log-analytics workspace

```
$LAW_ID = az monitor log-analytics workspace show --resource-group rg-bbva-dash-  
board --workspace-name law-bbva-dash-board --query customerId -o tsv
```

```
$LAW_KEY = az monitor log-analytics workspace get-shared-keys --resource-  
group rg-bbva-dash-board --workspace-name law-bbva-dash-board --  
query primarySharedKey -o tsv
```

Obtener el IDs

```
$SUBNET_ID = az network vnet subnet show --name snet-containerapp-infra --  
vnet-name vnet-bbva-dash-board --resource-group rg-bbva-red --  
query id -o tsv
```

```
$LAW_ID = az monitor log-analytics workspace show --resource-group rg-bbva-dash-  
board --workspace-name law-bbva-dash-board --query customerId -o tsv
```

```
$LAW_KEY = az monitor log-analytics workspace get-shared-keys --resource-group rg-bbva-dash-  
board --workspace-name law-bbva-dash-board --query primarySharedKey -o tsv
```

Crear el nuevo environment con VNET integration

```
az containerapp env create `
  --name managedEnvironment-vnet `
  --resource-group rg-bbva-dashboard `
  --location eastus `
  --infrastructure-subnet-resource-id $SUBNET_ID `
  --internal-only false `
  --logs-workspace-id $LAW_ID `
  --logs-workspace-key $LAW_KEY
```

2.9. Crear el Container App

2.9.1. Explicación del Uso

Sobre el environment anterior se crea la Container App bbva-backend-api, que ejecuta la imagen bbva-backend:v1 alojada en el ACR.

La captura muestra:

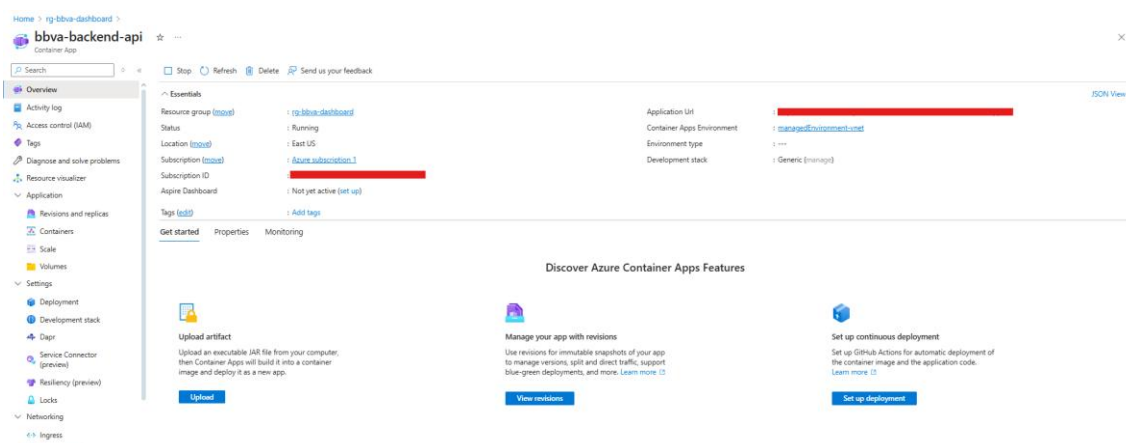
- Estado Running y URL pública de la aplicación
- Asociación al environment managedEnvironment-vnet
- Suscripción y resource group rg-bbva-dashboard

En esta Container App se configuran:

- Recursos (CPU/RAM) apropiados para una API ligera
- Variables de entorno con las cadenas de conexión a PostgreSQL y Storage, obtenidas desde Azure Key Vault
- Escalamiento automático basado en métricas (por ejemplo, número de requests o porcentaje de CPU), cumpliendo con los criterios de escalabilidad de la rúbrica

Esta API es el punto de entrada para el frontend React y otros posibles consumidores internos.

Figura 31. Vista general del servicio Container App



Fuente: Portal de Azure.

2.9.2. Comandos CLI

Crear variables

```
$DB_HOST = az keyvault secret show --vault-name kv-bbva-dashboard --name "db-host" --query value -o tsv
$DATABRICKS_WORKSPACE_URL = az keyvault secret show --vault-name kv-bbva-dashboard --name "databricks-workspace-url" --query value -o tsv
$AZURE_STORAGE_CONNECTION_STRING = az keyvault secret show --vault-name kv-bbva-dashboard --name "azure-storage-connection-string" --query value -o tsv
$ACR_PASSWORD = az keyvault secret show --vault-name kv-bbva-dashboard --name "acr-password" --query value -o tsv
$CLUSTER_ID = az keyvault secret show --vault-name kv-bbva-dashboard --name "databricks-cluster-id" --query value -o tsv
$DATABRICKS_TOKEN = az keyvault secret show --vault-name kv-bbva-dashboard --name "databricks-token" --query value -o tsv
$DB_PASSWORD = az keyvault secret show --vault-name kv-bbva-dashboard --name "db-password" --query value -o tsv
```

Crear el contenedor

```
$DB_HOST = az keyvault secret show --vault-name kv-bbva-dashboard --name "db-host" --query value -o tsv
```

```
# Crear el Container App
az containerapp create --name bbva-backend-api --resource-group rg-bbva-dashboard --environment managedEnvironment-vnet `
  --image acrbbvadashboard.azurecr.io/bbva-backend:v1 --registry-server acrbbvadashboard.azurecr.io `
  --registry-username acrbbvadashboard --registry-password $ACR_PASSWORD --target-port 5000 --ingress external `
  --cpu 0.5 --memory 1.0Gi --system-assigned --min-replicas 1 --max-replicas 1 `
  --env-vars `
    "DB_HOST=$DB_HOST" `
    "DB_PORT=5432" `
    "DB_USER=adminuser" `
    "DB_PASSWORD=$DB_PASSWORD" `
    "DB_NAME_PRACTITIONER=data_oro_practitioner" `
    "DB_NAME_CI=data_oro_ci" `
    "DATABRICKS_WORKSPACE_URL=$DATABRICKS_WORKSPACE_URL" `
    "DATABRICKS_TOKEN=$DATABRICKS_TOKEN" `
    "FLASK_ENV=production" `
    "AZURE_STORAGE_CONNECTION_STRING=$AZURE_STORAGE_CONNECTION_STRING" `
    "DATABRICKS_CLUSTER_ID=$CLUSTER_ID" `
    "KEY_VAULT_NAME=kv-bbva-dashboard"
```

Configurar permisos de Key Vault para Container App

```
$CONTAINER_APP_PRINCIPAL_ID = az containerapp show --name bbva-backend-api --resource-group rg-bbva-dashboard `
  --query identity.principalId -o tsv
$KV_ID = az keyvault show --name kv-bbva-dashboard --query id -o tsv

az role assignment create --assignee $CONTAINER_APP_PRINCIPAL_ID --role "Key"
```

```
Vault Secrets User" --scope $KV_ID
```

Actualizar la imagen

```
az containerapp update --name bbva-backend-api --resource-group rg-bbva-dashboard `
--image acrbbvadashboard.azurecr.io/bbva-backend:v1
```

2.10. Crear el Static Web App

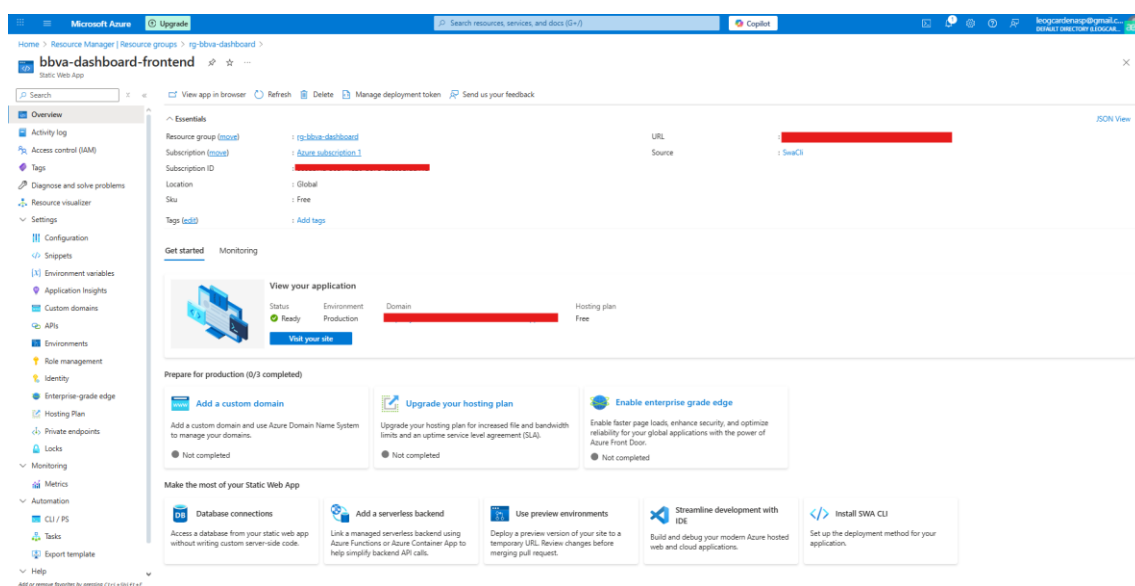
2.10.1. Explicación del Uso

Para la capa de presentación se utiliza Azure Static Web Apps, donde se despliega el dashboard web desarrollado en React.

En la primera captura se ve el recurso bbva-dashboard-frontend:

- Plan Free, suficiente para el laboratorio
- Dominio generado por Azure (URL de acceso público)
- Estado Ready, indicando que el último despliegue fue exitoso

Figura 32. Vista general del servicio Static Web App



Fuente: Portal de Azure.

En la segunda captura (sección Environments) se observa:

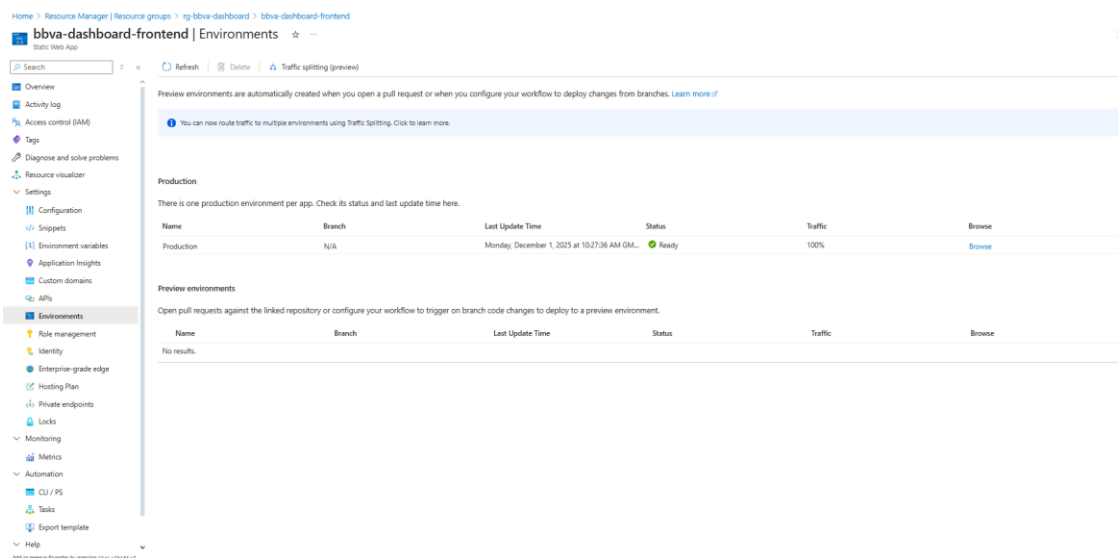
- Un entorno de producción activo al 100 % del tráfico
- La posibilidad de usar entornos preview vinculados a ramas o Pull Requests (útil para futuras mejoras de CI/CD)

El flujo de despliegue es:

- Clonar el repositorio del frontend
- Ejecutar npm install y npm run build para generar la carpeta de salida (dist o build)

- Configurar Static Web Apps para que tome esa carpeta como origen del contenido estático
- Cada push a la rama principal dispara un nuevo build y despliegue

Figura 33. Vista general de la sección Environments



Fuente: *Portal de Azure.*

2.10.2. Comandos CLI

```
az staticwebapp create --name bbva-dashboard-frontend --resource-group rg-bbva-dashboard --location eastus2 --sku Free
```

Obtener el URL del backend para reemplazarlo en el config del frontend

```
$API_BASE_URL = "HYPERLINK \"https://$(az containerapp show --name bbva-backend-api --resource-group rg-bbva-dashboard --query properties.configuration.ingress.fqdn -o tsv)\""
```

```
Write-Host $API_BASE_URL
```

Abrir una terminal en la carpeta del frontend para contruir la carpeta dist

```
npm install
npm run build
```

Subir la carpeta dist

```
$token = az staticwebapp secrets list --name bbva-dashboard-frontend --resource-group rg-bbva-dashboard --query properties.apiKey -o tsv
```

```
swa deploy --app-location "$env:USERPROFILE\Desktop\frontend\dist" --deployment-token "$token" --env "production"
```

2.11. Crear el Log Analytics workspace

2.11.1. Explicación del Uso

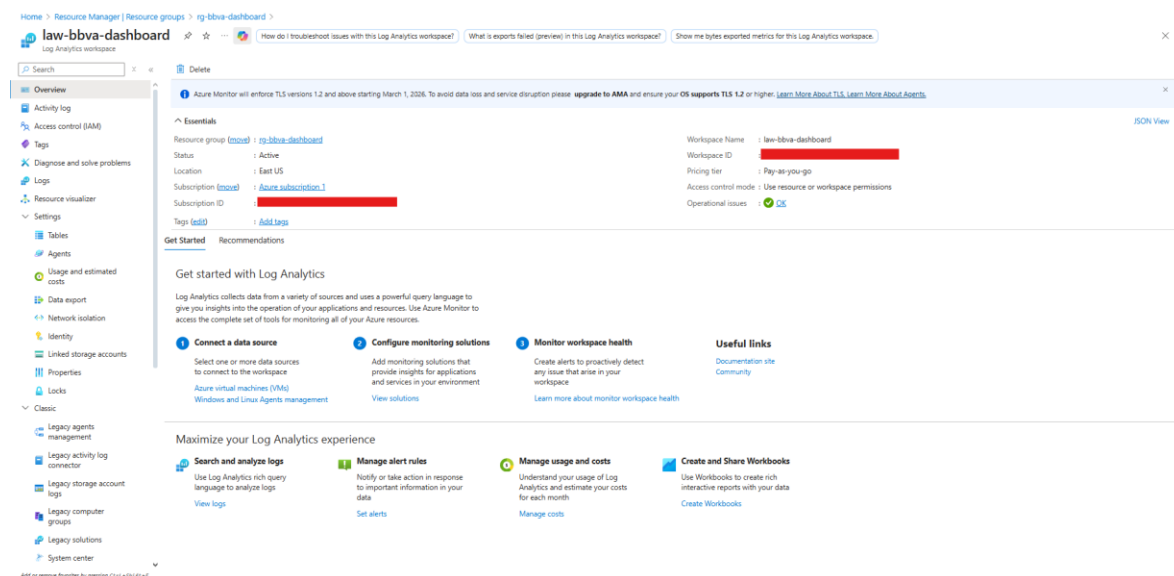
Para centralizar métricas y logs de toda la solución se crea un Log Analytics workspace denominado law-bbva-dashboard. Se puede observar:

- Workspace en la región East US
- Modelo de facturación Pay-as-you-go
- Integración con la suscripción y el resource group del proyecto

Este workspace es el backend de observabilidad sobre el que se apoyan:

- Azure Monitor
- Las métricas y logs de:
 - Storage Account (Data Lake)
 - Container Apps (backend)
 - Azure Database for PostgreSQL
 - Otros recursos que se deseen monitorear

Figura 34. Vista General del Servicio Log Analytics Workspace



Fuente: *Portal de Azure.*

Una vez configurado el envío de diagnósticos al workspace, es posible ejecutar consultas Kusto Query Language (KQL) para analizar el comportamiento del entorno. En la figura x se muestra un ejemplo de consulta sobre la tabla StorageBlobLogs.

Figura 35. Vista General de la Tabla StorageBlobLogs

Home > Resource Manager > Resource groups > rg-bbva-dashboard > law-bbva-dashboard

law-bbva-dashboard | Logs

Log Analytics workspace

Search

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Logs
Resource visualizer
Settings

Tables
Agents
Usage and estimated costs
Data export
Network isolation
Identity
Linked storage accounts
Properties
Locks
Classic
Legacy agents management
Legacy activity log connector
Legacy storage account logs
Legacy computer groups
Legacy solutions
System center

New Query 1

Time range: Last 24 hours Show: 1000 results

1 StorageBlobLogs
2 where ObjectKey contains '/Bronze/'
3 project TimeGenerated, OperationName, StatusCode, ObjectKey
4 order by TimeGenerated desc
5 take 20

Results Chart

TimeGenerated (UTC)	OperationName	StatusCode	ObjectKey
> 12/1/2023, 4:42:25.764 AM	ReadFile	206	/rthbdatalake/bronze/data_sucia/data_sucia_continuous_integration/_utils.csv
> 12/1/2023, 4:42:25.534 AM	ReadFile	206	/rthbdatalake/bronze/data_sucia/data_sucia_practitioner/practitioner_utils.csv
> 12/1/2023, 4:42:25.448 AM	ReadFile	206	/rthbdatalake/bronze/data_sucia/data_sucia_continuous_integration/_utils.csv
> 12/1/2023, 4:42:25.442 AM	ReadFile	206	/rthbdatalake/bronze/data_sucia/data_sucia_practitioner/practitioner_utils.csv
> 12/1/2023, 4:42:25.438 AM	ReadFile	206	/rthbdatalake/bronze/data_sucia/data_sucia_practitioner/practitioner_utils.csv
> 12/1/2023, 4:42:25.434 AM	ReadFile	206	/rthbdatalake/bronze/data_sucia/data_sucia_continuous_integration/_utils.csv
> 12/1/2023, 4:42:12.652 AM	GetItemStatus	200	/rthbdatalake/bronze/data_sucia/data_sucia_continuous_integration/_utils.csv
> 12/1/2023, 4:42:12.641 AM	GetItemStatus	200	/rthbdatalake/bronze/data_sucia/data_sucia_continuous_integration/_utils.csv
> 12/1/2023, 4:42:12.241 AM	GetItemStatus	200	/rthbdatalake/bronze/data_sucia/data_sucia_continuous_integration/_utils.csv
> 12/1/2023, 4:42:12.235 AM	GetItemStatus	200	/rthbdatalake/bronze/data_sucia/data_sucia_practitioner
> 12/1/2023, 4:42:12.226 AM	GetItemStatus	404	/rthbdatalake/bronze/data_sucia/data_sucia_continuous_integration/_spark_metadata
> 12/1/2023, 4:42:12.219 AM	GetItemStatus	404	/rthbdatalake/bronze/data_sucia/data_sucia_practitioner/_spark_metadata
> 12/1/2023, 4:42:12.203 AM	GetItemStatus	200	/rthbdatalake/bronze/data_sucia/data_sucia_continuous_integration
> 12/1/2023, 4:42:12.198 AM	GetItemStatus	200	/rthbdatalake/bronze/data_sucia/data_sucia_practitioner
> 12/1/2023, 4:42:11.900 AM	GetItemStatus	404	/rthbdatalake/bronze/_delta_log
> 12/1/2023, 4:42:11.896 AM	GetItemStatus	404	/rthbdatalake/bronze/_delta_log
> 12/1/2023, 4:42:11.890 AM	GetItemStatus	404	/rthbdatalake/bronze/data_sucia/_delta_log
> 12/1/2023, 4:42:11.886 AM	GetItemStatus	404	/rthbdatalake/bronze/data_sucia/_delta_log

16 68ms Display time (UTC+05:00)

Query details 1 - 19 of 20

Fuente: *Portal de Azure.*

Esta consulta permite:

- Ver las últimas operaciones de lectura/escritura realizadas sobre la capa Bronze del Data Lake
- Identificar errores (códigos 4xx/5xx) y patrones de acceso durante la ejecución del ETL

2.11.2. Comandos CLI

```
az monitor log-analytics workspace create \
  --resource-group rg-bbva-dashboard \
  --workspace-name law-bbva-dashboard \
  --location eastus
```

Ruta para ejecutar cualquier archivo

Para el Databricks (Desde el CLI)

- Cluster Events

```
databricks clusters events 1129-051945-6zvtepb6
```

- Para Jobs

```
databricks jobs list
databricks jobs list-runs --job-id <ID>
databricks jobs get-run-output --run-id <RUN_ID>
```

2.12. Crear las Alertas

2.12.1. Explicación del Uso

Se configuran reglas de alerta en Azure Monitor para reaccionar ante problemas de rendimiento, disponibilidad o capacidad. En la captura de Alert rules se visualizan, entre otras, las siguientes reglas:

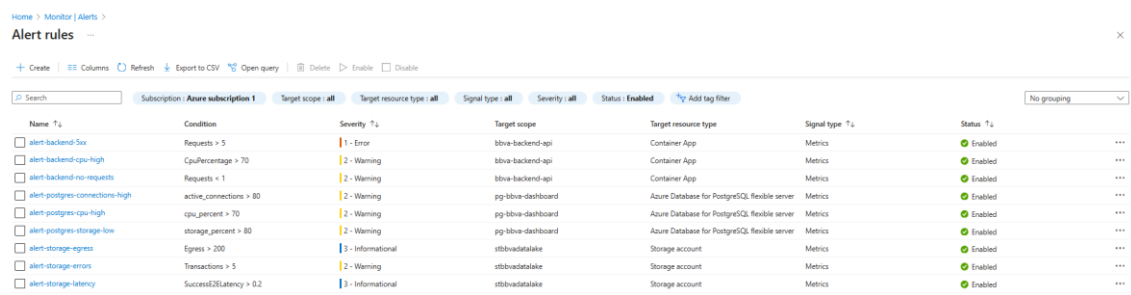
- alert-backend-5xx: se dispara cuando el backend devuelve más de 5 respuestas 5xx, indicando errores en la API
- alert-backend-cpu-high: alerta cuando el CPU de la Container App supera el 70 % durante un periodo sostenido
- alert-backend-no-requests: detecta ausencia de requests (posible caída de tráfico o problemas de conectividad)
- alert-postgres-connections-high: monitorea conexiones activas al servidor PostgreSQL, evitando saturar el límite configurado
- alert-postgres-cpu-high y alert-postgres-storage-low: vigilan el uso de CPU y el porcentaje de almacenamiento consumido
- alert-storage-egress, alert-storage-errors y alert-storage-latency: verifican volumen de tráfico, número de errores y latencia en el Data Lake

Cada regla define:

- Condición (métrica, umbral y ventana de tiempo)
- Severidad (Error, Warning o Informational)
- Action Group asociado (por ejemplo, envío de correo al equipo o notificación a Teams)

Con este set de alertas la solución cumple con los requisitos de monitoreo y gobernanza, permitiendo detectar incidentes con anticipación y tomar acciones correctivas.

Figura 36. Estructura de las reglas de Alerta



Name	Condition	Severity	Target scope	Target resource type	Signal type	Status
☐ alert-backend-5xx	Requests > 5	1 - Error	lbv-backend-api	Container App	Metrics	Enabled
☐ alert-backend-cpu-high	CpuPercentage > 70	2 - Warning	lbv-backend-api	Container App	Metrics	Enabled
☐ alert-backend-no-requests	Requests < 1	2 - Warning	lbv-backend-api	Container App	Metrics	Enabled
☐ alert-postgres-connections-high	active_connections > 80	2 - Warning	pg-lbva-dashboard	Azure Database for PostgreSQL flexible server	Metrics	Enabled
☐ alert-postgres-cpu-high	cpu_percent > 70	2 - Warning	pg-lbva-dashboards	Azure Database for PostgreSQL flexible server	Metrics	Enabled
☐ alert-postgres-storage-low	storage_percent > 80	2 - Warning	pg-lbva-dashboards	Azure Database for PostgreSQL flexible server	Metrics	Enabled
☐ alert-storage-egress	Egress > 200	1 - Informational	stbivadatalake	Storage account	Metrics	Enabled
☐ alert-storage-errors	Transactions > 5	2 - Warning	stbivadatalake	Storage account	Metrics	Enabled
☐ alert-storage-latency	Success2SLatency > 0.2	1 - Informational	stbivadatalake	Storage account	Metrics	Enabled

Fuente: Portal de Azure.

2.12.2. Comandos CLI

```
az group create --name rg-bbva-alerts --location eastus
```

Para el Datalake

- Error rate anormal (transacciones fallidas)

```
az monitor metrics alert create `
  --name alert-storage-errors `
  --resource-group rg-bbva-alerts `
  --scopes $STORAGE_ID `
  --condition "total Transactions > 5 where ResponseType includes Error" `
  --description "Errores de Storage > 5 en 5 minutos" `
  --evaluation-frequency 1m `
  --window-size 5m `
  --severity 2
```

- Latencia alta

```
az monitor metrics alert create `
  --name alert-storage-latency `
  --resource-group rg-bbva-alerts `
  --scopes $STORAGE_ID `
  --condition "avg SuccessE2ELatency > 0.2" `
  --description "Latencia del Storage > 200ms" `
  --evaluation-frequency 5m `
  --window-size 15m `
  --severity 3
```

- Costo inesperado por datos salientes

```
az monitor metrics alert create `
  --name alert-storage-egress `
  --resource-group rg-bbva-alerts `
  --scopes $STORAGE_ID `
  --condition "total Egress > 200" `
  --description "Egress del Storage mayor a 200 MB (posible fuga de datos)" `
  --evaluation-frequency 30m `
  --window-size 24h `
  --severity 3
```

Para el container app

- Error rate en el backend

```
$API_ID = az containerapp show --name bbva-backend-api --resource-group rg-
bbva-dashboard --query id -o tsv
```

```
az monitor metrics alert create `
  --name alert-backend-5xx `
  --resource-group rg-bbva-alerts `
  --scopes $API_ID `
  --condition "total Requests > 5 where statusCodeCategory includes 5XX" `
  --description 'Más de 5 errores HTTP 5xx en 5 min' `
```

```
--evaluation-frequency 1m `
--window-size 5m `
--severity 1
```

- Backend sin requests (posible caída)

```
az monitor metrics alert create `
--name alert-backend-no-requests `
--resource-group rg-bbva-alerts `
--scopes $API_ID `
--condition "total Requests < 1" `
--description "0 requests por 15 minutos → posible caída del backend" `
--evaluation-frequency 5m `
--window-size 15m `
--severity 2
```

- CPU muy alta (>70%)

```
az monitor metrics alert create `
--name alert-backend-cpu-high `
--resource-group rg-bbva-alerts `
--scopes $API_ID `
--condition "avg CpuPercentage > 70" `
--description "CPU del backend mayor al 70%" `
--evaluation-frequency 1m `
--window-size 5m `
--severity 2
```

Para el postgresql

- Consumo de CPU alto

```
$PG_ID = az postgres flexible-server show --name pg-bbva-dashboard --
resource-group rg-bbva-dashboard --query id -o tsv
```

```
az monitor metrics alert create `
--name alert-postgres-cpu-high `
--resource-group rg-bbva-alerts `
--scopes $PG_ID `
--condition "avg cpu_percent > 70" `
--description "CPU de PostgreSQL mayor al 70%" `
--evaluation-frequency 5m `
--window-size 15m `
--severity 2
```

- Conexiones máximas (>80%)

```
az monitor metrics alert create `
--name alert-postgres-connections-high `
--resource-group rg-bbva-alerts `
--scopes $PG_ID `
--condition "avg active_connections > 80" `
--description 'Conexiones activas por encima del 80% del máximo permitido'
,
--evaluation-frequency 5m `
--window-size 15m `
--severity 2
```

- Espacio en disco bajo (<20%)

```
az monitor metrics alert create `
--name alert-postgres-storage-low `
--resource-group rg-bbva-alerts `
--scopes $PG_ID `
--condition "avg storage_percent > 80" `
--description "PostgreSQL está usando más del 80% del almacenamiento" `
--evaluation-frequency 30m `
--window-size 1h `
--severity 2
```

2.13. Audit Logs

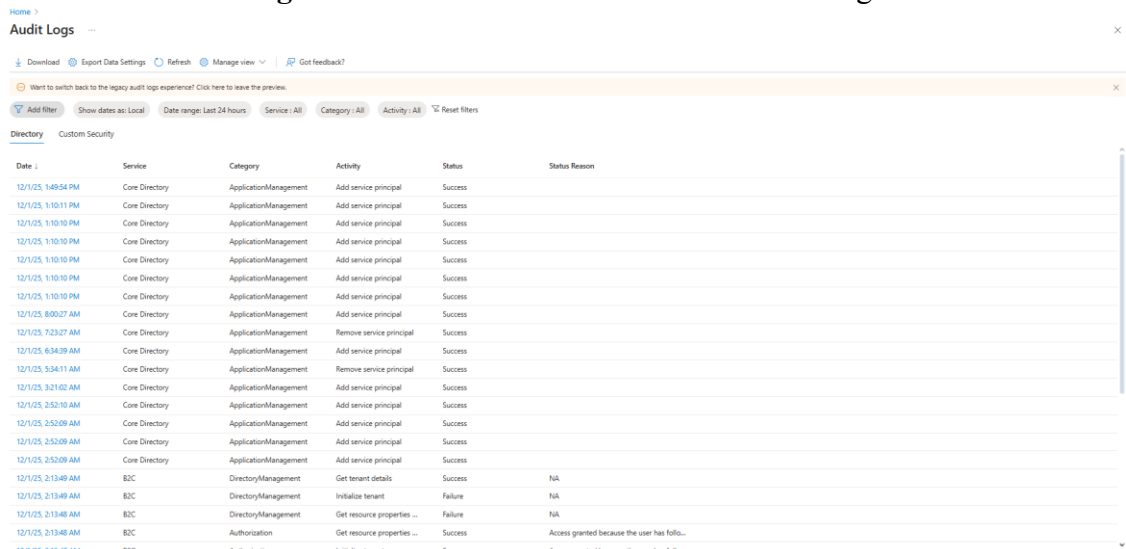
2.13.1. Explicación del Uso

Para cumplir con la parte de auditoría y gobernanza se habilitó el uso de Audit Logs en Microsoft Entra ID (Azure AD).

Estos logs permiten rastrear:

- Creación y eliminación de service principals usados por Databricks y Container Apps
- Cambios en asignaciones de roles y permisos
- Acciones de administración realizadas en el tenant

Figura 37. Vista General del Servicio Audit Logs



Fuente: Portal de Azure.

2.14. Crear el Backup Vault

2.14.1. Explicación del Uso

Para la estrategia de respaldo y recuperación ante desastres (DR) se configuró un Azure Backup Vault dedicado al proyecto. En este vault se centraliza la política de backup de:

- Azure Database for PostgreSQL flexible server
- (Opcional) Máquinas virtuales o recursos adicionales si el entorno creciera

Figura 38. Vista General del Servicio Backup Vault

Name	Type	Security level	Resource Group	Location	Subscription
rg-bbva-dash-board-backup	Backup vault	Fair	rg-bbva-dash-board	East US	Azure subscription 1

Fuente: *Portal de Azure.*

En las propiedades del vault se puede apreciar:

Figura 39: Vista General de las Propiedades del Servicio Backup Vault

Category	Setting	Value	Action
Vault Settings	Monitoring Settings	Update	Update
	Cross Subscription Restore	Enabled	Update
	Cross Region Restore	Enabled	View
	Security Settings	Fair	Update
Multi-User Authorization	Multi-User Authorization	Enabled Not locked	Settings
	Immutable vault	Enabled	Update
Soft delete	Soft delete	Enabled	Update
Encryption Settings	Encryption Settings	Using 'Microsoft-managed keys'	Update

Fuente: *Portal de Azure.*

Aspectos relevantes:

- Redundancia: Geo-redundant (GRS) para disponer de copias en una región secundaria
- Soft delete habilitado para proteger contra eliminaciones accidentales
- Cross Region Restore habilitado, lo que permite restaurar backups en otra región en caso de caída total de la principal

2.14.2 Comandos CLI

Registrar los providers

```
az provider register --namespace Microsoft.DataProtection
az provider register --namespace Microsoft.RecoveryServices
```

Crear Backup Vault

```
az dataprotection backup-vault create \
  --resource-group rg-bbva-dashboard \
  --vault-name rg-bbva-dashboard-backup \
  --location eastus \
  --storage-settings type="GeoRedundant" datastore-type="VaultStore"
```

2.15. Crear la Red

2.15.1. Explicación del Uso

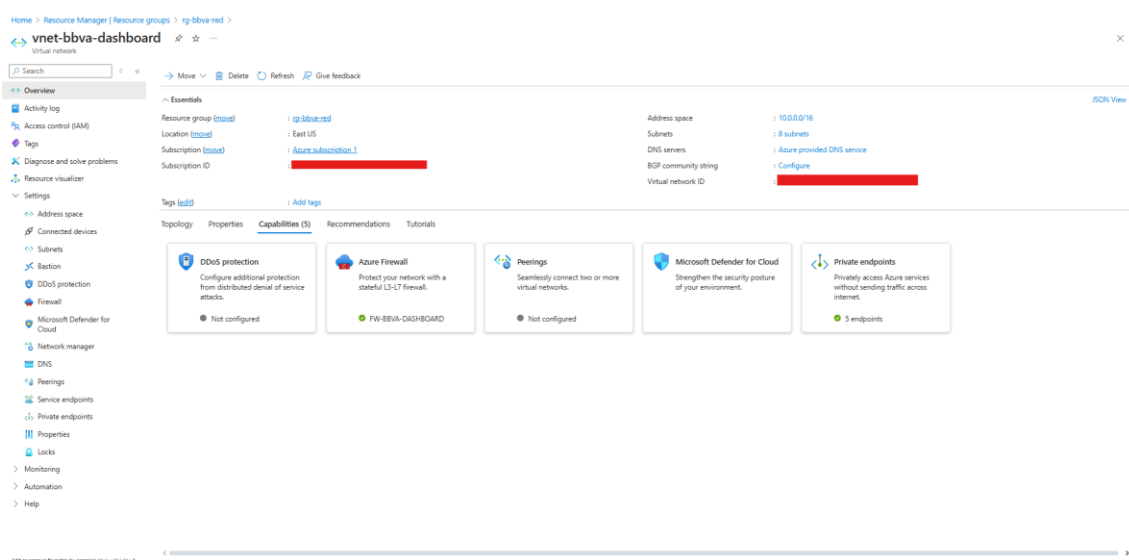
La conectividad interna del entorno se organiza alrededor de la VNet `vnet-bbva-dashboard`.

Características principales:

- Address space: 10.0.0.0/16
- Región: East US
- Integración con:
 - Azure Firewall para inspección y filtrado de tráfico saliente
 - Private Endpoints de Storage, PostgreSQL, ACR y Key Vault
 - Subnets específicas por servicio (según se detalla a continuación)

Esta VNet también se enlaza con otras capacidades de red que se crean más adelante (Private DNS Zones, Private Endpoints, etc.), de forma que los servicios PaaS se consumen por rutas privadas.

Figura 40. Vista General del Servicio Vnet



Fuente: *Portal de Azure.*

Dentro de `vnet-bbva-dashboard` se definieron subredes dedicadas por tipo de servicio: Subnets más relevantes:

- AzureFirewallSubnet – Reservada para el servicio de Azure Firewall
- snet-postgresql – Aloja el Private Endpoint hacia el servidor PostgreSQL
- snet-keyvault – Aloja el Private Endpoint de Key Vault
- snet-containerapp y snet-containerapp-infra – Dedicadas al entorno de Container Apps y su infraestructura
- snet-storage – Donde residen los Private Endpoints de Blob y DFS para el Data Lake
- snet-databricks-public y snet-databricks-private – Redes usadas por el workspace de Databricks (nodos de cluster y conectividad interna)

El objetivo es segmentar el tráfico por tipo de servicio, aplicar reglas de seguridad específicas y evitar que recursos con diferentes perfiles de riesgo compartan la misma subnet.

Figura 41. Vista General de las Subnets Asignadas

Name	IPv4	IPv6	Available IPs	Delegated to	Security group	Route table
AzureFirewallSubnet	10.0.0/24	-	248	-	-	-
snet-postgresql	10.0.2/24	-	250	-	nsg-postgresql	-
snet-keyvault	10.0.7/24	-	250	-	nsg-keyvault	-
snet-containerapp	10.0.1/24	-	251	Microsoft.App/environme...	nsg-containerapp	rt-firewall
snet-storage	10.0.5/24	-	247	-	nsg-storage	rt-firewall
snet-containerapp-infra	10.0.8/23	-	507	Microsoft.App/environme...	-	-
snet-databricks-public	10.0.3/24	-	251	-	nsg-databricks	rt-firewall
snet-databricks-private	10.0.4/24	-	251	-	nsg-databricks	rt-firewall

Fuente: Portal de Azure.

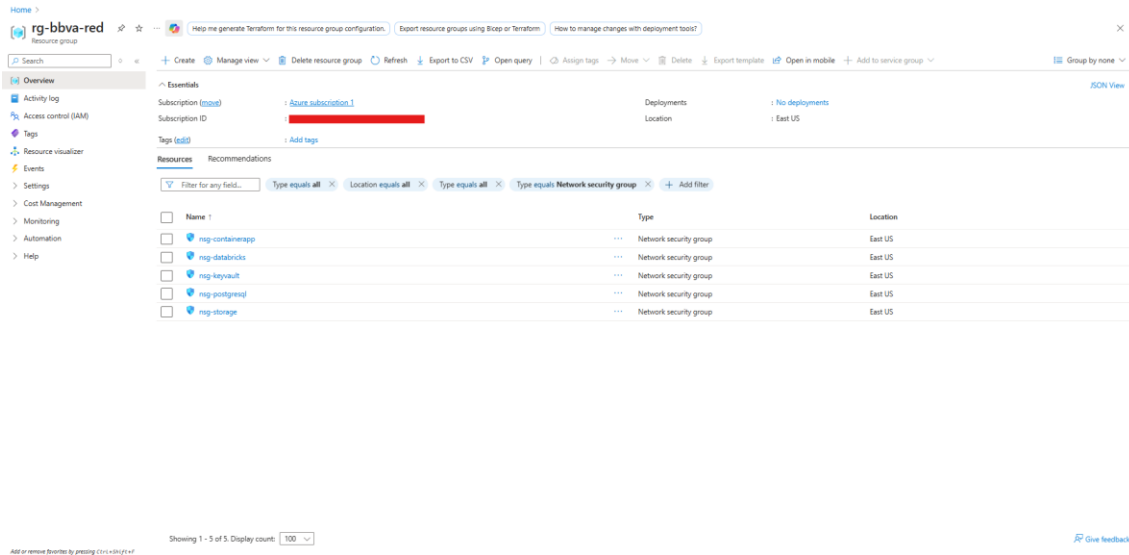
A) Crear los Network Security Groups (NSG)

Para controlar el tráfico a nivel de capa 4 se crearon Network Security Groups por tipo de servicio. Ejemplos de reglas:

- En nsg-postgresql:
 - Permitir tráfico entrante desde snet-containerapp y snet-databricks-private al puerto 5432
 - Denegar tráfico desde Internet
- En nsg-storage:
 - Permitir acceso desde subnets de Databricks y Container Apps a los endpoints de Storage
 - Restringir otros orígenes
- En nsg-databricks:
 - Controlar el tráfico saliente de los clusters hacia servicios internos y externos

Con esto se consigue un modelo zero-trust donde solo se permiten las comunicaciones necesarias entre componentes.

Figura 42. Vista General del Servicio Network Security Groups (NSG)

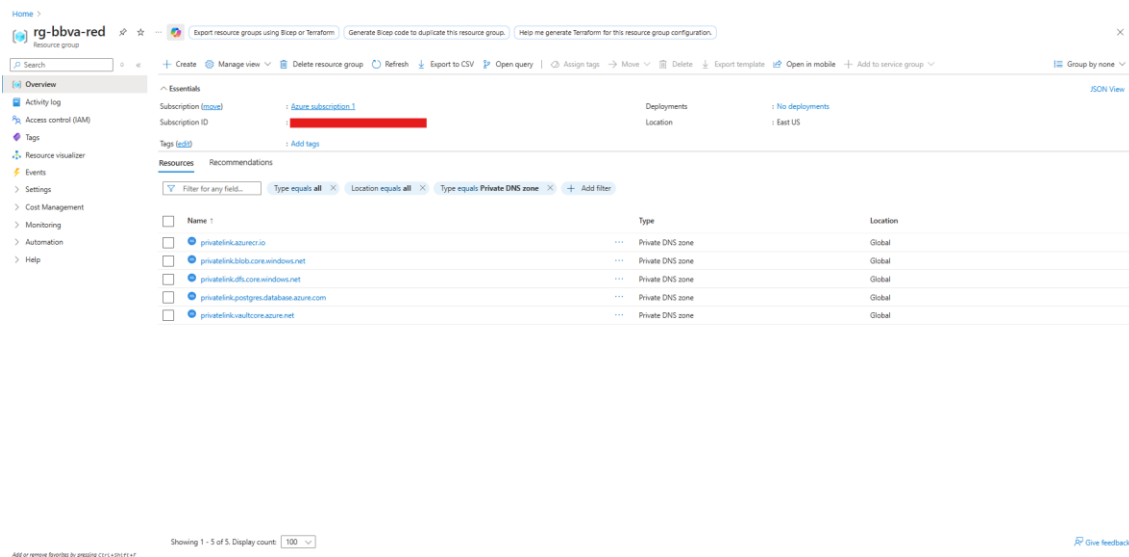


Fuente: *Portal de Azure.*

B) Crear las Private DNS Zones

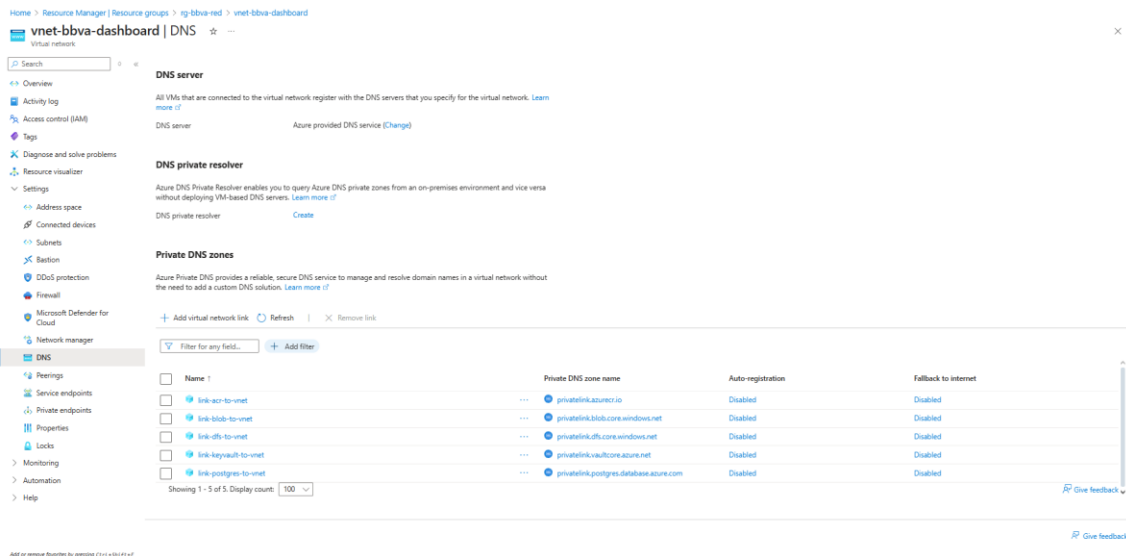
Para que los servicios PaaS sean accesibles por nombres privados se crearon varias Private DNS Zones:

Figura 43. Vista general de los Private DNS creados



Fuente: *Portal de Azure.*

Estas zonas se agregaron a la VNet vnet-bbva-dashboard:

Figura 44. Vista General de la Configuración del Servicio Vnet

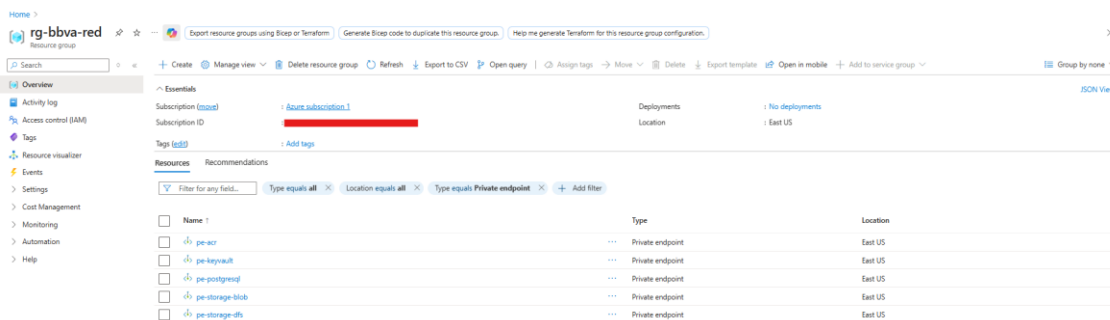
Fuente: *Portal de Azure.*

De esta forma:

- Cuando el backend o Databricks resuelven, por ejemplo, `stbvbatalake.dfs.core.windows.net`, obtienen una IP privada en la VNet
- Todo el tráfico hacia ACR, Storage, Key Vault y PostgreSQL fluye por la red privada de Azure, sin exponer endpoints públicos

C) Crear los Private Endpoints

Finalmente, se definieron Private Endpoints para los servicios críticos, mapeándolos a las subnets correspondientes.

Figura 45. Vista General de los servicios Principales

Fuente: *Portal de Azure.*

Figura 46. Vista General de los Private Endpoints definidos

Name	Subnet	Private endpoint network policy	Resource Group
PE-POSTGRESQL	snet-postgresql	Disabled	RG-BBVA-RED
PE-KEYVAULT	snet-keyvault	Disabled	RG-BBVA-RED
PE-ACR	snet-storage	Disabled	RG-BBVA-RED
PE-STORAGE-BLOB	snet-storage	Disabled	RG-BBVA-RED
PE-STORAGE-DFS	snet-storage	Disabled	RG-BBVA-RED

Fuente: *Portal de Azure.*

Private Endpoints configurados:

- pe-acr → Azure Container Registry (acrbbvadaashboard) en snet-storage o subnet específica
- pe-keyvault → Key Vault del proyecto en snet-keyvault
- pe-postgresql → Servidor PostgreSQL flexible pg-bbva-dashboard en snet-postgresql
- pe-storage-blob y pe-storage-dfs → Cuenta de Storage stbbvadalake (Blob y Data Lake Gen2) en snet-storage

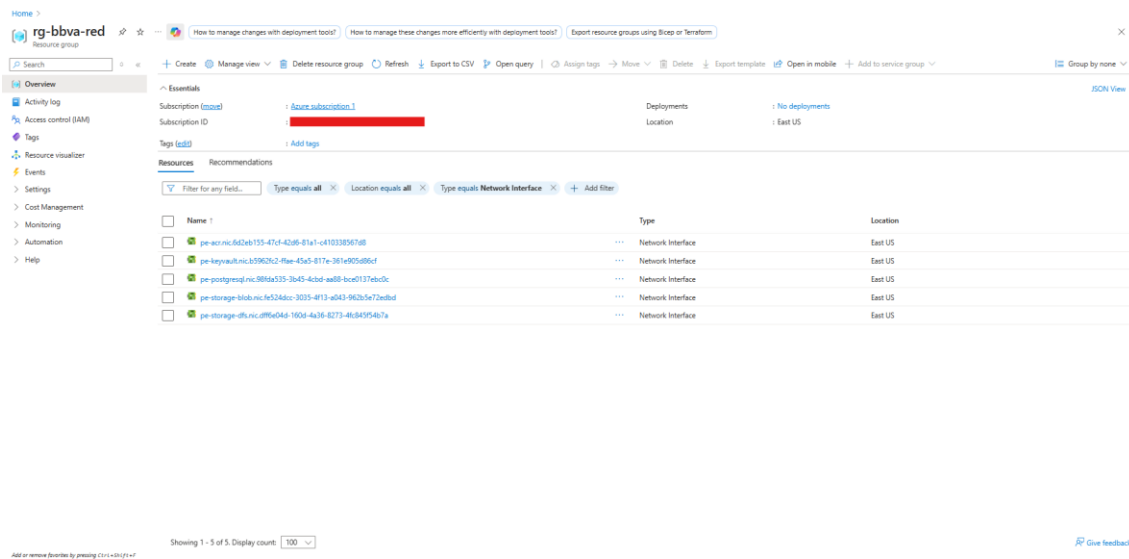
Gracias a estos endpoints:

- Container Apps, Databricks y otros servicios acceden a los recursos PaaS sin salir a Internet
- Se reduce la superficie de ataque al deshabilitar (o minimizar) el acceso público
- Se facilita el cumplimiento de políticas de seguridad y gobernanza de red

D) Network Interfaces

Las Network Interfaces (NIC) son los recursos que vinculan la red virtual con los servicios privados como Storage, PostgreSQL, Key Vault y el propio Azure Container Apps. Cada private endpoint crea de manera automática una NIC dentro del grupo de recursos de red, lo que permite controlar en detalle el tráfico que entra y sale de cada servicio.

Figura 47. Vista General de Network Interfaces (NIC) para cada servicio



Fuente: *Portal de Azure.*

En la figura se puede apreciar que se han creado cinco interfaces de red, cada una asociada a un private endpoint específico:

- pe-acr-nic – interfaz asociada al private endpoint del Azure Container Registry
- pe-keyvault-nic – interfaz asociada al private endpoint del Key Vault
- pe-postgresql-nic – interfaz asociada al private endpoint de la base de datos PostgreSQL flexible
- pe-storage-blob-nic y pe-storage-dfs-nic – interfaces asociadas a los private endpoints de Blob Storage y Data Lake (DFS)

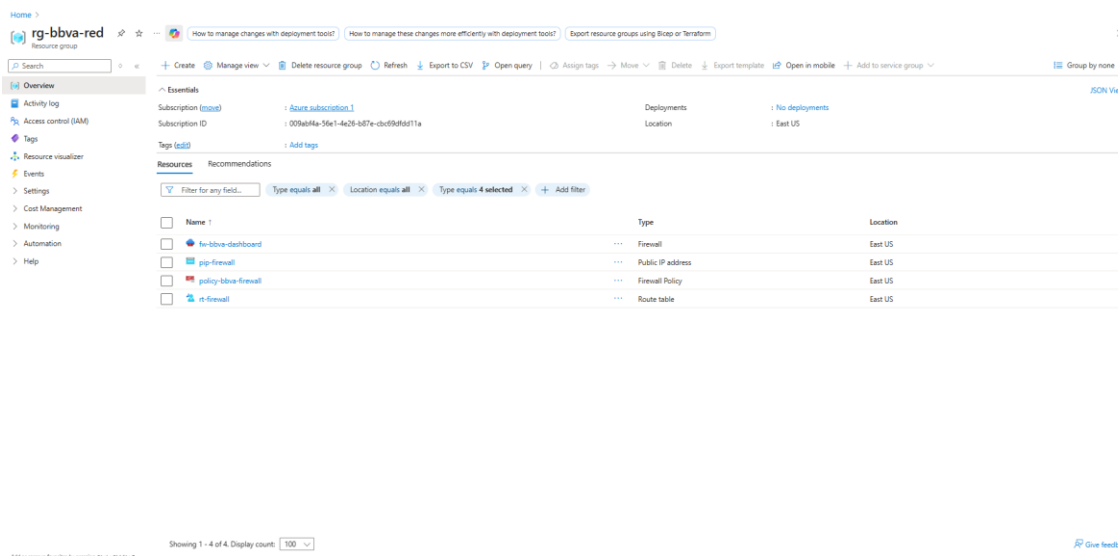
Tener estas NIC dedicadas permite:

- Aislar el tráfico de cada servicio dentro de la VNet
- Aplicar reglas de seguridad por subnet y por Network Security Group (NSG)
- Auditar de forma más precisa los accesos a nivel de red

E) Creación del Firewall y tabla de ruteo

Para centralizar el control del tráfico saliente y proteger el entorno frente a accesos no deseados desde Internet, se implementa un Azure Firewall dedicado, ubicado en el mismo grupo de recursos de red.

Figura 48. Vista General de los recursos asignados con el firewall respectivo



Fuente: Portal de Azure.

Se observan los recursos principales relacionados con el firewall:

- fw-bbva-dashboard – instancia de Azure Firewall que inspecciona el tráfico
- pip-firewall – IP pública asociada al firewall, necesaria para el tráfico de salida controlado
- policy-bbva-firewall – Firewall Policy donde se definen las reglas de:
 - Acceso a servicios de Azure (Storage, PostgreSQL, ACR, etc.)
 - Restricciones por FQDN o rangos de IP
- rt-firewall – tabla de ruteo que fuerza el tráfico de determinadas subnets (por ejemplo, Databricks, Container Apps, Storage) a pasar por el firewall

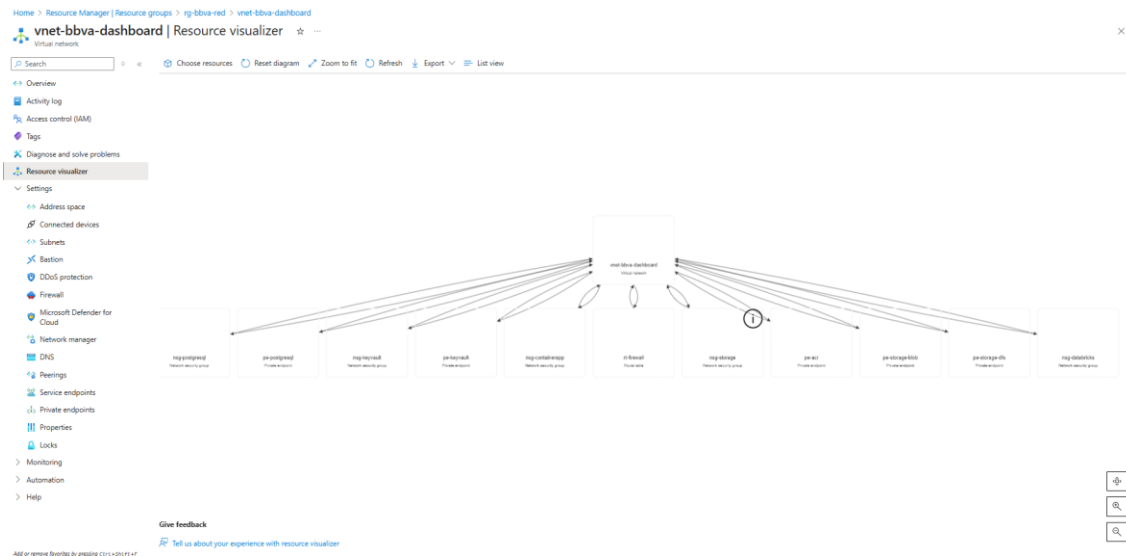
De esta manera, cualquier salida hacia Internet o hacia servicios PaaS expuestos por IP pública se canaliza a través del firewall, donde se pueden aplicar reglas de filtrado, logging y futuras políticas de inspección más avanzadas.

E) Visualización general de la topología de red

Para validar la configuración y evidenciar la conectividad entre los distintos componentes, se utiliza la funcionalidad de Resource Visualizer sobre la red virtual principal. Esta vista permite mostrar de forma gráfica la relación entre:

- La VNet vnet-bbva-dashboard
- Las subnets especializadas (storage, key vault, PostgreSQL, container app, Databricks)
- Los Network Security Groups (NSG) que protegen cada subnet
- Los Private Endpoints que conectan los servicios PaaS a la VNet
- El firewall y su tabla de ruteo

Figura 49. Topología de la red



Fuente: *Portal de Azure.*

En el diagrama se observa cómo:

- La VNet actúa como nodo central desde el cual se conectan los NSG, el firewall y los private endpoints
- El firewall se posiciona como punto de salida controlada, recibiendo tráfico desde las subnets de aplicación y redirigiéndolo hacia Internet o servicios externos
- Los private endpoints garantizan que PostgreSQL, Storage, Key Vault y el ACR sean consumidos de forma privada, sin exponer endpoints públicos

Esta visualización se incluye para que se tenga una comprensión rápida de la topología de red, apoyando la explicación previa de subnets, NSG, Private DNS Zones, Private Endpoints, Network Interfaces y Firewall.

2.15.2. Comandos CLI

Crear la Virtual Network con la primera subnet

```
az network vnet create \
  --name vnet-bbva-dashboard \
  --resource-group rg-bbva-red \
  --location eastus \
  --address-prefix 10.0.0.0/16 \
  --subnet-name snet-containerapp \
  --subnet-prefix 10.0.1.0/24
```

Crear subnet para el environment del ACR

```
az network vnet subnet create \
  --name snet-containerapp-infra \
  --resource-group rg-bbva-red \
  --vnet-name vnet-bbva-dashboard
```

```
--address-prefix 10.0.8.0/23
--delegations Microsoft.App/environments
```

Crear subnet para PostgreSQL

```
az network vnet subnet create `
  --name snet-postgresql `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --address-prefix 10.0.2.0/24
```

Crear subnet para Databricks (necesita 2 subnets)

```
az network vnet subnet create `
  --name snet-databricks-public `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --address-prefix 10.0.3.0/24
```

```
az network vnet subnet create `
  --name snet-databricks-private `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --address-prefix 10.0.4.0/24
```

Crear subnet para Storage Account (Private Endpoints)

```
az network vnet subnet create `
  --name snet-storage `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --address-prefix 10.0.5.0/24
```

Crear subnet para Azure Firewall

```
az network vnet subnet create `
  --name AzureFirewallSubnet `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --address-prefix 10.0.6.0/24
```

Crear subnet para Key Vault (Private Endpoint)

```
az network vnet subnet create `
  --name snet-keyvault `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --address-prefix 10.0.7.0/24
```

Network Security Groups

NSG para Container App

Crear NSG

```
az network nsg create `
  --name nsg-containerapp `
  --resource-group rg-bbva-red `
  --location eastus
```

Regla 1: Permitir HTTPS desde internet (puerto 443)

```
az network nsg rule create `
  --name AllowHTTPS `
  --nsg-name nsg-containerapp `
  --resource-group rg-bbva-red `
  --priority 100 `
  --source-address-prefixes Internet `
  --destination-port-ranges 443 `
  --protocol Tcp `
  --access Allow `
  --direction Inbound `
  --description "Permitir tráfico HTTPS público"
```

Regla 2: Permitir HTTP desde internet (puerto 80)

```
az network nsg rule create `
  --name AllowHTTP `
  --nsg-name nsg-containerapp `
  --resource-group rg-bbva-red `
  --priority 110 `
  --source-address-prefixes Internet `
  --destination-port-ranges 80 `
  --protocol Tcp `
  --access Allow `
  --direction Inbound `
  --description "Permitir tráfico HTTP público"
```

Regla 3: Permitir puerto 5000 para el backend Flask

```
az network nsg rule create `
  --name AllowBackendPort `
  --nsg-name nsg-containerapp `
  --resource-group rg-bbva-red `
  --priority 120 `
  --source-address-prefixes 10.0.0.0/16 `
  --destination-port-ranges 5000 `
  --protocol Tcp `
  --access Allow `
  --direction Inbound `
  --description "Permitir puerto 5000 del backend desde VNET"
```

Asociar NSG a la subnet

```
az network vnet subnet update `
  --name snet-containerapp `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --network-security-group nsg-containerapp
```

NSG para PostgreSQL

Crear NSG

```
az network nsg create `
  --name nsg-postgresql `
  --resource-group rg-bbva-red `
  --location eastus
```

Permitir conexiones PostgreSQL solo desde la VNET

```
az network nsg rule create `
  --name AllowPostgreSQL `
  --nsg-name nsg-postgresql `
  --resource-group rg-bbva-red `
  --priority 100 `
  --source-address-prefixes 10.0.0.0/16 `
  --destination-port-ranges 5432 `
  --protocol Tcp `
  --access Allow `
  --direction Inbound `
  --description "Permitir PostgreSQL solo desde la VNET"
```

Denegar todo el tráfico público entrante

```
az network nsg rule create `
  --name DenyAllInbound `
  --nsg-name nsg-postgresql `
  --resource-group rg-bbva-red `
  --priority 4096 `
  --source-address-prefixes '*' `
  --destination-port-ranges '*' `
  --protocol '*' `
  --access Deny `
  --direction Inbound `
  --description "Denegar todo el tráfico público"
```

Asociar NSG a la subnet

```
az network vnet subnet update `
  --name subnet-postgresql `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --network-security-group nsg-postgresql
```

NSG para Storage Account

Crear NSG

```
az network nsg create `
  --name nsg-storage `
  --resource-group rg-bbva-red `
  --location eastus
```

Permitir acceso HTTPS solo desde la VNET

```
az network nsg rule create `
  --name AllowStorageFromVNET `
```

```
--nsg-name nsg-storage `
--resource-group rg-bbva-red `
--priority 100 `
--source-address-prefixes 10.0.0.0/16 `
--destination-port-ranges 443 `
--protocol Tcp `
--access Allow `
--direction Inbound `
--description "Permitir Storage solo desde VNET"
```

Denegar acceso público

```
az network nsg rule create `
--name DenyPublicAccess `
--nsg-name nsg-storage `
--resource-group rg-bbva-red `
--priority 4096 `
--source-address-prefixes Internet `
--destination-port-ranges '*' `
--protocol '*' `
--access Deny `
--direction Inbound `
--description "Denegar acceso público al Storage"
```

Asociar NSG a la subnet

```
az network vnet subnet update `
--name snet-storage `
--resource-group rg-bbva-red `
--vnet-name vnet-bbva-dashboard `
--network-security-group nsg-storage
```

NSG para Databricks

Crear NSG

```
az network nsg create `
--name nsg-databricks `
--resource-group rg-bbva-red `
--location eastus
```

Regla 1: Permitir comunicación con el control plane de Databricks

```
az network nsg rule create `
--name AllowDatabricksControlPlane `
--nsg-name nsg-databricks `
--resource-group rg-bbva-red `
--priority 100 `
--source-address-prefixes AzureDatabricks `
--destination-port-ranges 443 `
--protocol Tcp `
--access Allow `
--direction Inbound `
--description "Permitir comunicación con control plane de Databricks"
```

Regla 2: Permitir comunicación interna entre nodos de Databricks

```

az network nsg rule create `
  --name AllowDatabricksInternal `
  --nsg-name nsg-databricks `
  --resource-group rg-bbva-red `
  --priority 110 `
  --source-address-prefixes VirtualNetwork `
  --destination-address-prefixes VirtualNetwork `
  --destination-port-ranges '*' `
  --protocol '*' `
  --access Allow `
  --direction Inbound `
  --description "Permitir comunicación interna de Databricks"

```

Asociar NSG a ambas subnets de Databricks

```

az network vnet subnet update `
  --name snet-databricks-public `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --network-security-group nsg-databricks

az network vnet subnet update `
  --name snet-databricks-private `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --network-security-group nsg-databricks

```

NSG para Key Vault

Crear NSG

```

az network nsg create `
  --name nsg-keyvault `
  --resource-group rg-bbva-red `
  --location eastus

```

Permitir HTTPS solo desde la VNET

```

az network nsg rule create `
  --name AllowKeyVaultFromVNET `
  --nsg-name nsg-keyvault `
  --resource-group rg-bbva-red `
  --priority 100 `
  --source-address-prefixes 10.0.0.0/16 `
  --destination-port-ranges 443 `
  --protocol Tcp `
  --access Allow `
  --direction Inbound `
  --description "Permitir Key Vault solo desde VNET"

```

Denegar acceso público

```

az network nsg rule create `
  --name DenyPublicAccess `
  --nsg-name nsg-keyvault `
  --resource-group rg-bbva-red `

```

```
--priority 4096 `
--source-address-prefixes Internet `
--destination-port-ranges '*' `
--protocol '*' `
--access Deny `
--direction Inbound `
--description "Denegar acceso público al Key Vault"
```

Asociar NSG a la subnet

```
az network vnet subnet update `
--name snet-keyvault `
--resource-group rg-bbva-red `
--vnet-name vnet-bbva-dashboard `
--network-security-group nsg-keyvault
```

Private DNS Zone

DNS Zone para Storage Account (Blob)

Crear Private DNS Zone para Blob Storage

```
az network private-dns zone create `
--name privatelink.blob.core.windows.net `
--resource-group rg-bbva-red
```

Vincular la DNS Zone a la VNET

```
az network private-dns link vnet create `
--name link-blob-to-vnet `
--resource-group rg-bbva-red `
--zone-name privatelink.blob.core.windows.net `
--virtual-network vnet-bbva-dashboard `
--registration-enabled false
```

DNS Zone para Storage Account (DFS - Data Lake)

Crear Private DNS Zone para Data Lake Storage (DFS)

```
az network private-dns zone create `
--name privatelink.dfs.core.windows.net `
--resource-group rg-bbva-red
```

Vincular la DNS Zone a la VNET

```
az network private-dns link vnet create `
--name link-dfs-to-vnet `
--resource-group rg-bbva-red `
--zone-name privatelink.dfs.core.windows.net `
--virtual-network vnet-bbva-dashboard `
--registration-enabled false
```

DNS Zone para PostgreSQL

Crear Private DNS Zone para PostgreSQL

```
az network private-dns zone create `
  --name privatelink.postgres.database.azure.com `
  --resource-group rg-bbva-red
```

Vincular la DNS Zone a la VNET

```
az network private-dns link vnet create `
  --name link-postgres-to-vnet `
  --resource-group rg-bbva-red `
  --zone-name privatelink.postgres.database.azure.com `
  --virtual-network vnet-bbva-dashboard `
  --registration-enabled false
```

DNS Zone para Key Vault

Crear Private DNS Zone para Key Vault

```
az network private-dns zone create `
  --name privatelink.vaultcore.azure.net `
  --resource-group rg-bbva-red
```

Vincular la DNS Zone a la VNET

```
az network private-dns link vnet create `
  --name link-keyvault-to-vnet `
  --resource-group rg-bbva-red `
  --zone-name privatelink.vaultcore.azure.net `
  --virtual-network vnet-bbva-dashboard `
  --registration-enabled false
```

DNS Zone para Container Registry

Crear Private DNS Zone para ACR

```
az network private-dns zone create `
  --name privatelink.azurecr.io `
  --resource-group rg-bbva-red
```

Vincular la DNS Zone a la VNET

```
az network private-dns link vnet create `
  --name link-acr-to-vnet `
  --resource-group rg-bbva-red `
  --zone-name privatelink.azurecr.io `
  --virtual-network vnet-bbva-dashboard `
  --registration-enabled false
```

Private Endpoints

Deshabilitar acceso público a los servicios

Para las llaves

```
az keyvault update `
  --name kv-bbva-dashboard `
  --resource-group rg-bbva-dashboard `
  --public-network-access Disabled
```

Para el contenedor

```
az acr update `
  --name acrbbvadashboard `
  --resource-group rg-bbva-dashboard `
  --sku Premium
  --public-network-enabled false
```

Private Endpoint para Storage Account (Blob)

Deshabilitar políticas de red en la subnet (requerido para Private Endpoints)

```
az network vnet subnet update `
  --name snet-storage `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --disable-private-endpoint-network-policies true
```

Crear Private Endpoint para Blob

```
az network private-endpoint create `
  --name pe-storage-blob `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --subnet snet-storage `
  --private-connection-resource-id $(az storage account show --name
stbbvadalake --resource-group rg-bbva-dashboard --query id -o tsv) `
  --group-id blob `
  --connection-name conn-storage-blob `
  --location eastus
```

Crear registro DNS automático para Blob

```
az network private-endpoint dns-zone-group create `
  --name zg-storage-blob `
  --resource-group rg-bbva-red `
  --endpoint-name pe-storage-blob `
  --private-dns-zone privatelink.blob.core.windows.net `
  --zone-name blob
```

Private Endpoint para Storage Account (DFS - Data Lake)

Crear Private Endpoint para DFS

```
az network private-endpoint create `
  --name pe-storage-dfs `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --subnet snet-storage `
  --private-connection-resource-id $(az storage account show --name
stbbvadalake --resource-group rg-bbva-dashboard --query id -o tsv) `
```

```
--group-id dfs `
--connection-name conn-storage-dfs `
--location eastus
```

Crear registro DNS automático para DFS

```
az network private-endpoint dns-zone-group create `
--name zg-storage-dfs `
--resource-group rg-bbva-red `
--endpoint-name pe-storage-dfs `
--private-dns-zone privatelink.dfs.core.windows.net `
--zone-name dfs
```

Private Endpoint para PostgreSQL

Deshabilitar políticas de red en la subnet de PostgreSQL

```
az network vnet subnet update `
--name snet-postgresql `
--resource-group rg-bbva-red `
--vnet-name vnet-bbva-dashboard `
--disable-private-endpoint-network-policies true
```

Crear Private Endpoint para PostgreSQL

```
az network private-endpoint create `
--name pe-postgresql `
--resource-group rg-bbva-red `
--vnet-name vnet-bbva-dashboard `
--subnet snet-postgresql `
--private-connection-resource-id $(az postgres flexible-server show --name
pg-bbva-dashboard --resource-group rg-bbva-dashboard --query id -o tsv) `
--group-id postgresqlServer `
--connection-name conn-postgresql `
--location eastus
```

Crear registro DNS automático

```
az network private-endpoint dns-zone-group create `
--name zg-postgresql `
--resource-group rg-bbva-red `
--endpoint-name pe-postgresql `
--private-dns-zone privatelink.postgres.database.azure.com `
--zone-name postgres
```

Private Endpoint para Key Vault

Deshabilitar políticas de red en la subnet de Key Vault

```
az network vnet subnet update `
--name snet-keyvault `
--resource-group rg-bbva-red `
--vnet-name vnet-bbva-dashboard `
--disable-private-endpoint-network-policies true
```

Crear Private Endpoint para Key Vault

```
az network private-endpoint create `
  --name pe-keyvault `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --subnet snet-keyvault `
  --private-connection-resource-id $(az keyvault show --name kv-bbva-
dashboard --resource-group rg-bbva-dashboard --query id -o tsv) `
  --group-id vault `
  --connection-name conn-keyvault `
  --location eastus
```

Crear registro DNS automático

```
az network private-endpoint dns-zone-group create `
  --name zg-keyvault `
  --resource-group rg-bbva-red `
  --endpoint-name pe-keyvault `
  --private-dns-zone privatelink.vaultcore.azure.net `
  --zone-name vault
```

Private Endpoint para Container Registry

Crear Private Endpoint para ACR

```
az network private-endpoint create `
  --name pe-acr `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --subnet snet-storage `
  --private-connection-resource-id $(az acr show --name acrbbvadashboard --
resource-group rg-bbva-dashboard --query id -o tsv) `
  --group-id registry `
  --connection-name conn-acr `
  --location eastus
```

Crear registro DNS automático

```
az network private-endpoint dns-zone-group create `
  --name zg-acr `
  --resource-group rg-bbva-red `
  --endpoint-name pe-acr `
  --private-dns-zone privatelink.azurecr.io `
  --zone-name registry
```

Firewall

Crear IP Pública para el Firewall

El Firewall necesita una IP pública estática

```
az network public-ip create `
  --name pip-firewall `
  --resource-group rg-bbva-red `
  --location eastus `
  --allocation-method Static `
```

```
--sku Standard
```

Crear Azure Firewall Policy

Crear política de firewall

```
az network firewall policy create `
  --name policy-bbva-firewall `
  --resource-group rg-bbva-red `
  --location eastus `
  --sku Standard
```

Crear colección de reglas de aplicación (URLs permitidas)

```
az network firewall policy rule-collection-group create `
  --name rcg-app-rules `
  --policy-name policy-bbva-firewall `
  --resource-group rg-bbva-red `
  --priority 100
```

Regla 1: Permitir acceso a servicios de Azure

```
az network firewall policy rule-collection-group collection add-filter-
collection `
  --name rc-allow-azure `
  --policy-name policy-bbva-firewall `
  --resource-group rg-bbva-red `
  --rcg-name rcg-app-rules `
  --collection-priority 100 `
  --action Allow `
  --rule-name AllowAzureServices `
  --rule-type ApplicationRule `
  --target-fqdns "*.azure.com" "*.microsoft.com" "*.windows.net" `
  --source-addresses 10.0.0.0/16 `
  --protocols Https=443
```

Regla 2: Permitir acceso a Databricks

```
az network firewall policy rule-collection-group collection rule add `
  --name AllowDatabricks `
  --policy-name policy-bbva-firewall `
  --resource-group rg-bbva-red `
  --rcg-name rcg-app-rules `
  --collection-name rc-allow-azure `
  --rule-type ApplicationRule ` rule-type
  --target-fqdns "*.databricks.net" "*.azuredatabricks.net" `
  --source-addresses 10.0.3.0/24 10.0.4.0/24 `
  --protocols Https=443
```

Regla 3: Permitir npm y paquetes de Python (para Databricks)

```
az network firewall policy rule-collection-group collection rule add `
  --name AllowPackages `
  --policy-name policy-bbva-firewall `
  --resource-group rg-bbva-red `
  --rcg-name rcg-app-rules `
  --collection-name rc-allow-azure `
  --rule-type ApplicationRule `
```

```
--target-fqdns "*.pypi.org" "*.npmjs.org" "*.github.com" `
--source-addresses 10.0.0.0/16 `
--protocols Https=443
```

Regla 4: Permitir el dominio del Container App

```
az network firewall policy rule-collection-group collection rule add `
--name AllowContainerAppBackend `
--policy-name policy-bbva-firewall `
--resource-group rg-bbva-red `
--rcg-name rcg-app-rules `
--collection-name rc-allow-azure `
--rule-type ApplicationRule `
--target-fqdns "*.azurecontainerapps.io" `
--source-addresses 10.0.0.0/16 `
--protocols Https=443
```

Regla 5: Permitir tráfico desde el dominio del Static Web App

```
az network firewall policy rule-collection-group collection rule add `
--name AllowStaticWebApp `
--policy-name policy-bbva-firewall `
--resource-group rg-bbva-red `
--rcg-name rcg-app-rules `
--collection-name rc-allow-azure `
--rule-type ApplicationRule `
--target-fqdns "*.azurestaticapps.net" `
--source-addresses "*" `
--protocols Https=443
```

Crear el Azure Firewall (NO RECOMENDADO)

Crear el Firewall

```
az network firewall create `
--name fw-bbva-dashboard `
--resource-group rg-bbva-red `
--location eastus `
--vnet-name vnet-bbva-dashboard `
--firewall-policy policy-bbva-firewall
```

Asociar la IP pública al Firewall

```
az network firewall ip-config create `
--name fw-config `
--firewall-name fw-bbva-dashboard `
--resource-group rg-bbva-red `
--vnet-name vnet-bbva-dashboard `
--public-ip-address pip-firewall
```

Obtener la IP privada del Firewall

```
$FIREWALL_PRIVATE_IP = az network firewall show `
--name fw-bbva-dashboard `
--resource-group rg-bbva-red `
--query 'ipConfigurations[0].privateIPAddress' -o tsv
```

```
Write-Host "Firewall Private IP: $FIREWALL_PRIVATE_IP"
```

Crear Route Table para dirigir tráfico al Firewall
 Crear Route Table

```
az network route-table create `
  --name rt-firewall `
  --resource-group rg-bbva-red `
  --location eastus
```

Crear ruta: Todo el tráfico de salida va al Firewall

```
az network route-table route create `
  --name route-to-firewall `
  --resource-group rg-bbva-red `
  --route-table-name rt-firewall `
  --address-prefix 0.0.0.0/0 `
  --next-hop-type VirtualAppliance `
  --next-hop-ip-address $FIREWALL_PRIVATE_IP
```

Asociar Route Table a las subnets

```
az network vnet subnet update `
  --name snet-containerapp `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --route-table rt-firewall
```

```
az network vnet subnet update `
  --name snet-databricks-public `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --route-table rt-firewall
```

```
az network vnet subnet update `
  --name snet-databricks-private `
  --resource-group rg-bbva-red `
  --vnet-name vnet-bbva-dashboard `
  --route-table rt-firewall
```

Configurar Diagnostic Settings para el Firewall (enviar logs a Log Analytics)

```
$FIREWALL_ID = az network firewall show `
  --name fw-bbva-dashboard `
  --resource-group rg-bbva-red `
  --query id -o tsv
```

```
$LAW_ID = az monitor log-analytics workspace show `
  --resource-group rg-bbva-dashboard `
  --workspace-name law-bbva-dashboard `
  --query id -o tsv
```

```
az monitor diagnostic-settings create `
  --name diag-firewall `
  --resource $FIREWALL_ID `
  --workspace $LAW_ID `
```

```
--logs
'[{ "category": "AzureFirewallApplicationRule", "enabled": true }, { "category": "AzureFirewallNetworkRule", "enabled": true }]'
--metrics ' [{ "category": "AllMetrics", "enabled": true } ]'
```

Algunas reglas adicionales

Esto hace que el firewall deje pasar la subida de archivos.

```
az network firewall policy rule-collection-group collection rule add `
--name AllowBlobStorage `
--policy-name policy-bbva-firewall `
--resource-group rg-bbva-red `
--rcg-name rcg-app-rules `
--collection-name rc-allow-azure `
--rule-type ApplicationRule `
--target-fqdns "*.blob.core.windows.net" "*.dfs.core.windows.net" `
--source-addresses 10.0.0.0/16 `
--protocols Https=443
```

Asignar permisos a la identidad del Container App

```
$CONTAINER_APP_PRINCIPAL_ID = az containerapp show `
--name bbva-backend-api `
--resource-group rg-bbva-dashboard `
--query identity.principalId -o tsv

az role assignment create `
--assignee $CONTAINER_APP_PRINCIPAL_ID `
--role "Storage Blob Data Contributor" `
--scope $(az storage account show --name stbbvadalake --resource-group rg-bbva-dashboard --query

az network firewall policy rule-collection-group collection rule add `
--name AllowStorageADLS `
--policy-name policy-bbva-firewall `
--resource-group rg-bbva-red `
--rcg-name rcg-app-rules `
--collection-name rc-allow-azure `
--rule-type ApplicationRule `
--target-fqdns "*.blob.core.windows.net" "*.dfs.core.windows.net"
"login.microsoftonline.com" "*.login.microsoftonline.com"
"*.aadcdn.microsoftonline-p.com" `
--source-addresses 10.0.0.0/16 `
--protocols Https=443

az network firewall policy rule-collection-group collection rule add `
--name AllowAzureManagement `
--policy-name policy-bbva-firewall `
--resource-group rg-bbva-red `
--rcg-name rcg-app-rules `
--collection-name rc-allow-azure `
--rule-type ApplicationRule `
--target-fqdns "management.azure.com" `
--source-addresses 10.0.0.0/16 `
--protocols Https=443
```

2.16. Servicios Adicionales

Habilitar identidad administrada (System Assigned)

```
az dataprotection backup-vault update `
  --resource-group rg-bbva-dashboard `
  --vault-name rg-bbva-dashboard-backup `
  --set identity.type="SystemAssigned"
```

Activar Soft Delete

```
az storage account blob-service-properties update `
  --account-name stbbvadalake `
  --resource-group rg-bbva-dashboard `
  --enable-delete-retention true `
  --delete-retention-days 30
```

Container Soft Delete

```
az storage account blob-service-properties update `
  --account-name stbbvadalake `
  --resource-group rg-bbva-dashboard `
  --enable-container-delete-retention true `
  --container-delete-retention-days 30
```

Restore Status

```
az storage container restore-status show `
  --account-name stbbvadalake `
  --resource-group rg-bbva-dashboard
```

Account Redundancy (GRS check)

```
az storage account show `
  --name stbbvadalake `
  --resource-group rg-bbva-dashboard `
  --query sku.name -o tsv
```

3. ARCHIVOS UTILIZADOS

3.1. Notebooks

Los notebooks constituyen la pieza principal del ETL en la nube. Se ejecutan en Azure Databricks y realizan tareas de limpieza, estandarización, transformación y cálculo de KPIs, siguiendo el patrón Medallion:

- **Bronze (raw):** datos crudos tal como llegan desde los CSV.
- **Silver (trusted):** datos limpios y tipificados.
- **Gold (refined):** datos curados/listos para consumo y para carga al modelo estrella en PostgreSQL.

En el proyecto se separan notebooks para Practitioner y Continuous Integration (CI), y dentro de cada dominio se mantiene la secuencia: Limpiar → Transformar/Cargar a DW → Vista (KPIs).

3.1.1. Practitioner Notebooks

A continuación, se describen los notebooks del dominio Practitioner, organizados en el orden en que se ejecutan dentro del pipeline (Bronze → Silver → PostgreSQL → Vista de KPIs).

A) Notebook: Limpiar Practitioner (Bronze → Silver)

El objetivo de este notebook es leer el CSV “sucio” desde la capa Bronze del Data Lake, aplicar reglas de depuración/estandarización y generar un único archivo limpio en Silver para el resto del pipeline. Este notebook corresponde a la etapa de “limpieza y validación” del ETL ejecutado en Databricks.

a) Entrada (Bronze)

```
# Databricks notebook source
from pyspark.sql import functions as F
from pyspark.sql.types
import DoubleType, StringType import re

# Rutas ADLS
input_path =
"abfss://bronze@stbbvadalake.dfs.core.windows.net/data_sucia/data_sucia_practitioner"
temp_path =
"abfss://silver@stbbvadalake.dfs.core.windows.net/data_limpiar/tmp_practitioner"
final_dir =
"abfss://silver@stbbvadalake.dfs.core.windows.net/data_limpiar/data_limpiar_practitioner"
final_file = f"{final_dir}/data_limpiar_practitioner.csv"

# Leer CSV desde Bronze
df = spark.read.option("header", True).csv(input_path)
```

b) Salida (Silver)

Se escribe primero a una ruta temporal y luego se consolida en un solo CSV mediante `coalesce(1)` y movimiento/renombrado de `part-*.csv`

```
# Guardar TEMPORAL (Spark crea muchos archivos)
df.coalesce(1).write.mode("overwrite").option("header", True).csv(temp_path)

# Encontrar el archivo part-*.csv y renombrarlo
files = dbutils.fs.ls(temp_path)
csv_temp = [f.path for f in files if f.name.endswith(".csv")][0]
```

c) Transformaciones principales

Eliminación de columnas no deseadas: Se descartan métricas no requeridas para el modelo final (ej. columnas relacionadas a “vulnerabilidades mes anterior”, “variación”, etc.).

```
# Eliminar columnas no deseadas
# =====
cols_a_eliminar = [
    "Vulnerab. de alto riesgo por cada mil líneas de código",
    "Vulnerab. de alto riesgo por cada mil líneas de código mes anterior",
    "Variación de la evolución de vulnerabilidades",
    "Nº de vulnerabilidades high en el mes anterior",
    "Líneas de código en el mes anterior"
]
df = df.drop(*[c for c in cols_a_eliminar if c in df.columns])
```

Renombrado (estandarización de esquema): Se unifican nombres de columnas (por ejemplo: `period_month` → `mes`, `ug_name` → `geografia`, `servicell_id` → `service1_id`, métricas numéricas a nombres `snake_case`).

```
# Renombrar columnas finales
# =====
cols_finales = {
    "period_month": "mes",
    "ug_name": "geografia",
    "uol1_name": "uol1_name",
    "uol2_name": "uol2_name",
    "servicell1_id": "service1_id",
    "servicell1_name": "service1_name",
    "Nº fichas RFO": "nro_fichas_rfo",
    "Nº SN2 con ficha RFO con status OK": "nro_fichas_rfo_ok",
    "Nº SN2 con dependencias Consumer": "nro_sn2_sn1_dependencias",
    "Nº SN2 del tipo Aplicación no Hijo": "nro_sn2_sn1",
    "Número de features deployed válidas":
    "nro_features_desplegadas_calidad",
    "Número de features deployed": "nro_features_desplegadas",
    "Valor de cumplimiento de objetivos en Op Model":
    "puntaje_total_adopcion_sn2",
    "Servicios N2 con medición en el modelo objetivo del Op Model":
    "sn2_sn1_medidos",
    "Nº de vulnerabilidades high": "total_vulnerabilidades_high",
    "Nº de Lineas de código": "total_lineas_codigo"
}
for old, new in cols_finales.items():
    if old in df.columns:
        df = df.withColumnRenamed(old, new)
```

Normalización de fechas (columna mes): Se implementa una UDF `transformar_mes` para convertir formatos tipo `mar-25`, `01-Jul`, etc., a un formato estándar `MM/DD/YYYY` (considerando abreviaturas en español e inglés).

```
# Transformación del mes
# =====
def transformar_mes(valor):
    if not valor:
        return valor
    meses = {
        "jan": "01", "feb": "02", "mar": "03", "apr": "04", "may": "05", "jun": "06",
        "jul": "07", "aug": "08", "sep": "09", "oct": "10", "nov": "11", "dec": "12",
        "ene": "01", "abr": "04", "ago": "08", "dic": "12"
    }
    valor = valor.strip().lower()
    import re
```

```

match = re.match(r"(\d{1,2})-([a-z]{3})", valor)
if match:
    dia, mes_str = match.groups()
    mes_num = meses.get(mes_str[:3], "01")
    return f"{mes_num}/25/2025"
match = re.match(r"([a-zñ+)-(\d{2})", valor)
if match:
    mes_str, anio = match.groups()
    mes_num = meses.get(mes_str[:3], "01")
    return f"{mes_num}/25/20{anio}"
return valor

spark.udf.register("transformar_mes", transformar_mes)
df = df.withColumn("mes", F.expr("transformar_mes(mes)"))

```

Limpieza de columnas numéricas: Se eliminan separadores de miles (por ejemplo “.”) y se castea a DoubleType para asegurar consistencia en cálculos posteriores.

```

# Limpiar enteros
# =====
cols_enteros = [

    "nro_fichas_rfo", "nro_fichas_rfo_ok", "nro_sn2_sn1_dependencias", "nro_sn2_sn1",
    "nro_features_desplegadas_calidad", "nro_features_desplegadas",
    "sn2_sn1_medidos", "total_vulnerabilidades_high", "total_lineas_codigo"
]

for col in cols_enteros:
    if col in df.columns:
        df = df.withColumn(col, F.regexp_replace(col, r"\.",
        "").cast(DoubleType()))

```

Ordenamiento: Se ordena por mes, geografía, uol1_name, uol2_name, service1_id, facilitando auditoría visual y consistencia de salida.

```

# Ordenar
# =====
df = df.orderBy("mes", "geografia", "uol1_name", "uol2_name", "service1_id")

```

Resultado. Dataset “trusted” (Silver) listo para construir dimensiones/hechos y cargar al Data Warehouse.

```

# Mover a ruta final
dbutils.fs.rm(final_dir, recurse=True)
dbutils.fs.mkdirs(final_dir)
dbutils.fs.mv(csv_temp, final_file)

# Borrar temporal
dbutils.fs.rm(temp_path, recurse=True)

print("✓ Limpieza completa y archivo final creado:", final_file)

```

B) Notebook: Transformar Practitioner

Su objetivo es tomar el CSV limpio desde Silver y construir el modelo dimensional (estrella) en PostgreSQL administrado: dimensiones (dim_tiempo, dim_geografia,

dim_servicio_n1) y la tabla de hechos (fact_mediciones_practitioner). Esta fase se alinea con la carga hacia el DW relacional descrito en la arquitectura cloud.

a) Entrada (Silver)

```
silver_path = "abfss://silver@stbbvadalake.dfs.core.windows.net"
input_path =
f"{silver_path}/data_limpiar/data_limpiar_practitioner/data_limpiar_practitioner
.csv"
```

b) Salida (DW en PostgreSQL)

- dim_tiempo
- dim_geografia
- dim_servicio_n1
- fact_mediciones_practitioner

c) Transformaciones y lógica

Lectura e inferencia de esquema desde Silver.

```
# LEER CSV DESDE SILVER
# =====
df = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(input_path)
)
```

Construcción de dimensiones:

Dimensión Tiempo (dim_tiempo): a partir de mes se genera fecha, día, mes, año y se ordena cronológicamente.

```
# DIMENSION TIEMPO
# =====
dim_tiempo = (
    df.select(F.col("mes").alias("fecha"))
    .distinct()
    .withColumn("dia", F.dayofmonth("fecha"))
    .withColumn("mes", F.month("fecha"))
    .withColumn("anio", F.year("fecha"))
    .orderBy("fecha")
)
```

Dimensión Geografía (dim_geografia): lista distinta de geografías (renombrada como nombre_geografia).

```
# DIMENSION GEOGRAFIA
# =====
dim_geografia = (
    df.select("geografia")
    .distinct()
    .withColumnRenamed("geografia", "nombre_geografia")
)
```

)

Dimensión Servicio N1 (dim_servicio_n1): se deduplican servicios por servicio_n1_id e incluye atributos (nombre_servicio, nombre_uol1, nombre_uol2).

```
# DIMENSION SERVICIO N1
# =====
dim_servicio_n1 = (
    df.select(
        F.col("service1_id").cast("string").alias("servicio_n1_id"),
        F.col("service1_name").alias("nombre_servicio"),
        F.col("uol1_name").alias("nombre_uol1"),
        F.col("uol2_name").alias("nombre_uol2")
    )
    .dropDuplicates(["servicio_n1_id"])
)
```

Carga controlada (reinicio del DW). Antes de insertar, el notebook ejecuta TRUNCATE ... RESTART IDENTITY CASCADE sobre las tablas objetivo para asegurar reproducibilidad del DW (idempotencia por ejecución).

```
# LIMPIAR TABLAS (TRUNCATE)
# =====
import psycopg2

conn = psycopg2.connect(
    host="pg-bbva-dashboard.postgres.database.azure.com",
    database="data_oro_practitioner",
    user="adminuser",
    password="SecurePass123!",
    sslmode="require"
)

cur = conn.cursor()

cur.execute("TRUNCATE TABLE fact_mediciones_practitioner RESTART IDENTITY CASCADE;")
cur.execute("TRUNCATE TABLE dim_tiempo RESTART IDENTITY CASCADE;")
cur.execute("TRUNCATE TABLE dim_geografia RESTART IDENTITY CASCADE;")
cur.execute("TRUNCATE TABLE dim_servicio_n1 RESTART IDENTITY CASCADE;")

conn.commit()
cur.close()
conn.close()
```

Insertión de dimensiones por JDBC (modo append).

```
# INSERTAR DIMENSIONES EN POSTGRESQL
# =====
dim_tiempo.write.jdbc(jdbc_url, "dim_tiempo", mode="append",
    properties=properties)
dim_geografia.write.jdbc(jdbc_url, "dim_geografia", mode="append",
    properties=properties)
dim_servicio_n1.write.jdbc(jdbc_url, "dim_servicio_n1", mode="append",
    properties=properties)
```

Recuperación de IDs: se releen las dimensiones desde PostgreSQL para obtener claves sustitutas generadas (IDs).

```
# LEER DIMENSIONES DESDE POSTGRES CON ID ASIGNADOS
# =====
dim_tiempo_psql = spark.read.jdbc(jdbc_url, "dim_tiempo",
properties=properties)
dim_geografia_psql = spark.read.jdbc(jdbc_url, "dim_geografia",
properties=properties)
dim_servicio_n1_psql = spark.read.jdbc(jdbc_url, "dim_servicio_n1",
properties=properties)
```

Creación de tabla de hechos (fact): se normalizan llaves (fecha, geografia_norm, service1_id_str) y se realizan joins contra dimensiones para derivar:

- tiempo_id
- geografia_id
- servicio_n1_id
- métricas numéricas del dataset (RFO, dependencias, features, vulnerabilidades, líneas de código, etc.)

```
# CREAR TABLA DE HECHOS
# =====
fact = (
    df_norm
    .join(dim_tiempo_psql, df_norm["fecha"] == dim_tiempo_psql["fecha"],
"inner")
    .join(dim_g_alias, df_norm["geografia_norm"] ==
dim_g_alias["geografia_norm"], "inner")
    .join(dim_s_alias, df_norm["service1_id_str"] ==
dim_s_alias["servicio_n1_id_str"], "inner")
    .select(
        dim_tiempo_psql["tiempo_id"],
        dim_g_alias["geografia_id"],
        dim_s_alias["servicio_n1_id"],
        df_norm["nro_fichas_rfo"],
        df_norm["nro_fichas_rfo_ok"],
        df_norm["nro_sn2_sn1_dependencias"],
        df_norm["nro_sn2_sn1"],
        df_norm["nro_features_desplegadas_calidad"],
        df_norm["nro_features_desplegadas"],
        df_norm["puntaje_total_adopcion_sn2"],
        df_norm["sn2_sn1_medidos"],
        df_norm["total_vulnerabilidades_high"],
        df_norm["total_lineas_codigo"]
    )
)
```

Insertión de hechos en fact_mediciones_practitioner por JDBC.

```
# INSERTAR HECHOS EN POSTGRESQL
# =====
fact.write.jdbc(jdbc_url, "fact_mediciones_practitioner", mode="append",
properties=properties)
```

Nota de seguridad. Aunque el código muestra credenciales en texto plano, en la arquitectura objetivo los secretos y cadenas de conexión deben gestionarse con **Azure Key Vault** (o Secret Scopes de Databricks vinculados a Key Vault), evitando credenciales hardcodeadas.

C) Notebook: Vista Practitioner

Su objetivo es generar una vista de negocio con KPIs listos para consumo por el dashboard/API: el notebook lee tablas del modelo estrella en PostgreSQL, calcula indicadores y publica una vista vw_practitioner_kpis. Este tipo de notebook corresponde a la etapa “Vista”, que en el patrón del proyecto encapsula KPIs consultables por la capa de aplicación.

a) Entradas (PostgreSQL)

- fact_mediciones_practitioner
- dim_tiempo
- dim_geografia
- dim_servicio_n1

```
# Databricks notebook source
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, when, round as spark_round, lag, lit, sum as Fsum,
    concat_ws, to_date, monotonically_increasing_id
)
from pyspark.sql.window import Window
import psycopg2

# CONFIG – CONEXIÓN AZURE POSTGRESQL
# =====

pg_url = "jdbc:postgresql://pg-bbva-
dashboard.postgres.database.azure.com:5432/data_oro_practitioner"
pg_properties = {
    "user": "adminuser",
    "password": "SecurePass123!",
    "driver": "org.postgresql.Driver",
    "sslmode": "require"
}

# LEER TABLAS DEL DATAWAREHOUSE (EN POSTGRESQL)
# =====

fact = spark.read.jdbc(pg_url, "fact_mediciones_practitioner",
    properties=pg_properties)
dim_tiempo = spark.read.jdbc(pg_url, "dim_tiempo", properties=pg_properties)
dim_geografia = spark.read.jdbc(pg_url, "dim_geografia",
    properties=pg_properties)
dim_servicio = spark.read.jdbc(pg_url, "dim_servicio_n1",
    properties=pg_properties)
```

b) Proceso

Join del modelo estrella (hechos + dimensiones) para obtener un dataset analítico con: tiempo (mes/año), geografía, servicio y métricas base.

```
# UNIR FACT + DIMENSIONES (MODELO ESTRELLA)
# =====

df = (
    fact.alias("f")
    .join(dim_tiempo.alias("t"), col("f.tiempo_id") == col("t.tiempo_id"))
    .join(dim_geografia.alias("g"), col("f.geografia_id") ==
col("g.geografia_id"))
    .join(dim_servicio.alias("s"), col("f.servicio_n1_id") ==
col("s.servicio_n1_id"))
    .select(
        col("t.mes"),
        col("t.anio"),
        col("g.nombre_geografia").alias("geografia"),
        col("s.nombre_servicio").alias("service1_name"),

        col("f.nro_fichas_rfo"),
        col("f.nro_fichas_rfo_ok"),
        col("f.nro_sn2_sn1"),
        col("f.nro_sn2_sn1_dependencias"),
        col("f.sn2_sn1_medidos"),
        col("f.puntaje_total_adopcion_sn2"),
        col("f.nro_features_desplegadas"),
        col("f.nro_features_desplegadas_calidad"),
        col("f.total_vulnerabilidades_high"),
        col("f.total_lineas_codigo")
    )
)
```

Construcción de fecha (formato YYYY-MM-01) para ordenar series temporales por mes.

```
# CREAR FECHA COMPLETA YYYY-MM-01
# =====

df = df.withColumn(
    "fecha",
    to_date(concat_ws("-", col("anio"), col("mes"), lit("01")), "yyyy-M-dd")
)
```

Cálculo de KPIs principales:

- **rfo_ok_pct** = $(\text{nro_fichas_rfo_ok} / \text{nro_fichas_rfo}) \times 100$
- **dep_pct** = $(\text{nro_sn2_sn1_dependencias} / \text{nro_sn2_sn1}) \times 100$
- **adopcion_sn2_pct** = $(\text{puntaje_total_adopcion_sn2} / \text{sn2_sn1_medidos}) \times 100$
- **calidad_features_pct** = $(\text{nro_features_desplegadas_calidad} / \text{nro_features_desplegadas}) \times 100$
- **vuln_actual** = $(\text{total_vulnerabilidades_high} / \text{total_lineas_codigo}) \times 1000$
(vulnerabilidades por mil líneas).

```
# CALCULAR KPIs
```

```
# =====

df_calc = (
    df
    .withColumn("rfo_ok_pct",
        when(col("nro_fichas_rfo") > 0,
            spark_round(col("nro_fichas_rfo_ok") / col("nro_fichas_rfo") *
100, 2)
        )
    )
    .withColumn("dep_pct",
        when(col("nro_sn2_sn1") > 0,
            spark_round(col("nro_sn2_sn1_dependencias") / col("nro_sn2_sn1")
* 100, 2)
        )
    )
    .withColumn("adopcion_sn2_pct",
        when(col("sn2_sn1_medidos") > 0,
            spark_round(col("puntaje_total_adopcion_sn2") /
col("sn2_sn1_medidos") * 100, 2)
        )
    )
    .withColumn("calidad_features_pct",
        when(col("nro_features_desplegadas") > 0,
            spark_round(col("nro_features_desplegadas_calidad") /
col("nro_features_desplegadas") * 100, 2)
        )
    )
    .withColumn("vuln_actual",
        when(col("total_lineas_codigo") > 0,
            col("total_vulnerabilidades_high") / col("total_lineas_codigo")
* 1000
        )
    )
)
```

Cálculo con mes anterior (ventana): para cada par (geografia, service1_name) se usa lag() para obtener métricas del mes previo y calcular:

- **vuln_anterior** (si aplica)
- **evol_vuln** = $((\text{vuln_actual} - \text{vuln_anterior}) / \text{vuln_anterior}) \times 100$

```
# NECESITA MES ANTERIOR
# =====

df_calc = df_calc.withColumn(
    "necesita_mes_anterior",
    when((col("total_lineas_codigo") <= 0)
        | (col("vuln_actual") < 0.04)
        | (col("vuln_actual") > 0.2),
        False
    ).otherwise(True)
)

# =====
# CALCULO CON MES ANTERIOR
# =====
```

```

window_prev = Window.partitionBy("geografia",
"service1_name").orderBy("fecha")

df_calc = (
    df_calc
    .withColumn("total_vulnerabilidades_high_prev",
        when(col("necesita_mes_anterior"),
            lag("total_vulnerabilidades_high").over(window_prev)
        )
    )
    .withColumn("total_lineas_codigo_prev",
        when(col("necesita_mes_anterior"),
            lag("total_lineas_codigo").over(window_prev)
        )
    )
    .withColumn("vuln_anterior",
        when(~col("necesita_mes_anterior"), None)
        .when(col("total_lineas_codigo_prev").isNull(), lit(-99))
        .when(col("total_lineas_codigo_prev") <= 0, lit(-99))
        .otherwise(col("total_vulnerabilidades_high_prev") /
col("total_lineas_codigo_prev") * 1000)
    )
    .withColumn("evol_vuln",
        when(~col("necesita_mes_anterior"), None)
        .when(col("vuln_anterior") == -99, None)
        .when(col("vuln_anterior") == 0, None)
        .otherwise((col("vuln_actual") - col("vuln_anterior")) /
col("vuln_anterior") * 100)
    )
)

```

KPI de seguridad (seguridad_pct): se asigna un porcentaje según umbrales (por ejemplo, 100% si vuln_actual es muy bajo, 0% si es muy alto; en rangos intermedios se pondera por la evolución vs mes anterior).

```
# KPI SEGURIDAD
```

```
# =====
```

```

df_calc = df_calc.withColumn(
    "seguridad_pct",
    when(col("total_lineas_codigo") <= 0, None)
    .when(col("vuln_actual") < 0.04, lit(100))
    .when(col("vuln_actual") > 0.2, lit(0))
    .when(col("vuln_anterior") == -99, None)
    .when(col("evol_vuln").isNull(), None)
    .when(col("evol_vuln") == -100, lit(0))
    .when(col("evol_vuln") <= -10, lit(100))
    .when((col("evol_vuln") > -10) & (col("evol_vuln") < 0), lit(75))
    .when(col("evol_vuln") == 0, lit(50))
    .when((col("evol_vuln") > 0) & (col("evol_vuln") < 10), lit(25))
    .when(col("evol_vuln") >= 10, lit(0))
)

```

Adopción total ponderada (adopcion_total_pct): promedio ponderado considerando solo KPIs no nulos, con pesos:

- rfo_ok_pct: 13

- dep_pct: 13
- adopcion_sn2_pct: 27
- calidad_features_pct: 20
- seguridad_pct: 27

```
# ADOPCIÓN TOTAL (PONDERADA)
# =====

pesos = {
    "rfo_ok_pct": 13,
    "dep_pct": 13,
    "adopcion_sn2_pct": 27,
    "calidad_features_pct": 20,
    "seguridad_pct": 27
}

df_calc = df_calc.withColumn(
    "suma_pesada",
    sum(when(col(k).isNotNull(), col(k) * v).otherwise(0) for k, v in
pesos.items())
).withColumn(
    "suma_pesos",
    sum(when(col(k).isNotNull(), lit(v)).otherwise(0) for k, v in
pesos.items())
).withColumn(
    "adopcion_total_pct",
    when(col("suma_pesos") > 0,
        spark_round(col("suma_pesada") / col("suma_pesos"), 2)
    ).otherwise(None)
)
```

Publicación en PostgreSQL:

- Se escribe una tabla física table_practitioner_kpis (overwrite).
- Se crea/reemplaza la vista vw_practitioner_kpis como SELECT * FROM table_practitioner_kpis;

Salida (para consumo del dashboard)

- Vista final: vw_practitioner_kpis (dataset con fecha, geografia, service1_name y KPIs calculados).

```
# TABLA FINAL PARA LA VISTA
# =====

vista_df = df_calc.select(
    "fecha",
    "geografia",
    "service1_name",
    "rfo_ok_pct",
    "dep_pct",
    "adopcion_sn2_pct",
    "calidad_features_pct",
    "seguridad_pct",
    "adopcion_total_pct"
)
```

```
# =====
# CREAR / REEMPLAZAR VISTA NORMAL
# =====

def run_pg(query):
    conn = psycopg2.connect(
        host="pg-bbva-dashboard.postgres.database.azure.com",
        port=5432,
        database="data_oro_practitioner",
        user="adminuser",
        password="SecurePass123!",
        sslmode="require"
    )
    cur = conn.cursor()
    cur.execute(query)
    conn.commit()
    cur.close()
    conn.close()

# Borrar vista sin afectar tabla
run_pg("DROP VIEW IF EXISTS vw_practitioner_kpis;")

# Crear tabla base y luego vista
vista_df.write.jdbc(
    url=pg_url,
    table="table_practitioner_kpis",
    mode="overwrite",
    properties=pg_properties
)

run_pg("""
    CREATE VIEW vw_practitioner_kpis AS
    SELECT * FROM table_practitioner_kpis;
""")

print("✓ Vista creada correctamente: vw_practitioner_kpis")
```

3.1.2. Continuous Integration Notebooks

A) Notebook: Limpiar Continuous Integration

Su objetivo es leer el CSV crudo desde Bronze, estandarizar el esquema (renombrado de columnas), normalizar el campo de fecha (*mes*) y tipificar/corregir columnas numéricas para generar un CSV limpio en Silver.

a) Entrada (Bronze)

```
# Databricks notebook source
from pyspark.sql import functions as F
from pyspark.sql.types import DoubleType, StringType
import re, time

input_path =
"abfss://bronze@stbbvadalake.dfs.core.windows.net/data_sucia/data_sucia_con
tinuous_integration"
```

```
temp_path =
"abfss://silver@stbbvadalake.dfs.core.windows.net/tmp_continuous_integration"
"
```

Salida (Silver)

```
final_dir =
"abfss://silver@stbbvadalake.dfs.core.windows.net/data_limpiar/data_limpiar_continuous_integration"
final_file = f"{final_dir}/data_limpiar_continuous_integration.csv"
```

c) Transformaciones principales

Lectura del CSV con cabecera y delimitador coma.

```
df = spark.read.option("header", True).option("delimiter",
",").csv(input_path)
```

Renombrado y estandarización del esquema: se homogenizan nombres a *snake_case* y se alinean con la nomenclatura del modelo, por ejemplo:

- period_month → mes, ug_name → geografia, servicell_id → service1_id, servicell_name → service1_name.
- issues_review_menor_7_dias → issues_analysis_in_review_menor_7_dias.
- pipelines_ejecuciones_exitosas → ejecuciones_exitosas_pipelines, etc.

```
# RENAME COLUMNS
# =====
cols_finales = {
    "period_month": "mes",
    "ug_name": "geografia",
    "uol1_name": "uol1_name",
    "uol2_name": "uol2_name",
    "servicell_id": "service1_id",
    "servicell_name": "service1_name",
    "issues_review_menor_7_dias": "issues_analysis_in_review_menor_7_dias",
    "issues_en_revision_total": "issues_analysis_in_review",
    "historias_fix_version_deploy": "historias_deployed_fix_version",
    "cantidad_historias_deploy": "nro_historias_deployed",
    "repos_activos_con_nomenclatura_estandar":
"repositorios_activos_nomenclatura_estandar",
    "total_repositorios_en_uso": "total_repositorios_activos",
    "promedio_tiempo_aprobacion_pr": "tiempo_medio_aprobacion_pr",
    "promedio_tamano_pr": "tamano_medio_pr",
    "repos_gobernados_chimera_activos":
"repositorios_activos_gobernados_chimera",
    "promedio_tiempo_integracion": "tiempo_medio_integracion",
    "promedio_tiempo_builds": "tiempo_medio_construcciones",
    "pipelines_ejecuciones_exitosas": "ejecuciones_exitosas_pipelines",
    "pipelines_ejecuciones_totales": "ejecuciones_totales_pipelines",
    "promedio_tiempo_reparacion_builds": "tiempo_medio_reparacion_builds",
    "repos_sonarqube_ok_activos": "repositorios_activos_sonarqube_ok",
    "repos_conectados_sonarqube": "repositorios_conectados_sonarqube",
    "items_pruebas_desplegados_xray": "items_pruebas_desplegados_xray",
    "items_desplegados_totales": "items_desplegados_totales"
}

for old, new in cols_finales.items():
```

```
if old in df.columns:
    df = df.withColumnRenamed(old, new)
```

Normalización de fechas (columna mes): se usa una UDF transformar_mes para convertir formatos tipo 01-Jul o mar-25 a MM/DD/YYYY usando abreviaturas en español/inglés.

```
# UDF FECHA
# =====
def transformar_mes(valor):
    if not valor:
        return valor

    meses = {
        "jan": "01", "feb": "02", "mar": "03", "apr": "04", "may": "05", "jun": "06",
        "jul": "07", "aug": "08", "sep": "09", "oct": "10", "nov": "11", "dec": "12",
        "ene": "01", "abr": "04", "ago": "08", "dic": "12"
    }

    valor = valor.strip().lower()

    m1 = re.match(r"(\d{1,2})-([a-zñ]{3})", valor)
    if m1:
        _, mes = m1.groups()
        return f"{meses.get(mes, '01')}/25/2025"

    m2 = re.match(r"([a-zñ]{3})-(\d{2})", valor)
    if m2:
        mes, anio = m2.groups()
        return f"{meses.get(mes, '01')}/25/20{anio}"

    return valor

spark.udf.register("transformar_mes", transformar_mes, StringType())
df = df.withColumn("mes", F.expr("transformar_mes(mes)"))
```

Limpieza y tipificación de numéricos:

- Para columnas enteras: se eliminan separadores de miles (p. ej. “.”) y se castea a DoubleType.
- Para columnas decimales: se castea directamente a DoubleType.

```
# LIMPIAR NÚMEROS
# =====
cols_int = [
    "issues_analysis_in_review_menor_7_dias", "issues_analysis_in_review",
    "historias_deployed_fix_version", "nro_historias_deployed",
    "repositorios_activos_nomenclatura_estandar", "total_repositorios_activos",
    "repositorios_activos_gobernados_chimera",
    "ejecuciones_exitosas_pipelines", "ejecuciones_totales_pipelines",
    "repositorios_activos_sonarqube_ok", "repositorios_conectados_sonarqube",
    "items_pruebas_desplegados_xray", "items_desplegados_totales"
]

for c in cols_int:
    if c in df.columns:
```

```

df = df.withColumn(c, F.regexp_replace(F.col(c), r"\.",
"").cast(DoubleType()))

cols_float = [
    "tiempo_medio_aprobacion_pr", "tamano_medio_pr",
    "tiempo_medio_integracion", "tiempo_medio_construcciones",
    "tiempo_medio_reparacion_builds"
]

for c in cols_float:
    if c in df.columns:
        df = df.withColumn(c, F.col(c).cast(DoubleType

```

Ordenamiento por mes, geografia, uol1_name, uol2_name, service1_id para consistencia.

```

# ORDENAR
# =====
df = df.orderBy("mes", "geografia", "uol1_name", "uol2_name", "service1_id")

```

Consolidación en un solo archivo: se escribe temporalmente con coalesce(1) y se mueve el part-*.csv al nombre final en la ruta Silver. Luego se elimina la carpeta temporal.

```

# GUARDAR EN TEMPORAL
# =====
df.coalesce(1).write.mode("overwrite").option("header", True).csv(temp_path)

# =====
# MOVER CSV FINAL
# =====
files = dbutils.fs.ls(temp_path)
csv_file = [f.path for f in files if f.path.endswith(".csv")][0]

dbutils.fs.mkdirs(final_dir)
dbutils.fs.rm(final_file, True)
dbutils.fs.mv(csv_file, final_file)

# =====
# LIMPIAR TEMP
# =====
dbutils.fs.rm(temp_path, True)

print("Archivo final generado correctamente en:")
print(final_file)

```

Resultado. Dataset CI limpio en Silver, listo para construir dimensiones y hechos en el Data Warehouse.

B) Notebook: Transformar Continuous Integration (Silver → PostgreSQL / Modelo estrella)

Su objetivo es tomar el CSV limpio desde Silver y poblar el Data Warehouse en PostgreSQL con un modelo estrella: dimensiones (dim_tiempo, dim_geografia, dim_servicio_n1) y tabla de hechos (fact_mediciones_ci).

a) Entrada (Silver)

abfss://silver@stbbvadalake.dfs.core.windows.net/data_limpiar/data_limpiar_continuous_integration/data_limpiar_continuous_integration.csv

b) Salida (PostgreSQL)

- dim_tiempo
- dim_geografia
- dim_servicio_n1
- fact_mediciones_ci

c) Transformaciones y lógica.

Lectura del CSV (inferSchema) y creación de fecha desde mes (MM/dd/yyyy).

```
# Leer CSV desde ADLS (Silver)
# =====
path_ci =
"abfss://silver@stbbvadalake.dfs.core.windows.net/data_limpiar/data_limpiar_c
ontinuous_integration/data_limpiar_continuous_integration.csv"

df = spark.read.csv(path_ci, header=True, inferSchema=True)
df = df.withColumn("fecha", F.to_date(F.col("mes"), "MM/dd/yyyy"))
```

Construcción de dimensiones:

- **dim_tiempo:** fecha, día, mes, año, ordenado por (año, mes) ascendente.
- **dim_geografia:** valores distintos de geografía como nombre_geografia.
- **dim_servicio_n1:** servicio_n1_id (int), nombre_servicio, nombre_uo11, nombre_uo12, deduplicado por ID.

```
# CREAR DIMENSIONES
```

```
# ----- DIM TIEMPO (ordenada por mes ascendente)
dim_tiempo = (
    df.select("fecha")
        .distinct()
        .withColumn("dia", F.dayofmonth("fecha"))
        .withColumn("mes", F.month("fecha"))
        .withColumn("año", F.year("fecha"))
        .orderBy("año", "mes")           # ← orden correcto
)
```

```
# ----- DIM GEOGRAFIA
dim_geografia = (
    df.select("geografia")
        .distinct()
        .withColumnRenamed("geografia", "nombre_geografia")
)
```

```
# ----- DIM SERVICIO N1
dim_servicio_n1 = (
    df.select(
        F.col("service1_id").cast("int").alias("servicio_n1_id"),
        F.col("service1_name").alias("nombre_servicio"),
        F.col("uo11_name").alias("nombre_uo11"),
        F.col("uo12_name").alias("nombre_uo12")
    )
)
```

```

    )
    .dropDuplicates(["servicio_n1_id"])
    .repartition(1)
)

```

Carga controlada (idempotencia):

- Se ejecuta TRUNCATE ...RESTARTIDENTITY CASCADE en fact_mediciones_ci, dim_tiempo, dim_geografia.
- Se limpia dim_servicio_n1 con DELETE (y se reinician secuencias donde aplica).

```

# TRUNCATE + RESET IDs
# =====
conn = psycopg2.connect(
    host="pg-bbva-dashboard.postgres.database.azure.com",
    database="data_oro_ci",
    user="adminuser",
    password="SecurePass123!",
    sslmode="require"
)
cur = conn.cursor()

cur.execute("TRUNCATE TABLE fact_mediciones_ci RESTART IDENTITY CASCADE;")
cur.execute("TRUNCATE TABLE dim_tiempo RESTART IDENTITY CASCADE;")
cur.execute("TRUNCATE TABLE dim_geografia RESTART IDENTITY CASCADE;")
cur.execute("DELETE FROM dim_servicio_n1;")

# Reset proper identity sequences
cur.execute("ALTER SEQUENCE dim_tiempo_tiempo_id_seq RESTART WITH 1;")
cur.execute("ALTER SEQUENCE dim_geografia_geografia_id_seq RESTART WITH 1;")

conn.commit()
cur.close()
conn.close()

```

Insertión de dimensiones por JDBC.

```

# INSERTAR DIMENSIONES
# =====
dim_tiempo.write.jdbc(url=jdbc_url, table="dim_tiempo", mode="append",
    properties=properties)
dim_geografia.write.jdbc(url=jdbc_url, table="dim_geografia", mode="append",
    properties=properties)
dim_servicio_n1.write.jdbc(url=jdbc_url, table="dim_servicio_n1",
    mode="append", properties=properties)

```

Lectura de dimensiones desde PostgreSQL para obtener IDs asignados.

```

# LEER DIMENSIONES INSERTADAS
# =====
dim_tiempo_psql = spark.read.jdbc(url=jdbc_url, table="dim_tiempo",
    properties=properties)
dim_geografia_psql = spark.read.jdbc(url=jdbc_url, table="dim_geografia",
    properties=properties)
dim_servicio_n1_psql = spark.read.jdbc(url=jdbc_url, table="dim_servicio_n1",
    properties=properties)

```

Construcción de tabla de hechos fact_mediciones_ci:

- Normalización de geografía (geografia_norm) para joins robustos.
- Join contra dim_tiempo por fecha, contra dim_geografia por geografia_norm, y contra dim_servicio_n1 por service1_id.
- Selección final: claves (tiempo_id, geografia_id, servicio_n1_id) + métricas CI (issues, PRs, pipelines, SonarQube, Xray, etc.), casteadas a tipos int/float según corresponda.

```
# BUILDEAR FACT CI
# =====
df_norm = df.withColumn("geografia_norm",
F.upper(F.trim(F.col("geografia"))))
dim_g_alias = dim_geografia_psql.withColumn("geografia_norm",
F.upper(F.trim(F.col("nombre_geografia"))))

fact = (
    df_norm
    .join(dim_tiempo_psql, df_norm["fecha"] == dim_tiempo_psql["fecha"],
"inner")
    .join(dim_g_alias, df_norm["geografia_norm"] ==
dim_g_alias["geografia_norm"], "inner")
    .join(dim_servicio_n1_psql, df_norm["service1_id"].cast("int") ==
dim_servicio_n1_psql["servicio_n1_id"], "inner")
    .select(
        dim_tiempo_psql["tiempo_id"],
        dim_g_alias["geografia_id"],
        dim_servicio_n1_psql["servicio_n1_id"],
        df_norm["issues_analysis_in_review_menor_7_dias"].cast("int"),
        df_norm["issues_analysis_in_review"].cast("int"),
        df_norm["historias_deployed_fix_version"].cast("int"),
        df_norm["nro_historias_deployed"].cast("int"),
        df_norm["repositorios_activos_nomenclatura_estandar"].cast("int"),
        df_norm["total_repositorios_activos"].cast("int"),
        df_norm["tiempo_medio_aprobacion_pr"].cast("float"),
        df_norm["tamano_medio_pr"].cast("float"),
        df_norm["repositorios_activos_gobernados_chimera"].cast("int"),
        df_norm["tiempo_medio_integracion"].cast("float"),
        df_norm["tiempo_medio_construcciones"].cast("float"),
        df_norm["ejecuciones_exitosas_pipelines"].cast("int"),
        df_norm["ejecuciones_totales_pipelines"].cast("int"),
        df_norm["tiempo_medio_reparacion_builds"].cast("float"),
        df_norm["repositorios_activos_sonarqube_ok"].cast("int"),
        df_norm["repositorios_conectados_sonarqube"].cast("int"),
        df_norm["items_pruebas_desplegados_xray"].cast("int"),
        df_norm["items_desplegados_totales"].cast("int")
    )
)
```

Insertión final en fact_mediciones_ci (modo append).

```
# INSERTAR FACT CI
# =====
fact.write.jdbc(
    url=jdbc_url,
    table="fact_mediciones_ci",
    mode="append",
```

```

        properties=properties
    )

```

Nota técnica (buena práctica). Para un entorno productivo, las credenciales (usuario/password) no deberían estar embebidas en el notebook: se recomienda gestionarlas con **Azure Key Vault** o **Databricks Secret Scopes**.

C) Notebook: Vista Continuous Integration (DW → KPIs / Vista de negocio)

Su objetivo es construir una vista analítica lista para consumo en dashboards: se leen hechos y dimensiones desde PostgreSQL, se calculan KPIs CI y se publica una vista vw_ci_kpis.

a) Entradas (PostgreSQL)

- fact_mediciones_ci
- dim_tiempo
- dim_geografia
- dim_servicio_n1

```

# Databricks notebook source
from pyspark.sql import functions as F
from pyspark.sql.window import Window
import psycopg2

# =====
# Conexión a PostgreSQL (AZURE)
# =====
pg_url = "jdbc:postgresql://pg-bbva-
dashboard.postgres.database.azure.com:5432/data_oro_ci?sslmode=require"

pg_properties = {
    "user": "adminuser",
    "password": "SecurePass123!",
    "driver": "org.postgresql.Driver"
}

# =====
# Leer tabla de hechos + dimensiones
# =====
fact_mediciones_ci = spark.read.jdbc(pg_url, "fact_mediciones_ci",
properties=pg_properties)
dim_tiempo = spark.read.jdbc(pg_url, "dim_tiempo", properties=pg_properties)
dim_geografia = spark.read.jdbc(pg_url, "dim_geografia",
properties=pg_properties)
dim_servicio_n1 = spark.read.jdbc(pg_url, "dim_servicio_n1",
properties=pg_properties)

```

b) Proceso

Star Join (hechos + dimensiones) para obtener dataset con fecha, mes, año, geografía, service1_name y métricas base.

```

# Star Join
# =====

```

```

df = (
    fact_mediciones_ci.alias("fm")
    .join(dim_tiempo.alias("dt"), F.col("fm.tiempo_id") ==
F.col("dt.tiempo_id"))
    .join(dim_geografia.alias("dg"), F.col("fm.geografia_id") ==
F.col("dg.geografia_id"))
    .join(dim_servicio_n1.alias("dsn1"), F.col("fm.servicio_n1_id") ==
F.col("dsn1.servicio_n1_id"))
    .select(
        F.col("dt.fecha"),
        F.col("dt.mes"),
        F.col("dt.anio"),
        F.col("dg.nombre_geografia").alias("geografia"),
        F.col("dsn1.nombre_servicio").alias("service1_name"),

        # Métricas originales (fact)
        "issues_analysis_in_review_menor_7_dias",
        "issues_analysis_in_review",
        "historias_deployed_fix_version",
        "nro_historias_deployed",
        "repositorios_activos_nomenclatura_estandar",
        "total_repositorios_activos",
        "tiempo_medio_aprobacion_pr",
        "tamano_medio_pr",
        "repositorios_activos_gobernados_chimera",
        "tiempo_medio_integracion",
        "tiempo_medio_construcciones",
        "ejecuciones_exitosas_pipelines",
        "ejecuciones_totales_pipelines",
        "tiempo_medio_reparacion_builds",
        "repositorios_activos_sonarqube_ok",
        "repositorios_conectados_sonarqube",
        "items_pruebas_desplegados_xray",
        "items_desplegados_totales"
    )
)

```

Cálculo de KPIs (12 indicadores)

- **analisis_review_7dias_pct:** % de issues en review menores a 7 días

$$= (\text{issues_analysis_in_review_menor_7_dias} / \text{issues_analysis_in_review}) \times 100$$
- **historias_fix_version_pct:** % de historias deploy con fix version

$$= (\text{historias_deployed_fix_version} / \text{nro_historias_deployed}) \times 100$$
- **repos_nomenclatura_pct:** % repos activos con nomenclatura estándar

$$= (\text{repositorios_activos_nomenclatura_estandar} / \text{total_repositorios_activos}) \times 100$$
- **aprobacion_pr_pct (score por umbral):**

$$<1 \rightarrow 90, <1.5 \rightarrow 70, <3 \rightarrow 30, \geq 3 \rightarrow 0$$
- **tamano_pr_pct (score por umbral):**

$$<300 \rightarrow 90, <400 \rightarrow 70, <600 \rightarrow 30, \geq 600 \rightarrow 0$$
- **repos_chimera_pct:** % repos activos gobernados por Chimera

$$= (\text{repositorios_activos_gobernados_chimera} / \text{total_repositorios_activos}) \times 100$$
- **tiempo_integracion_pct (score por umbral):**

$$<3 \rightarrow 90, <4 \rightarrow 70, <6 \rightarrow 30, \geq 6 \rightarrow 0$$
- **tiempo_construcciones_pct (score por umbral):**

$$<20 \rightarrow 90, <35 \rightarrow 70, <45 \rightarrow 30, \geq 45 \rightarrow 0$$

- **construcciones_correctas_pct:** % ejecuciones exitosas de pipelines

$$= (\text{ejecuciones_exitosas_pipelines} / \text{ejecuciones_totales_pipelines}) \times 100$$
- **tiempo_arreglar_construcciones_pct (score por umbral):**

$$<60 \rightarrow 90, <120 \rightarrow 70, <180 \rightarrow 30, \geq 180 \rightarrow 0$$
- **calidad_codigo_pct:** % repos OK en SonarQube

$$= (\text{repositorios_activos_sonarqube_ok} / \text{repositorios_conectados_sonarqube}) \times 100$$
- **items_pruebas_xray_pct:** % items de pruebas desplegados en Xray

$$= (\text{items_pruebas_desplegados_xray} / \text{items_desplegados_totales}) \times 100$$

Calcular los 13 KPI de CI

=====

```
df_calc = (
    df
    .withColumn("analisis_review_7dias_pct",
        F.when(F.col("issues_analysis_in_review") > 0,
            F.round(F.col("issues_analysis_in_review_menor_7_dias") /
                F.col("issues_analysis_in_review") * 100, 2))
    )
    .withColumn("historias_fix_version_pct",
        F.when(F.col("nro_historias_deployed") > 0,
            F.round(F.col("historias_deployed_fix_version") /
                F.col("nro_historias_deployed") * 100, 2))
    )
    .withColumn("repos_nomenclatura_pct",
        F.when(F.col("total_repositorios_activos") > 0,
            F.round(F.col("repositorios_activos_nomenclatura_estandar") /
                F.col("total_repositorios_activos") * 100, 2))
    )
    .withColumn("aprobacion_pr_pct",
        F.when(F.col("tiempo_medio_aprobacion_pr") < 1, F.lit(90))
        .when(F.col("tiempo_medio_aprobacion_pr") < 1.5, F.lit(70))
        .when(F.col("tiempo_medio_aprobacion_pr") < 3, F.lit(30))
        .otherwise(F.lit(0))
    )
    .withColumn("tamano_pr_pct",
        F.when(F.col("tamano_medio_pr") < 300, F.lit(90))
        .when(F.col("tamano_medio_pr") < 400, F.lit(70))
        .when(F.col("tamano_medio_pr") < 600, F.lit(30))
        .otherwise(F.lit(0))
    )
    .withColumn("repos_chimera_pct",
        F.when(F.col("total_repositorios_activos") > 0,
            F.round(F.col("repositorios_activos_gobernados_chimera") /
                F.col("total_repositorios_activos") * 100, 2))
    )
    .withColumn("tiempo_integracion_pct",
        F.when(F.col("tiempo_medio_integracion") < 3, F.lit(90))
        .when(F.col("tiempo_medio_integracion") < 4, F.lit(70))
        .when(F.col("tiempo_medio_integracion") < 6, F.lit(30))
        .otherwise(F.lit(0))
    )
    .withColumn("tiempo_construcciones_pct",
        F.when(F.col("tiempo_medio_construcciones") < 20, F.lit(90))
        .when(F.col("tiempo_medio_construcciones") < 35, F.lit(70))
        .when(F.col("tiempo_medio_construcciones") < 45, F.lit(30))
        .otherwise(F.lit(0))
    )
)
```

```

)
.withColumn("construcciones_correctas_pct",
  F.when(F.col("ejecuciones_totales_pipelines") > 0,
    F.round(F.col("ejecuciones_exitosas_pipelines") /
      F.col("ejecuciones_totales_pipelines") * 100, 2))
)
.withColumn("tiempo_arreglar_construcciones_pct",
  F.when(F.col("tiempo_medio_reparacion_builds") < 60, F.lit(90))
  .when(F.col("tiempo_medio_reparacion_builds") < 120, F.lit(70))
  .when(F.col("tiempo_medio_reparacion_builds") < 180, F.lit(30))
  .otherwise(F.lit(0))
)
.withColumn("calidad_codigo_pct",
  F.when(F.col("repositorios_conectados_sonarqube") > 0,
    F.round(F.col("repositorios_activos_sonarqube_ok") /
      F.col("repositorios_conectados_sonarqube") * 100, 2))
)
.withColumn("items_pruebas_xray_pct",
  F.when(F.col("items_desplegados_totales") > 0,
    F.round(F.col("items_pruebas_desplegados_xray") /
      F.col("items_desplegados_totales") * 100, 2))
)
)
)

```

Adopción total ponderada (adopcion_total_pct)

Se calcula como promedio ponderado considerando solo KPIs no nulos:

Pesos CI:

- analisis_review_7dias_pct: 5
- historias_fix_version_pct: 6
- repos_nomenclatura_pct: 6
- aprobacion_pr_pct: 10
- tamano_pr_pct: 5
- repos_chimera_pct: 7
- tiempo_integracion_pct: 16
- tiempo_construcciones_pct: 9
- construcciones_correctas_pct: 13
- tiempo_arreglar_construcciones_pct: 8
- calidad_codigo_pct: 10
- items_pruebas_xray_pct: 5

```

# Adopción total ponderada (igual que Practitioner)
# =====
pesos_ci = {
  "analisis_review_7dias_pct": 5,
  "historias_fix_version_pct": 6,
  "repos_nomenclatura_pct": 6,
  "aprobacion_pr_pct": 10,
  "tamano_pr_pct": 5,
  "repos_chimera_pct": 7,
  "tiempo_integracion_pct": 16,
  "tiempo_construcciones_pct": 9,
  "construcciones_correctas_pct": 13,
  "tiempo_arreglar_construcciones_pct": 8,
  "calidad_codigo_pct": 10,

```

```

        "items_pruebas_xray_pct": 5
    }

df_calc = df_calc.withColumn(
    "suma_pesada",
    sum(
        F.when(F.col(k).isNotNull(), F.col(k) * v)
        .otherwise(F.lit(0))
        for k, v in pesos_ci.items()
    )
).withColumn(
    "suma_pesos",
    sum(
        F.when(F.col(k).isNotNull(), F.lit(v))
        .otherwise(F.lit(0))
        for k, v in pesos_ci.items()
    )
).withColumn(
    "adopcion_total_pct",
    F.when(F.col("suma_pesos") > 0,
        F.round(F.col("suma_pesada") / F.col("suma_pesos"), 2))
    .otherwise(F.lit(None).cast("double"))
)

```

Salida final (dataset para vista)

Se seleccionan las columnas: fecha, geografía, service1_name, los 12 KPIs calculados y adopcion_total_pct.

```

# Selección final
# =====
fact_final_ci = df_calc.select(
    "fecha",
    "geografia",
    "service1_name",
    "analisis_review_7dias_pct",
    "historias_fix_version_pct",
    "repos_nomenclatura_pct",
    "aprobacion_pr_pct",
    "tamano_pr_pct",
    "repos_chimera_pct",
    "tiempo_integracion_pct",
    "tiempo_construcciones_pct",
    "construcciones_correctas_pct",
    "tiempo_arreglar_construcciones_pct",
    "calidad_codigo_pct",
    "items_pruebas_xray_pct",
    "adopcion_total_pct"
)

```

c) Publicación en PostgreSQL (mismo patrón que Practitioner)

- Se elimina la vista si existe: DROP VIEW IF EXISTS vw_ci_kpis;
- Se escribe una tabla física: table_ci_kpis (overwrite)
- Se crea la vista final:
CREATE VIEW vw_ci_kpis AS SELECT * FROM table_ci_kpis;

```

# Crear la VISTA normal en PostgreSQL (MISMO PATRÓN QUE PRACTITIONER)
# =====
def execute_postgres(query):
    conn = psycopg2.connect(
        host="pg-bbva-dashboard.postgres.database.azure.com",
        port=5432,
        database="data_oro_ci",
        user="adminuser",
        password="SecurePass123!",
        sslmode="require"
    )
    cur = conn.cursor()
    cur.execute(query)
    conn.commit()
    cur.close()
    conn.close()

# Borrar vista si existe
execute_postgres("DROP VIEW IF EXISTS vw_ci_kpis;")

# Crear tabla FINAL (NO temporal)
fact_final_ci.write.jdbc(
    url=pg_url,
    table="table_ci_kpis",    # <-- Igual que Practitioner
    (table_practitioner_kpis)
    mode="overwrite",
    properties=pg_properties
)

# Crear vista que apunta a la tabla
execute_postgres("""
    CREATE VIEW vw_ci_kpis AS
    SELECT * FROM table_ci_kpis;
""")
print("✓ Vista creada correctamente: vw_ci_kpis")

```

Resultado. Vista final disponible para consumo del dashboard: **vw_ci_kpis**.

3.2. Querys

En esta sección se documentan los scripts SQL utilizados para definir la estructura del Data Warehouse en PostgreSQL. Estos archivos contienen la creación de tablas del modelo dimensional en estrella, separando tablas de dimensiones (catálogos) y tablas de hechos (métricas), con sus respectivas claves primarias y foráneas.

Los scripts se organizan por dominio funcional:

- **Practitioner (query_practitioner.sql):** Define dimensiones comunes y la tabla de hechos asociada a las métricas de adopción/seguridad del dominio Practitioner.
- **Continuous Integration (query_ci.sql):** Define dimensiones comunes y la tabla de hechos asociada a métricas de CI (issues, PRs, pipelines, calidad de código, etc.).

El objetivo de estos scripts es inicializar el esquema sobre el cual luego los notebooks de transformación cargan datos mediante JDBC (inserción de dimensiones y hechos), garantizando consistencia referencial y una base sólida para la creación de vistas de KPIs.

3.2.1. Practitioner Querys

Su propósito es crear las tablas del modelo estrella para el dominio Practitioner, compuesto por:

a) Dimensión Tiempo (dim_tiempo)

Almacena el calendario base para análisis temporal.

- PK: tiempo_id (SERIAL)
- Campos: fecha, dia, mes, anio

b) Dimensión Geografía (dim_geografia)

Catálogo de geografías asociadas a los registros.

- PK: geografia_id (SERIAL)
- Campo: nombre_geografia

c) Dimensión Servicio N1 (dim_servicio_n1)

Catálogo de servicios N1 y sus atributos organizacionales.

- PK: servicio_n1_id (VARCHAR)
- Campos: nombre_servicio, nombre_uo1, nombre_uo2

d) Tabla de hechos (fact_mediciones_practitioner)

Contiene las métricas principales por combinación de tiempo, geografía y servicio.

- PK: fact_id (SERIAL)
- FKs:
 - tiempo_id → dim_tiempo(tiempo_id)
 - geografia_id → dim_geografia(geografia_id)
 - servicio_n1_id → dim_servicio_n1(servicio_n1_id)
- Métricas: fichas RFO, dependencias, features, puntajes de adopción, vulnerabilidades y líneas de código.

A. Creación de la Tabla Hechos y Dimensiones (Modelo Estrella)

El archivo query_practitioner.sql crea el modelo estrella para el dominio Practitioner, compuesto por:

```
-- =====
-- DIMENSION TIEMPO
-- =====
CREATE TABLE IF NOT EXISTS dim_tiempo (
    tiempo_id SERIAL PRIMARY KEY,
    fecha DATE NOT NULL,
    dia INT,
    mes INT,
    anio INT
```

```

);

-- =====
-- DIMENSION GEOGRAFIA
-- =====
CREATE TABLE IF NOT EXISTS dim_geografia (
    geografia_id SERIAL PRIMARY KEY,
    nombre_geografia VARCHAR(200) NOT NULL
);

-- =====
-- DIMENSION SERVICIO N1
-- =====
CREATE TABLE IF NOT EXISTS dim_servicio_n1 (
    servicio_n1_id VARCHAR(50) PRIMARY KEY,
    nombre_servicio VARCHAR(200),
    nombre_uol1 VARCHAR(200),
    nombre_uol2 VARCHAR(200)
);

-- =====
-- TABLA DE HECHOS PRACTITIONER
-- =====
CREATE TABLE IF NOT EXISTS fact_mediciones_practitioner (
    fact_id SERIAL PRIMARY KEY,

    tiempo_id INT NOT NULL,
    geografia_id INT NOT NULL,
    servicio_n1_id VARCHAR(50) NOT NULL,

    nro_fichas_rfo INT,
    nro_fichas_rfo_ok INT,
    nro_sn2_sn1_dependencias INT,
    nro_sn2_sn1 INT,
    nro_features_desplegadas_calidad INT,
    nro_features_desplegadas INT,
    puntaje_total_adopcion_sn2 FLOAT,
    sn2_sn1_medidos INT,
    total_vulnerabilidades_high INT,
    total_lineas_codigo INT,

    FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo(tiempo_id),
    FOREIGN KEY (geografia_id) REFERENCES dim_geografia(geografia_id),
    FOREIGN KEY (servicio_n1_id) REFERENCES dim_servicio_n1(servicio_n1_id)
);

```

B. Consultas de Validación y Control de Calidad de Datos

Además de crear tablas, se incluyeron consultas SQL para validar la integridad y calidad de los datos cargados en PostgreSQL. Estas consultas permiten comprobar que la transformación ejecutada en Databricks fue correcta, detectando valores nulos críticos, inconsistencias y registros duplicados en dimensiones o hechos. Estas validaciones se ejecutan después de la carga (post-carga) y son útiles tanto para depuración durante el desarrollo como para asegurar que el Data Warehouse quede en condiciones adecuadas antes de construir KPIs o dashboards.

a) Borrar nulos (validación y limpieza controlada)

Query 1: Identificar nulos críticos en la tabla de hechos (FKs o métricas esenciales)

```
SELECT *
FROM fact_mediciones_practitioner
WHERE tiempo_id IS NULL
    OR geografia_id IS NULL
    OR servicio_n1_id IS NULL;
```

Query 2: Eliminar registros inválidos en hechos (si llegaron con llaves nulas)

```
DELETE FROM fact_mediciones_practitioner
WHERE tiempo_id IS NULL
    OR geografia_id IS NULL
    OR servicio_n1_id IS NULL;
```

b) Ver duplicados (control de unicidad e integridad)

Query 1: Duplicados en dimensión servicio (mismo ID repetido)

```
SELECT servicio_n1_id, COUNT(*) AS total
FROM dim_servicio_n1
GROUP BY servicio_n1_id
HAVING COUNT(*) > 1;
SELECT *
FROM fact_mediciones_practitioner
WHERE tiempo_id IS NULL
    OR geografia_id IS NULL
    OR servicio_n1_id IS NULL;
```

Query 2: Duplicados en hechos (misma combinación de llaves dimensionales)

```
SELECT tiempo_id, geografia_id, servicio_n1_id, COUNT(*) AS total
FROM fact_mediciones_practitioner
GROUP BY tiempo_id, geografia_id, servicio_n1_id
HAVING COUNT(*) > 1;
```

3.2.2. Continuous Integration Querys

Su propósito es crear las tablas del modelo estrella para el dominio Continuous Integration, compuesto por:

a) Dimensión Tiempo (dim_tiempo).

Tabla de calendario para series mensuales.

- PK: tiempo_id (SERIAL)
- Campos: fecha, día, mes, año

b) Dimensión Geografía (dim_geografia).

Catálogo de geografías.

- PK: geografia_id (SERIAL)
- Campo: nombre_geografia

c) Dimensión Servicio N1 (dim_servicio_n1).

Catálogo de servicios N1 del dominio CI.

- PK: servicio_n1_id (INT)
- Campos: nombre_servicio, nombre_uol1, nombre_uol2

d) Tabla de hechos (fact_mediciones_ci).

Concentra métricas operativas de CI por tiempo, geografia y servicio.

- PK: fact_id (SERIAL)
- FKs:
 - tiempo_id → dim_tiempo(tiempo_id)
 - geografia_id → dim_geografia(geografia_id)
 - servicio_n1_id → dim_servicio_n1(servicio_n1_id)
- Métricas: issues en review, PRs, repositorios, tiempos promedio, ejecuciones de pipelines, calidad (SonarQube) y pruebas (Xray).

A. Creación de la Tabla Hechos y Dimensiones (Modelo Estrella)

El archivo query_ci.sql crea el modelo estrella para el dominio Continuous Integration, compuesto por:

```
-- =====
-- DIMENSION TIEMPO
-- =====
CREATE TABLE dim_tiempo (
    tiempo_id SERIAL PRIMARY KEY,
    fecha DATE NOT NULL,
    dia INT,
    mes INT,
    anio INT
);

-- =====
-- DIMENSION GEOGRAFIA
-- =====
CREATE TABLE dim_geografia (
    geografia_id SERIAL PRIMARY KEY,
    nombre_geografia VARCHAR(200) NOT NULL
);

-- =====
-- DIMENSION SERVICIO N1
-- =====
CREATE TABLE dim_servicio_n1 (
    servicio_n1_id INT PRIMARY KEY,
    nombre_servicio VARCHAR(200),
    nombre_uol1 VARCHAR(200),
    nombre_uol2 VARCHAR(200)
);
```

```

-- =====
-- FACT CI
-- =====
CREATE TABLE fact_mediciones_ci (
    fact_id SERIAL PRIMARY KEY,

    tiempo_id INT NOT NULL,
    geografia_id INT NOT NULL,
    servicio_n1_id INT NOT NULL,

    issues_analysis_in_review_menor_7_dias INT,
    issues_analysis_in_review INT,
    historias_deployed_fix_version INT,
    nro_historias_deployed INT,
    repositorios_activos_nomenclatura_estandar INT,
    total_repositorios_activos INT,
    tiempo_medio_aprobacion_pr FLOAT,
    tamano_medio_pr FLOAT,
    repositorios_activos_gobernados_chimera INT,
    tiempo_medio_integracion FLOAT,
    tiempo_medio_construcciones FLOAT,
    ejecuciones_exitosas_pipeline INT,
    ejecuciones_totales_pipeline INT,
    tiempo_medio_reparacion_builds FLOAT,
    repositorios_activos_sonarqube_ok INT,
    repositorios_conectados_sonarqube INT,
    items_pruebas_desplegados_xray INT,
    items_desplegados_totales INT,

    FOREIGN KEY (tiempo_id) REFERENCES dim_tiempo(tiempo_id),
    FOREIGN KEY (geografia_id) REFERENCES dim_geografia(geografia_id),
    FOREIGN KEY (servicio_n1_id) REFERENCES dim_servicio_n1(servicio_n1_id)
);

```

B. Consultas de Validación y Control de Calidad de Datos

Para asegurar que la carga realizada por los notebooks de Databricks sea consistente, se incluyen consultas orientadas a validación de integridad (nulos no permitidos, llaves faltantes) y control de duplicidad (unicidad en dimensiones y granularidad correcta en hechos). Esto permite detectar errores típicos en ETL como duplicaciones por re-proceso, joins incorrectos o normalización insuficiente de claves.

a. Borrar nulos (validación y limpieza controlada)

Query 1: Identificar registros inválidos en hechos (FKs nulas)

```

SELECT *
FROM fact_mediciones_ci
WHERE tiempo_id IS NULL
    OR geografia_id IS NULL
    OR servicio_n1_id IS NULL;

```

Query 2: Eliminar hechos inválidos (si llegaron con FKs nulas)

```

DELETE FROM fact_mediciones_ci
WHERE tiempo_id IS NULL

```

```
OR geografia_id IS NULL
OR servicio_n1_id IS NULL;
```

b. Ver duplicados (control de unicidad e integridad)

Query 1: Duplicados en dimensión servicio (mismo ID repetido)

```
SELECT servicio_n1_id, COUNT(*) AS total
FROM dim_servicio_n1
GROUP BY servicio_n1_id
HAVING COUNT(*) > 1;
```

Query 2: Duplicados en hechos (misma combinación tiempo + geografía + servicio)

```
SELECT tiempo_id, geografia_id, servicio_n1_id, COUNT(*) AS total
FROM fact_mediciones_ci
GROUP BY tiempo_id, geografia_id, servicio_n1_id
HAVING COUNT(*) > 1;
```

3.3. Data Cruda

En esta sección se describe la data cruda (raw) que ingresa a la solución antes de aplicar cualquier proceso de limpieza o transformación. Estos archivos representan la fuente inicial del pipeline y se almacenan en la capa Bronze del Data Lake, manteniendo la estructura y nombres de columnas originales tal como fueron exportados. La data cruda es utilizada como input por los notebooks de Databricks (etapa Limpiar), donde se aplican reglas de estandarización (renombrar de campos, normalización de fechas, tipificación numérica, etc.) para generar datasets confiables en Silver y posteriormente cargar el Data Warehouse (PostgreSQL).

3.3.1. Data Cruda Practitioner

La data cruda del dominio Practitioner contiene métricas vinculadas a adopción, cumplimiento y seguridad por mes, geografía y servicio. Su granularidad está asociada a combinaciones de: `period_month` (periodo), `ug_name` (geografía), `uol1_name`, `uol2_name` y `servicell_id/servicell_name`.

Contenido:

- **Identificadores y contexto:** `period_month`, `ug_name`, `uol1_name`, `uol2_name`, `servicell_id`, `servicell_name`.
- **Métricas Practitioner:** por ejemplo: número de fichas RFO, SN2 con ficha RFO OK, SN2 con dependencias, features desplegadas (totales y válidas), puntaje/cumplimiento del modelo objetivo, SN2 medidos, vulnerabilidades high y líneas de código.

En esta capa es común encontrar: formatos de fecha heterogéneos (mes en distintos formatos), números como texto con separadores de miles, y columnas con nombres extensos/no estandarizados. Estas condiciones se corrigen en el notebook Limpiar Practitioner antes de generar el dataset en Silver.

Figura 50. Data Cruda Practitioner

1	uol1_name	uol2_name	servicel1_id	servicel1_name	Nº fichas RFC	Nº SN2 con ficha RFC con status OK	Nº SN2 con dependencias Consumos	Nº SN2 del tipo
2	SYSTEMS ENGINEERING	CORPORATE FUNCTIONS SYSTEMS	4	RIESGOS DE CREDITO	8	8	8	6
3	SYSTEMS ENGINEERING	CORPORATE FUNCTIONS SYSTEMS	4	RIESGOS DE CREDITO	8	8	8	6
4	SYSTEMS ENGINEERING	CORPORATE FUNCTIONS SYSTEMS	4	RIESGOS DE CREDITO	8	8	8	6
5	SYSTEMS ENGINEERING	CORPORATE FUNCTIONS SYSTEMS	4	RIESGOS DE CREDITO	8	8	8	6
6	SYSTEMS ENGINEERING	CORPORATE FUNCTIONS SYSTEMS	4	RIESGOS DE CREDITO	8	8	8	6
7	SYSTEMS ENGINEERING	CORPORATE FUNCTIONS SYSTEMS	4	RIESGOS DE CREDITO	8	8	8	6
8	SYSTEMS ENGINEERING	CORPORATE FUNCTIONS SYSTEMS	4	RIESGOS DE CREDITO	8	8	8	6
9	RISK		6	RETAIL CREDIT RISK	0	0	0	2
10	RISK		6	RETAIL CREDIT RISK	0	0	0	3
11	RISK		6	RETAIL CREDIT RISK	0	0	0	3
12	RISK		6	RETAIL CREDIT RISK	0	0	0	3
13	RISK		6	RETAIL CREDIT RISK	0	0	0	3
14	RISK		6	RETAIL CREDIT RISK	0	0	0	3
15	RISK		6	RETAIL CREDIT RISK	0	0	0	3
16	RISK		7	PROVISIONES	3	3	3	2

Fuente: Excel

3.3.2. Data Cruda Continuous Integration

La data cruda del dominio Continuous Integration (CI) contiene métricas relacionadas a prácticas de integración continua: revisión de issues, desempeño de PRs, repositorios, pipelines, calidad de código y pruebas. Al igual que Practitioner, se organiza por periodo, geografía y servicio.

Contenido:

- **Identificadores y contexto:** period_month, ug_name, uol1_name, uol2_name, servicel1_id, servicel1_name.
- **Métricas CI (raw):** por ejemplo: issues en review < 7 días, total issues en revisión, historias con fix version desplegadas, cantidad de historias deploy, repos activos con nomenclatura estándar, total repos activos, tiempos promedio (aprobación PR, integración, builds, reparación), ejecuciones exitosas/totales de pipelines, repos conectados a SonarQube y repos OK, items de pruebas desplegados en Xray, etc.

En CI también se presentan nombres originales no uniformes (que luego se renombran a un estándar), campos numéricos como texto y variaciones en el formato del periodo. Estas inconsistencias se resuelven en el notebook Limpiar Continuous Integration, previo a la carga a Silver y al DW.

Figura 51. Data Cruda Continuous Integration

1	A	B	C	D	E	F	G	H
2	period_month	ug_name	uol1_name	uol2_name	servicel1_id	servicel1_name	issues_review_menor_7_dias	issues_en_revision_tot
3	1/06/2025	VENEZUELA	SYSTEMS	PAYMENT	685	ISSUING(EMISOR/AUTORIZADOR)	11	12
4	1/06/2025	PERU	SYSTEMS	RETAIL	734	MOBILE CHANNELS	10	10
5	1/06/2025	CIB	SW DELIVERY	FIXED INCOME	1183	FI BOOKINGS & PORTFOLIO MANAGEMENT	10	11
6	1/06/2025	COLOMBIA	SYSTEMS ENGINEERING	CPS-FINANZAS-TAC-R&C	569	FINANZAS	11	12
7	1/07/2025	ESPAÑA	SYSTEMS ENGINEERING	CORPORATE FUNCTIONS SYSTEMS	291	IMPAGO Y COBERTURAS	11	12
8	1/06/2025	MEXICO	SYSTEMS ENGINEERING	RETAIL	436	MOBILE RETAIL	7	7
9	1/06/2025	COMMERCIAL CLIENT SOLUTIONS	CHANNELS & PROCESSES	SERVICING & CLIENT EXPERIENCE TRANSFORMATION	1073	PIVOT IMPLEMENTATION	12	13
10	1/06/2025	PERU	SYSTEMS	PAYMENT METHODS	752	ISSUING	12	13
11	1/06/2025	CIB	DATA ENGINEERING	ANALYTICS	1144	SUSTAINABILITY SOLUTIONS	9	9
12	1/07/2025	RETAIL CLIENT SOLUTIONS	RETAIL	RETAIL - PAYMENTS	886	PLATAFORMA ADQUISICION	12	13
13	1/06/2025	ESPAÑA	DATA ENGINEERING	DATA HUB	847	CROSS SYSTEM	11	12
14	1/06/2025	COLOMBIA	DATA TRANSFORMATION & ENGINEERING	SALESFORCE	873	SALESFORCE - DATA	10	11
15	1/06/2025	COLOMBIA	SYSTEMS ENGINEERING	CORE	564	LOANS	3	3
16	1/07/2025	ESPAÑA	SYSTEMS ENGINEERING	INSURANCE AND ASSET MANAGEMENT SYSTEMS	292	INSURANCE - AMORBO Y PREVISION	11	12
17	1/06/2025	COMMERCIAL CLIENT SOLUTIONS	VALUE PROPOSITION	HIGH VALUE SOLUTIONS	1055	FINANCIACION ESPECIALIZADA	11	11
18	1/06/2025	MEXICO	SYSTEMS ENGINEERING	RETAIL	438	OPEN MARKET	12	12
19	1/06/2025	COMMERCIAL CLIENT SOLUTIONS	CHANNELS & PROCESSES	COREX & BANKING PLATFORM EVOLUTION	1064	CREDIT EXPERIENCE	2	2

Fuente: Excel.

3.4. Stack Tecnológico

En esta sección se describe el stack tecnológico utilizado para la capa de aplicación de la solución, diferenciando claramente los componentes de Backend y Frontend. Esta separación permite mantener una arquitectura modular: el backend se encarga de la lógica de negocio, conexión a servicios (Databricks, almacenamiento, base de datos) y

exposición de endpoints, mientras que el frontend se encarga de la interfaz de usuario y consumo de datos para su visualización.

A continuación, se presenta la organización de carpetas/archivos que conforman cada componente, junto con su propósito dentro de la solución.

3.4.1. Backend

El backend corresponde a la capa encargada de orquestar el acceso a los datos y servicios en la nube. Su función principal es actuar como intermediario entre el Data Warehouse / servicios de Azure y el cliente web, centralizando la lógica de consulta, configuración y conexión. Adicionalmente, se incluye la configuración necesaria para despliegue y ejecución en contenedores (Docker), así como scripts de inicialización.

Componentes principales identificados en el backend:

- **app.py:** Punto de entrada de la aplicación (levantamiento del servidor y registro del módulo principal).
- **routes.py:** Definición de rutas/endpoints que expone el backend para ser consumidos por el frontend.
- **database.py:** Capa de conexión y operaciones con la base de datos (PostgreSQL del Data Warehouse).
- **databricks_client.py:** Cliente de integración para consumir recursos de Databricks (consultas, jobs o acceso a resultados).
- **azure_storage.py:** Utilidades para interacción con almacenamiento en Azure (por ejemplo, lectura/escritura o soporte de archivos).
- **config.py:** Configuración centralizada (parámetros, URLs, variables de entorno).
- **requirements.txt:** Dependencias Python necesarias para ejecutar la aplicación.
- **Dockerfile:** Definición de construcción de la imagen del backend para despliegue.
- **init-db.sh:** Script de inicialización (por ejemplo, preparación de BD o ejecución de scripts al inicio).
- **wait-for-it.sh:** Script de sincronización para esperar disponibilidad de servicios (útil en despliegues con múltiples contenedores).
- **__pycache__/:** Artefactos generados por Python (no forman parte de la lógica).

Figura 52. Fragmento de estructura del Backend

Name	Date modified	Type	Size
__pycache__	12/15/2025 2:23 AM	File folder	
app.py	12/15/2025 2:23 AM	Python Source File	1 KB
azure_storage.py	12/15/2025 2:23 AM	Python Source File	3 KB
config.py	12/15/2025 2:23 AM	Python Source File	2 KB
database.py	12/15/2025 2:23 AM	Python Source File	17 KB
databricks_client.py	12/15/2025 2:23 AM	Python Source File	12 KB
Dockerfile	12/15/2025 2:23 AM	File	1 KB
init-db.sh	12/15/2025 2:23 AM	SH Source File	1 KB
requirements.txt	12/15/2025 2:23 AM	Documento de tex...	1 KB
routes.py	12/15/2025 2:23 AM	Python Source File	10 KB
wait-for-it.sh	12/15/2025 2:23 AM	SH Source File	1 KB

Fuente: Carpeta de PC.

3.4.2. Frontend

El frontend corresponde a la capa de presentación, responsable de mostrar la información al usuario final mediante una interfaz web. Este componente consume los datos servidos por el backend (endpoints/API) y los presenta en vistas y componentes del dashboard. La estructura observada indica un proyecto moderno basado en tooling de construcción y empaquetado para aplicaciones web.

Componentes principales identificados en el frontend (según estructura del repositorio):

- **src/**: Código fuente principal de la aplicación (componentes, vistas, lógica de UI).
- **public/**: Recursos públicos estáticos accesibles directamente.
- **dist/**: salida de compilación/build lista para despliegue (generada por el proceso de build).
- **index.html**: Plantilla base/entry point del frontend.
- **package.json** y **package-lock.json**: Definición de dependencias y control de versiones (ecosistema Node).
- **vite.config.js**: Configuración del bundler/herramienta de desarrollo Vite (dev server, build, rutas, etc.).
- **eslint.config.js**: Configuración de linting/estándares de código.
- **README.md**: Documentación del proyecto (uso, instalación, comandos).

Figura 53. Fragmento de estructura del Frontend

Name	Date modified	Type	Size
dist	12/15/2025 2:23 AM	File folder	
public	12/15/2025 2:23 AM	File folder	
src	12/15/2025 2:23 AM	File folder	
eslint.config.js	12/15/2025 2:23 AM	JavaScript Source ...	1 KB
index.html	12/15/2025 2:23 AM	Opera Web Docu...	3 KB
package.json	12/15/2025 2:23 AM	JSON Source File	1 KB
package-lock.json	12/15/2025 2:23 AM	JSON Source File	96 KB
README.md	12/15/2025 2:23 AM	Markdown Source...	2 KB
vite.config.js	12/15/2025 2:23 AM	JavaScript Source ...	1 KB

Fuente: Carpeta de PC.

3.5. Kqls

En esta sección se documentan las consultas KQL (Kusto Query Language) utilizadas para monitoreo y observabilidad de la solución desplegada en Azure. Estas consultas se ejecutan en Azure Monitor / Log Analytics y permiten revisar actividad reciente, detectar errores operativos y validar el funcionamiento de los componentes principales: Data Lake (Bronze/Silver), PostgreSQL y Container Apps (backend).

El objetivo de incluir KQLs en el informe es evidenciar que la solución cuenta con mecanismos de trazabilidad, diagnóstico y verificación post-despliegue, facilitando la identificación rápida de fallas y el seguimiento de eventos relevantes.

3.5.1. KQLs para Data Lake (Bronze y Silver)

Estas consultas están orientadas a revisar logs de operaciones sobre blobs en el Data Lake (por ejemplo, lecturas/escrituras, movimientos y estados). Se utilizan principalmente para:

- verificar que se estén generando archivos en las capas correctas (Bronze/Silver),
- auditar operaciones recientes,
- identificar rápidamente errores HTTP (códigos ≥ 400).

a) Últimos 20 eventos en Bronze

```
StorageBlobLogs
| where ObjectKey contains "/bronze/"
| project TimeGenerated, OperationName, StatusCode, ObjectKey
| order by TimeGenerated desc
| take 20
```

b) Errores en Bronze (StatusCode ≥ 400)

```
StorageBlobLogs
| where ObjectKey contains "/bronze/"
| extend StatusCodeInt = toint(StatusCode)
| where StatusCodeInt  $\geq$  400
| project TimeGenerated, OperationName, StatusCodeInt, ObjectKey
| take 20
```

c) Últimos 20 eventos en Silver

```
StorageBlobLogs
| where ObjectKey contains "/silver/"
| project TimeGenerated, OperationName, StatusCode, ObjectKey
| order by TimeGenerated desc
| take 20
```

d) Errores en Silver (StatusCode ≥ 400)

```
StorageBlobLogs
| where ObjectKey contains "/silver/"
| extend StatusCodeInt = toint(StatusCode)
| where StatusCodeInt  $\geq$  400
| project TimeGenerated, OperationName, StatusCodeInt, ObjectKey
| take 20
```

3.5.2. KQLs para PostgreSQL

Estas consultas se enfocan en eventos registrados por Azure para el servicio de PostgreSQL. Resultan útiles para:

- revisar actividad reciente del motor (eventos/diagnósticos),
- validar que el servidor esté recibiendo conexiones,
- detectar patrones de autenticación y desconexión.

a) Últimos 20 logs del servicio PostgreSQL

```
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.DBFORPOSTGRESQL"
| project TimeGenerated, OperationName, errorLevel_s, sqlerrcode_s, Message
| order by TimeGenerated desc
| take 20
```

b) Conexiones recibidas

```
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.DBFORPOSTGRESQL"
| where Message contains "connection received"
| project TimeGenerated, Message
| take 20
```

c) Autenticaciones exitosas

```
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.DBFORPOSTGRESQL"
| where Message contains "authenticated"
| project TimeGenerated, Message
| take 20
```

d) Desconexiones

```
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.DBFORPOSTGRESQL"
| where Message contains "disconnection"
| project TimeGenerated, Message
| take 20
```

3.5.3. KQLs para Azure Container Apps (Backend)

Estas consultas se utilizan para observar el comportamiento del backend desplegado en Azure Container Apps, permitiendo:

- inspeccionar los últimos logs disponibles,
- filtrar rápidamente excepciones/errores,
- obtener un resumen de eventos por tipo (Reason),
- revisar eventos de escalado, revisiones y estado de salud.

a) Últimos logs del backend

```
ContainerAppSystemLogs_CL
| project TimeGenerated, Reason_s, ContainerAppName_s, ReplicaName_s, Log_s
| order by TimeGenerated desc
| take 20
```

b) Logs de errores del backend

```
ContainerAppSystemLogs_CL
| where Log_s contains "ERROR" or Log_s contains "Exception"
| project TimeGenerated, ContainerAppName_s, ReplicaName_s, Log_s
| order by TimeGenerated desc
```

```
| take 20
```

c) Resumen de eventos por tipo

```
ContainerAppSystemLogs_CL
| summarize Count = count() by Reason_s
| order by Count desc
```

d) Últimos logs de escalado / revisiones / health

```
ContainerAppSystemLogs_CL
| where Reason_s contains "Scale"
    or Reason_s contains "Revision"
    or Reason_s contains "Health"
| project TimeGenerated, Reason_s, ContainerAppName_s, Log_s
| order by TimeGenerated desc
| take 20
```

4. VISUALIZACIÓN DEL DASHBOARD

El Dashboard Niveles de Madurez es una aplicación web integral de Business Intelligence diseñada para monitorear, visualizar y analizar los Indicadores Clave de Rendimiento (KPIs) asociados a los niveles de madurez tecnológica de los equipos y servicios del BBVA. Su propósito es brindar una vista consolidada y dinámica del avance de los servicios en los niveles Practitioner y Continuous Integration, apoyando la toma de decisiones estratégicas mediante reportes interactivos.

En la parte superior, tienes tres botones que te permiten moverte por el dashboard:

- **SUBIR DATOS (Botón Azul):** Este es el botón activo en esta pantalla. Al hacer clic aquí, llegas a esta misma vista, donde puedes cargar nuevos archivos. Es el punto de partida.
- **VER DASHBOARD GLOBAL:** Este botón está desactivado (apagado) cuando no hay datos cargados. Una vez que subas un archivo y el sistema lo procese, podrás hacer clic aquí para ver una vista general de todos los servicios, con gráficos y tablas resumen.
- **VER DASHBOARD POR SERVICIO:** Este botón también está desactivado inicialmente. Cuando tengas datos cargados, te llevará a una vista más profunda, donde podrás analizar el rendimiento de un servicio específico en detalle.

Figura 54. Botones de Navegación Superior

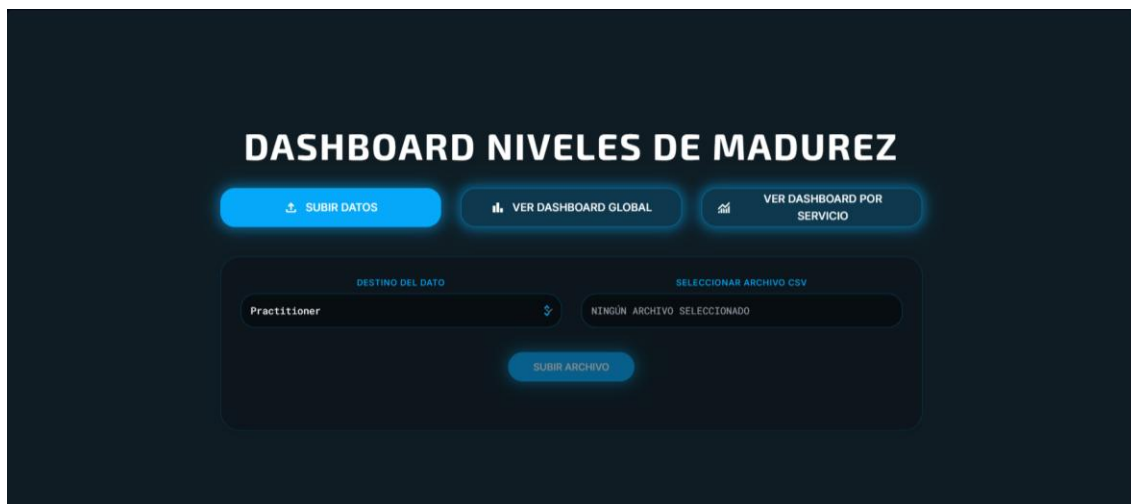


Fuente: *Dashboard BI BBVA.*

4.1. Interfaz Subir Datos

Esta interfaz constituye la puerta de entrada al pipeline de datos. Permite al usuario cargar archivos CSV que contienen la información descargada del Marco Playbook, validando su estructura antes de iniciar el procesamiento automático (Los Jobs definidos en el Databricks). A través de un diseño limpio e intuitivo, la aplicación guía al usuario en la selección del tipo de dataset y en la ejecución del flujo de transformación y carga.

Figura 55. Interfaz de carga inicial del Dashboard Niveles de Madurez



Fuente: *Dashboard BI BBVA.*

Este es el corazón de la pantalla. Aquí es donde tú, como usuario, interactúas con el sistema para subir la información.

A) Campo: DESTINO DEL DATO

- Qué es: Un menú desplegable (una cajita con una flechita abajo).
- Qué hace: Te permite decirle al sistema a qué nivel de madurez pertenecen los datos que vas a subir. Solo tienes dos opciones:
 - Practitioner: Si tu archivo CSV contiene métricas sobre este nivel.
 - Continuous Integration: Si tu archivo CSV contiene métricas sobre este otro nivel.
- Por qué es importante: El sistema necesita saber esto porque los archivos de estos dos niveles tienen estructuras diferentes. Al seleccionar uno, el sistema prepara todo su interior para recibir y procesar ese tipo de datos específicamente. Es como elegir el idioma en una máquina antes de usarla.

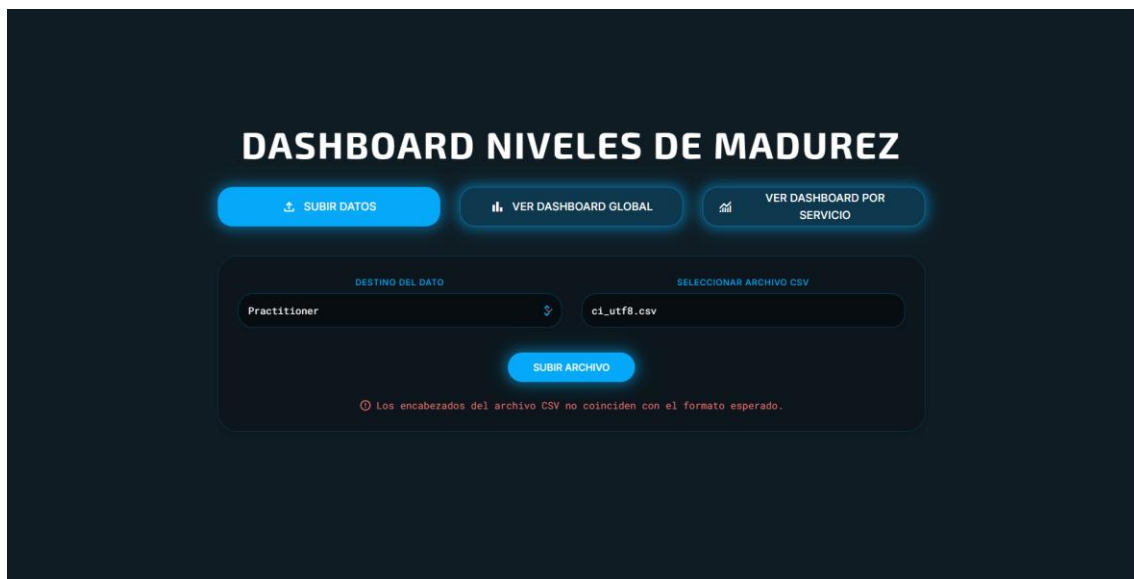
B) Campo: SELECCIONAR ARCHIVO CSV

- Qué es: Un botón que dice NINGÚN ARCHIVO SELECCIONADO.
- Qué hace: Cuando haces clic en él, se abre una ventana de tu computadora (como el Explorador de Archivos de Windows o Finder de Mac) para que busques y selecciones el archivo .csv que descargaste desde el Marco Playbook.
- Qué pasa después: Una vez que eliges un archivo, el texto en el botón cambiará para mostrar el nombre del archivo que seleccionaste (por ejemplo, datos_practitioner_marzo.csv). Si cambias de opinión, puedes volver a hacer clic para seleccionar otro.

C) Botón: SUBIR ARCHIVO

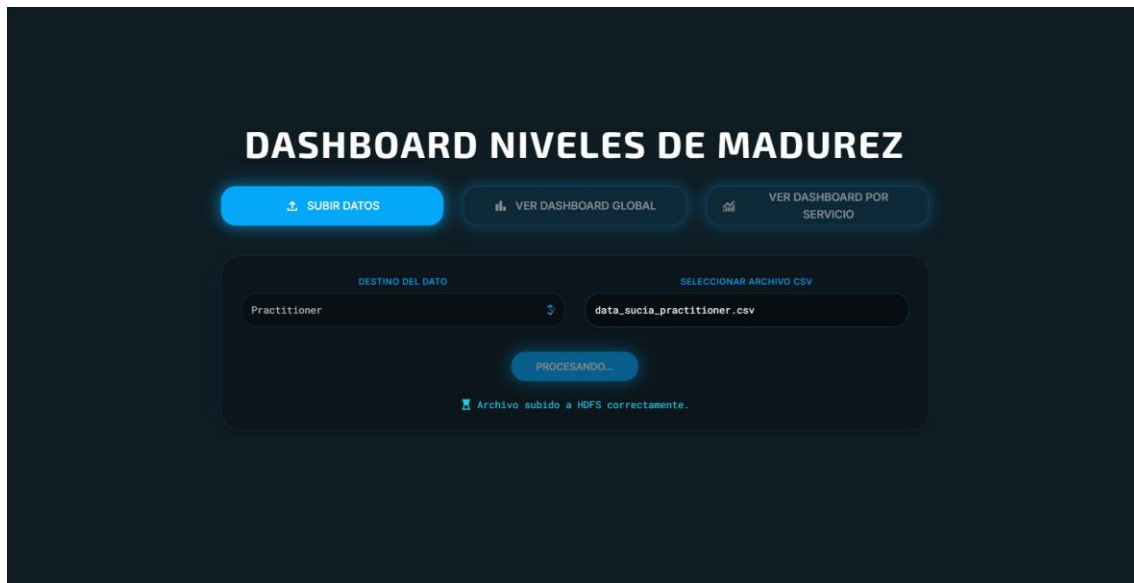
- Qué es: El botón azul grande y brillante en el centro.
- Qué hace: Cuando haces clic en él, inicias el proceso de carga.
- Qué pasa cuando lo presionas:
 - Validación: El sistema primero revisa que el archivo que seleccionaste tenga el formato correcto (.csv) y que las columnas dentro del archivo coincidan con lo que espera para el nivel que elegiste (Practitioner o Continuous Integration). Si algo no coincide, te mostrará un mensaje de error explicando qué salió mal.
 - Carga y Procesamiento: Si todo está bien, el sistema toma tu archivo y lo envía a su "laboratorio interno" (el backend). Allí, el archivo será limpiado, organizado y transformado para que pueda ser usado en los gráficos.
 - Feedback al Usuario: Mientras el sistema trabaja (lo cual puede tomar unos segundos), la pantalla se actualizará para mostrarte el progreso. Verás mensajes como "Limpiando...", "Subiendo...", "Procesando..." hasta que finalmente aparezca "¡Carga completada!".
 - Resultado Final: Una vez terminado, los otros dos botones de la barra superior (VER DASHBOARD GLOBAL y VER DASHBOARD POR SERVICIO) se activarán, y podrás navegar a las vistas de análisis.

Figura 56. Validación de estructura de archivo CSV con formato incorrecto



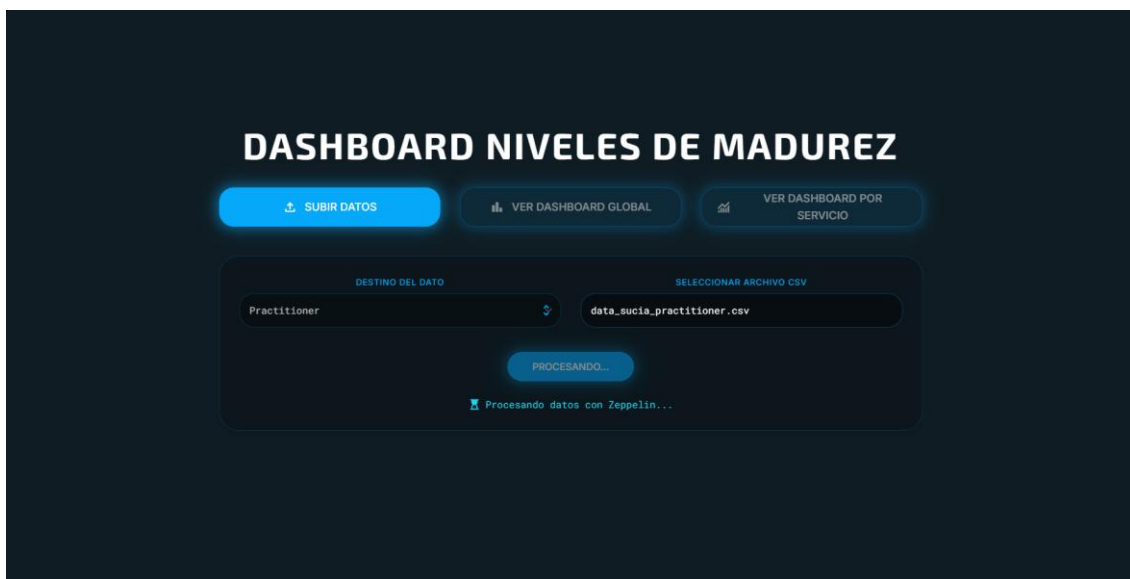
Fuente: *Dashboard BI BBVA.*

Figura 57. Confirmación de carga exitosa del archivo CSV al Datalake



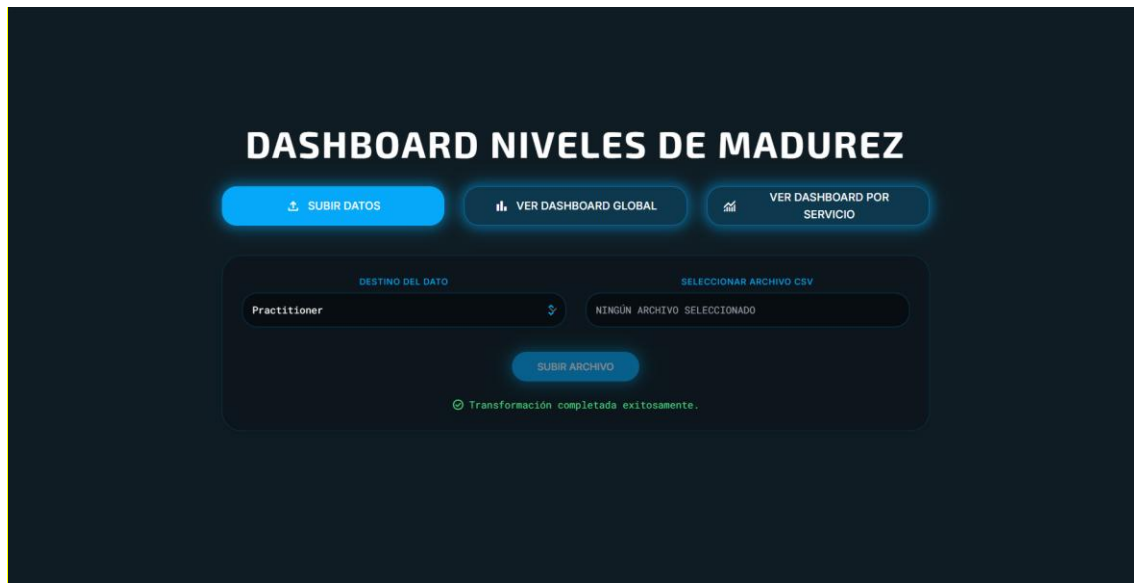
Fuente: *Dashboard BI BBVA.*

Figura 58. Ejecución del proceso ETL en Databricks desde el Dashboard



Fuente: *Dashboard BI BBVA.*

Figura 59. Mensaje de finalización del proceso de transformación exitoso

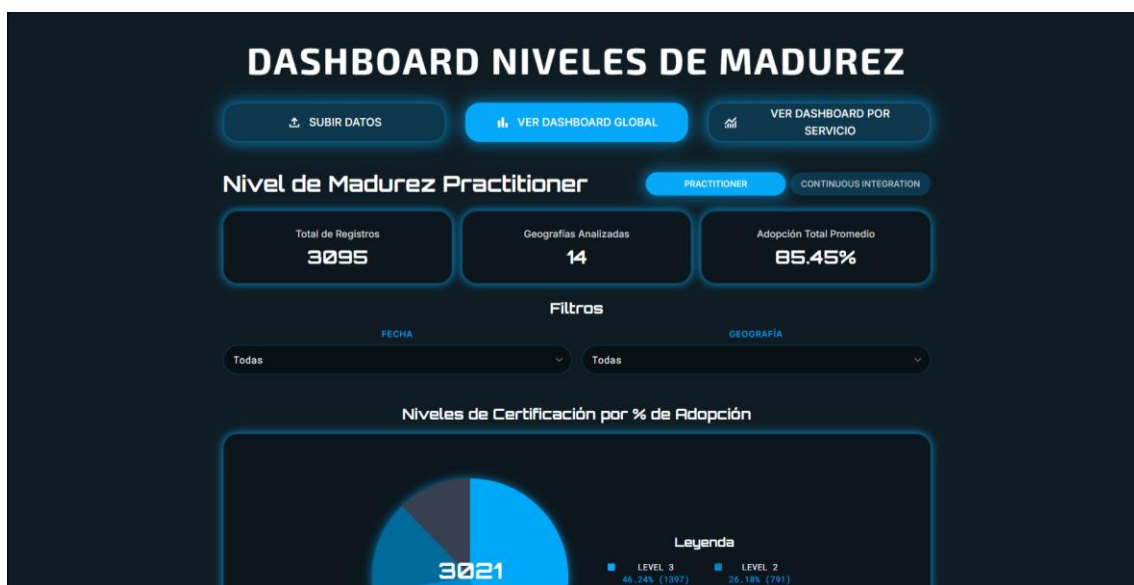


Fuente: *Dashboard BI BBVA.*

4.2. Interfaz Ver Dashboard Global

Esta vista ofrece una perspectiva macro de los niveles de madurez, consolidando los datos de todos los servicios. Permite a los usuarios analizar tendencias generales, comparar el rendimiento entre geografías y filtrar por fechas específicas.

Figura 60. Vista general del Dashboard Global

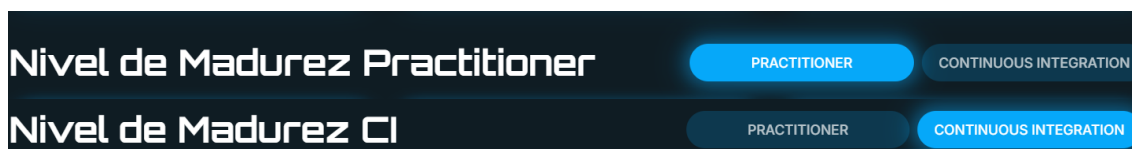


Fuente: *Dashboard BI BBVA.*

A) Selección del Nivel de Madurez

Justo debajo del título de la pantalla, encontrarás dos botones que te permiten cambiar entre los dos niveles de certificación.

Figura 61. Controles de selección entre niveles de madurez: Practitioner y Continuous Integration



Fuente: *Dashboard BI BBVA.*

- **PRACTITIONER:** Al hacer clic en este botón, la pantalla se actualiza para mostrar todos los datos, gráficos y tablas relacionados con el nivel Practitioner. El botón se pondrá en azul brillante para indicar que está seleccionado.
- **CONTINUOUS INTEGRATION:** Al hacer clic en este botón, la pantalla cambia para mostrar los datos del nivel Continuous Integration. El botón se resaltará y el contenido de la pantalla se actualizará completamente.

B) Tarjetas Resumen (Summary Cards)

Muestra tres tarjetas en la parte superior con métricas clave agregadas: total de registros, geografías analizadas y adopción total promedio. Estas métricas se calculan mediante funciones de agregación SQL sobre las vistas materializadas y se actualizan según el nivel de madurez seleccionado.

Figura 62. Tarjetas resumen con datos clave



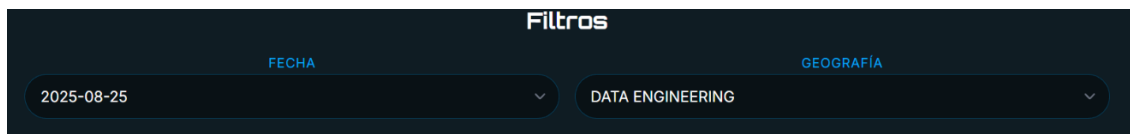
Fuente: *Dashboard BI BBVA.*

- **Total de Registros:** Muestra el número total de registros (filas) que se han cargado para el nivel de madurez seleccionado. Por ejemplo, para Practitioner, muestra 3095.
- **Geografías Analizadas:** Indica cuántas regiones o países diferentes están siendo evaluadas. En el ejemplo, son 14.
- **Adopción Total Promedio:** Este es el KPI más importante. Muestra el promedio ponderado de adopción de todas las métricas para el nivel seleccionado. Para Practitioner, es 81.92%, y para CI, es 79.53%.

C) Filtros de Análisis

Debajo de las tarjetas, encontrarás dos filtros que te permiten ajustar la vista de los datos.

Figura 63. Filtros por Fecha y Geografía



Fuente: *Dashboard BI BBVA.*

- Filtro FECHA: Un menú desplegable que te permite seleccionar un mes específico o elegir Todas para ver todos los datos disponibles. Esto es útil si quieres analizar el rendimiento de un periodo concreto.
- Filtro GEOGRAFÍA: Otro menú desplegable que te permite seleccionar una región específica (por ejemplo, ARGENTINA, PERU, MEXICO) o elegir Todas para ver la vista global.

D) Niveles de Certificación por % de Adopción

Visualiza mediante un gráfico de tarta la distribución de servicios clasificados en cuatro niveles de certificación (Level 3, Level 2, Level 1 y No certificado) según su porcentaje de adopción. El backend clasifica los servicios usando umbrales definidos ($\geq 90\%$, $\geq 80\%$, $\geq 70\%$, $< 70\%$) y cuenta cuántos caen en cada categoría. Además, este gráfico, se ve afectado por los filtros fecha y geografía.

Figura 64. Distribución de niveles de certificación por porcentaje de adopción



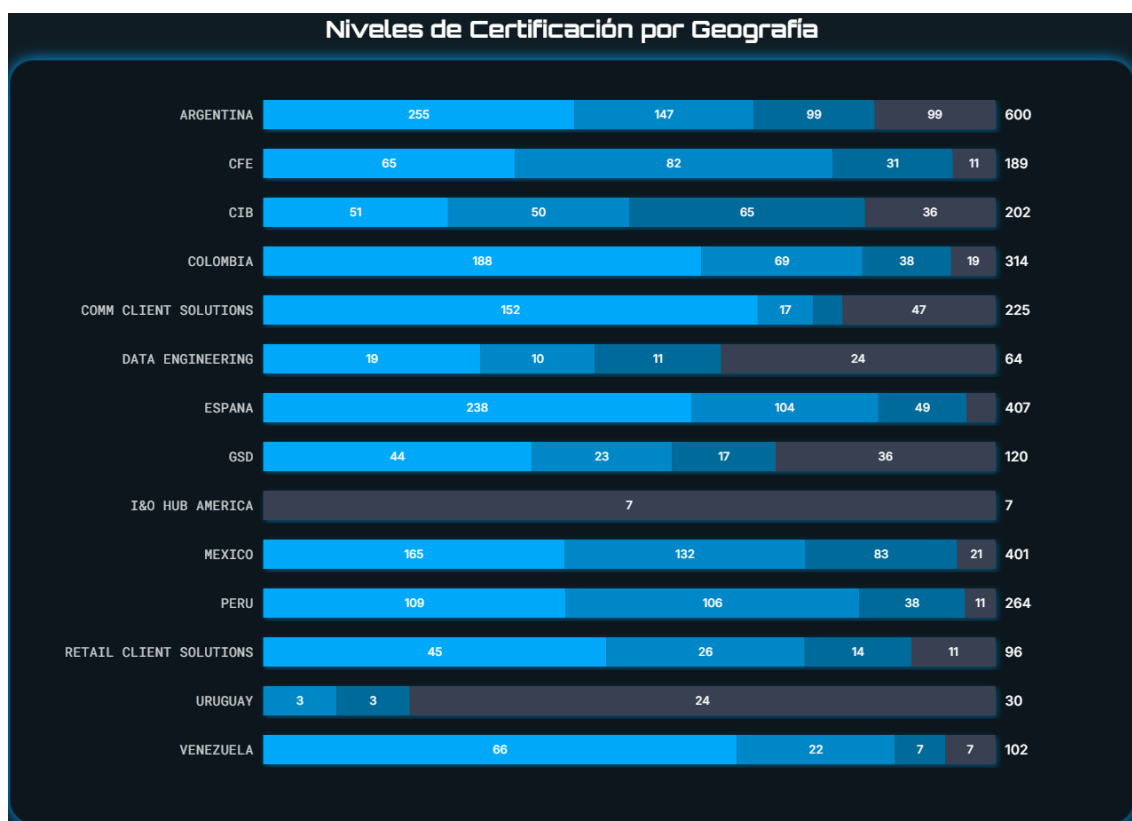
Fuente: *Dashboard BI BBVA.*

- Qué representa: El gráfico divide el círculo en cuatro sectores, cada uno representando un nivel de certificación:
 - LEVEL 3: Servicios con 90% o más de adopción. (Color Azul Claro)
 - LEVEL 2: Servicios con entre 80% y 89% de adopción. (Color Azul Medio)
 - LEVEL 1: Servicios con entre 70% y 79% de adopción. (Color Azul Oscuro)
 - No Certificado: Servicios con menos del 70% de adopción. (Color Gris)
- Qué ves en el centro: El número total de servicios que cumplen con los criterios de filtro (por ejemplo, 3021).
- Leyenda: A la derecha, una leyenda explica qué color corresponde a cada nivel y muestra el porcentaje y el número de servicios en cada categoría.

E) Niveles de Certificación por Geografía

Presenta un gráfico de barras apiladas que desglosa los niveles de certificación por cada geografía. Permite comparar visualmente cómo se distribuyen los servicios certificados entre diferentes regiones, usando la misma clasificación de niveles pero agrupada por dimensión geográfica. Además, este gráfico, se ve afectado por los filtros fecha y geografía.

Figura 65. Niveles de certificación distribuidos por geografía



Fuente: Dashboard BI BBVA.

- Qué representa: Cada barra horizontal corresponde a una geografía (por ejemplo, ARGENTINA, PERU, ESPAÑA). Dentro de cada barra, los colores de los segmentos muestran la cantidad de servicios en cada nivel de certificación (LEVEL 3, LEVEL 2, LEVEL 1, No Certificado).

- Qué ves al final de cada barra: El número total de servicios analizados en esa geografía.
- Cómo leerlo: Puedes ver fácilmente qué regiones tienen una mayor proporción de servicios en LEVEL 3 (azul claro) y cuáles tienen muchos servicios sin certificar (gris).

F) Cálculo de los KPI's

Muestra una tabla detallada con todos los KPIs de cada servicio, con soporte para paginación y filtrado dinámico. El backend ejecuta dos consultas: una para contar el total de registros y otra para obtener solo los datos de la página actual, aplicando los filtros seleccionados por el usuario mediante cláusulas WHERE construidas dinámicamente.

Figura 66. Tabla detallada de cálculo de los KPI's por servicio y geografía

FECHA	GEOGRAFIA	SERVICE1 NAME	RFO OK PCT	DEP PCT	ADOPCION SN2 PCT	CALIDAD FEATURES PCT	SEGURIDAD PCT	ADOPCION TOTAL PCT
2025-09-01	ARGENTINA	ALPHA	100.00%	100.00%	100.00%	95.52%	100.00%	99.10%
2025-09-01	ARGENTINA	AM03 CONSUMER FINANCE	100.00%	100.00%	50.00%	100.00%	100.00%	86.50%
2025-09-01	ARGENTINA	AM04 CONSUMER FINANCE	100.00%	83.33%	0.00%	-	100.00%	63.54%
2025-09-01	ARGENTINA	AM05 CONSUMER FINANCE	100.00%	100.00%	77.78%	100.00%	100.00%	94.00%
2025-09-01	ARGENTINA	AR01 AUTOSERVICIOS SUCURSALES	100.00%	75.00%	100.00%	-	100.00%	95.94%
2025-09-01	ARGENTINA	AR02 ENTERPRISE WEB CORE	100.00%	100.00%	-	-	-	100.00%
2025-09-01	ARGENTINA	AR03 PRESTAMOS HIPOTECARIOS CANALES INTERNOS	100.00%	100.00%	0.00%	100.00%	100.00%	73.00%
2025-09-01	ARGENTINA	AR04 MEDIOS ELECTRONICOS DE PAGOS - MOBILE	100.00%	0.00%	0.00%	-	100.00%	50.00%
2025-09-01	ARGENTINA	AR05 LOYALTY	100.00%	100.00%	0.00%	-	100.00%	66.25%
2025-09-01	ARGENTINA	AR06 COBRANZAS EMISION TC	100.00%	100.00%	100.00%	100.00%	-	100.00%

Mostrando 1 a 10 de 3095 resultados

Anterior Siguiente

Fuente: Dashboard BI BBVA.

- Qué contiene: La tabla tiene columnas para:
 - FECHA: El mes al que corresponden los datos.
 - GEOGRAFÍA: La región del servicio.
 - SERVICE1 NAME: El nombre del servicio.
 - RFO OK PCT, DEP PCT, ADOPCION SN2 PCT, CALIDAD FEATURES PCT, SEGURIDAD PCT: Los porcentajes individuales de cada KPI.
 - ADOPCION TOTAL PCT: El porcentaje de adopción total ponderado, que es el resultado final.

- Paginación: Debajo de la tabla, hay botones Anterior y Siguiente para navegar por las páginas. También muestra cuántos registros se están mostrando (por ejemplo, Mostrando 1 a 10 de 3095).

4.3. Interfaz Ver Dashboard por Servicio

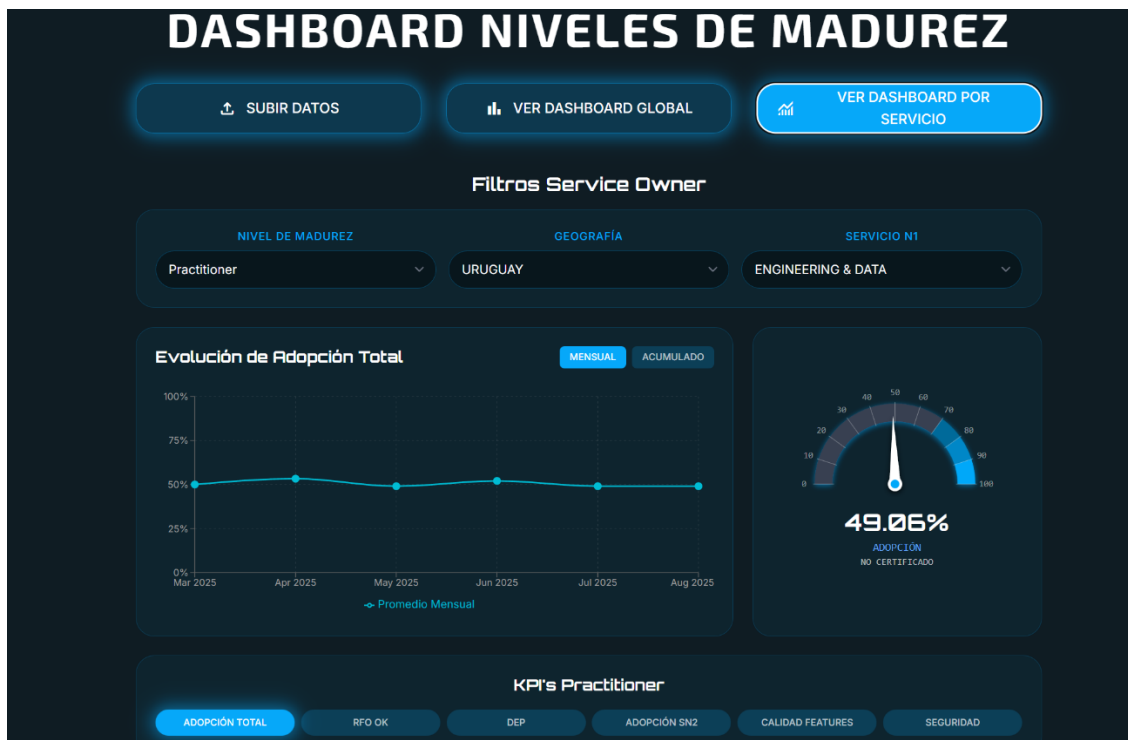
Esta es una vista de "drill-down" que permite a los Service Owners o gerentes analizar en profundidad el rendimiento de un servicio específico a lo largo del tiempo. Ofrece gráficos evolutivos, un medidor de adopción actual y un análisis comparativo de KPIs mediante un gráfico de radar.

Figura 67. Vista inicial del Dashboard por Servicio



Fuente: Dashboard BI BBVA.

Figura 68. Ejemplo del Dashboard por Servicio

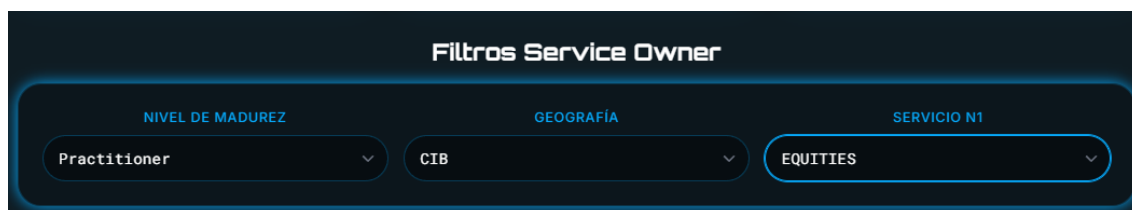


Fuente: Dashboard BI BBVA.

A) Filtros Service Owner

En esta pantalla, los filtros son el corazón del sistema. Son los controles que te permiten elegir exactamente qué servicio quieres analizar. El diseño es inteligente porque los filtros se actualizan en cascada, lo que significa que la selección de uno afecta las opciones disponibles en el siguiente.

Figura 69. Filtros en cascada para Nivel de Madurez, Geografía y Servicio N1



Fuente: *Dashboard BI BBVA.*

a) NIVEL DE MADUREZ

- Qué es: Un menú desplegable que te permite seleccionar el nivel de certificación que deseas analizar: Practitioner o Continuous Integration.
- Qué hace: Al hacer clic en este filtro, el sistema consulta la base de datos y te muestra solo las geografías que tienen datos para ese nivel de madurez. Esto evita que veas opciones inválidas.
- Por qué es importante: Es el primer paso para enfocar tu análisis. Si quieres ver cómo va un servicio en Practitioner, debes seleccionar este nivel primero.

b) GEOGRAFÍA

- Qué es: Otro menú desplegable que te permite seleccionar una región específica (por ejemplo, ARGENTINA, PERU, URUGUAY).
- Qué hace: Una vez que has seleccionado un nivel de madurez, este filtro se actualiza automáticamente para mostrarte solo las geografías que tienen datos para ese nivel. Por ejemplo, si seleccionas Practitioner, no podrás elegir una geografía que solo tenga datos de CI.
- Por qué es importante: Te permite acotar tu análisis a una región específica. Si eres el Service Owner de Uruguay, puedes filtrar solo por esa geografía.

c) SERVICIO N1

- Qué es: Un tercer menú desplegable que te permite seleccionar un servicio específico dentro de la geografía que elegiste.
- Qué hace: Al seleccionar una geografía, este filtro se actualiza para mostrarte solo los servicios que existen en esa región para el nivel de madurez seleccionado. Por ejemplo, si eliges URUGUAY y Practitioner, te mostrará servicios como ENGINEERING & DATA o RETAIL CLIENT SOLUTIONS.
- Por qué es importante: Este es el filtro final que te lleva al análisis de un servicio concreto.

B) Grafico Evolutivo

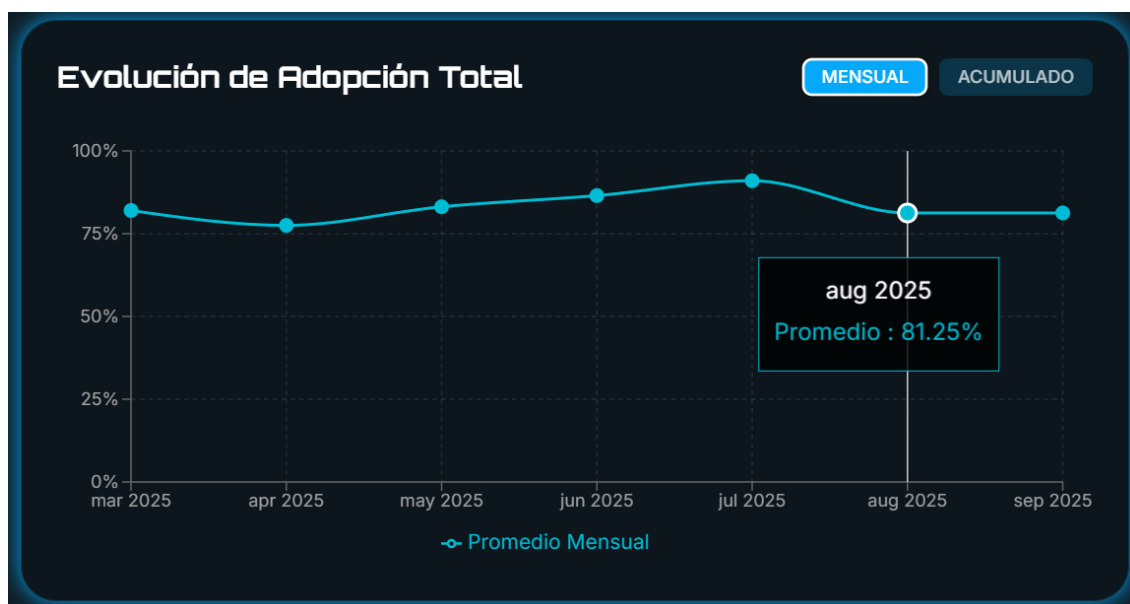
Muestra la evolución mensual de un KPI seleccionado mediante un gráfico de líneas con dos modos de visualización: mensual (datos puntuales) y acumulado (promedio progresivo calculado en el frontend). El backend agrupa los datos por mes usando `TO_CHAR(fecha, 'YYYY-MM')` y calcula promedios `AVG` para los últimos 12 meses del servicio seleccionado. Esta gráfica cambia según el KPI seleccionado en en Filtrar Gráfico por KPI'S.

a) Gráfico de Evolución de Adopción Total

Este es un gráfico de líneas que te muestra cómo ha cambiado el porcentaje de adopción total del servicio a lo largo del tiempo. Cada punto en la línea corresponde al porcentaje de adopción total del servicio para un mes específico. Por ejemplo, puedes ver cómo fue en marzo, abril, mayo, etc.

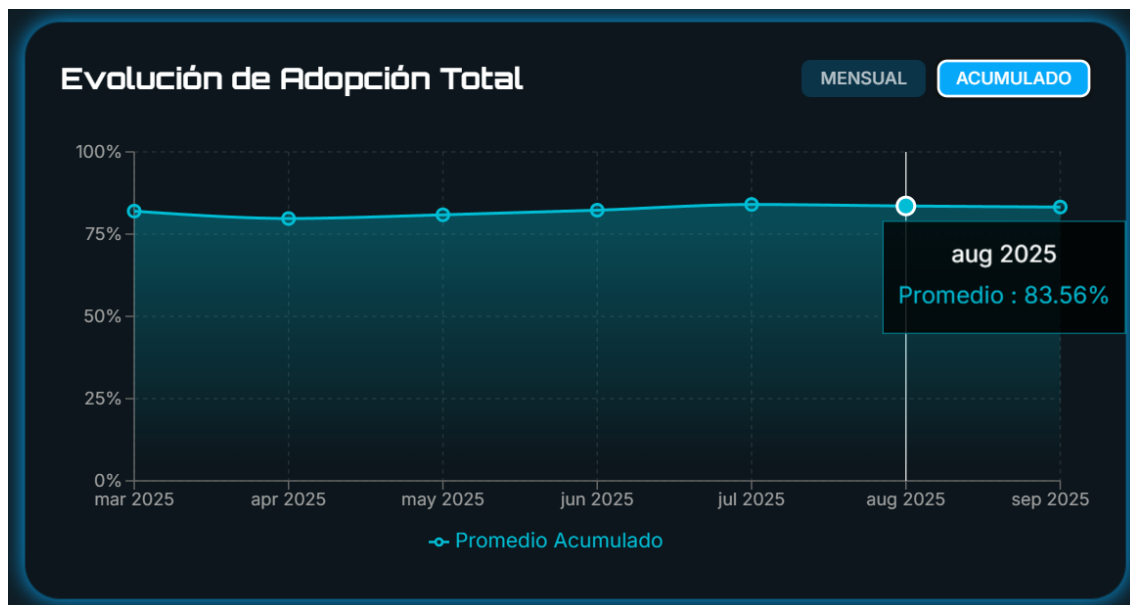
- Modos de Visualización:
 - MENSUAL: Muestra los datos puntuales de cada mes. Es útil para ver los picos y valles.
 - ACUMULADO: Muestra un promedio progresivo. Es útil para ver la tendencia general a lo largo del tiempo.
- Cómo funciona: El sistema toma los datos de los últimos 12 meses del servicio seleccionado y los grafica. Puedes ver claramente si el servicio está mejorando, empeorando o manteniéndose estable.

Figura 70. Evolución mensual de la adopción total (modo mensual)



Fuente: Dashboard BI BBVA.

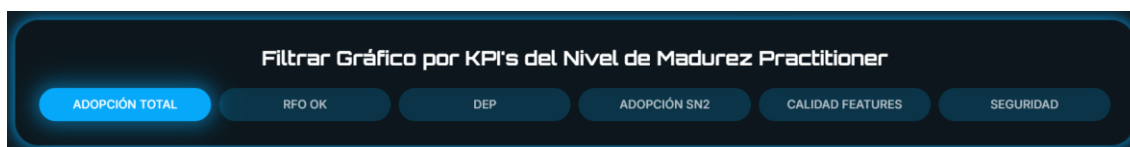
Figura 71. Evolución acumulada de la adopción total (modo acumulado)



Fuente: Dashboard BI BBVA.

b) Selector de KPI's: Este es un conjunto de botones que te permiten cambiar el gráfico de evolución para ver otros KPIs individuales del servicio.

Figura 72. Selector de KPI's para actualizar el gráfico evolutivo



Fuente: Elaboración propia.

- **Qué hace:** Al hacer clic en un botón (por ejemplo, RFO OK, DEP, CALIDAD FEATURES), el gráfico de evolución cambia para mostrar la tendencia de ese KPI específico en lugar de la adopción total.
- **Por qué es importante:** Te permite profundizar en un aspecto concreto del servicio. Si ves que la adopción total está baja, puedes usar este selector para ver si el problema está en la calidad de las features, en las dependencias o en otro KPI.

C) Gauge Chart (Semáforo)

Muestra el valor más reciente de la métrica seleccionada. Si es "Adopción Total", se visualiza como un GaugeChart tipo velocímetro con zonas de color; para otras métricas se muestra el valor numérico. El dato se extrae del último elemento del array de datos ya cargado para el gráfico evolutivo, reutilizando la información sin hacer peticiones adicionales. Este apartado cambia según el KPI seleccionado en Filtrar Gráfico por KPI'S.

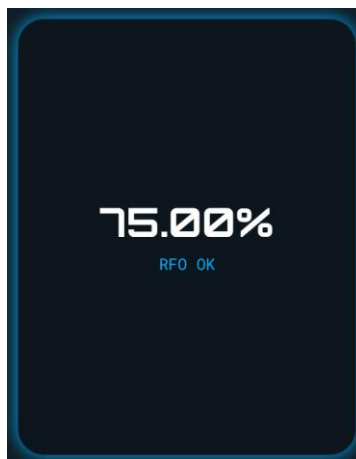
- Colores: El velocímetro tiene zonas de color:
 - Verde: Indica un buen estado ($\geq 90\%$).
 - Amarillo: Indica un estado medio (70-89%).
 - Rojo: Indica un estado crítico ($< 70\%$).
- Texto: Debajo del número, te dice en qué categoría de certificación se encuentra el servicio (por ejemplo, NO CERTIFICADO).

Figura 73. Indicador de Adopción Total mediante Gauge Chart (velocímetro)



Fuente: *Dashboard BI BBVA.*

Figura 74. Indicador numérico del KPI RFO OK del servicio seleccionado



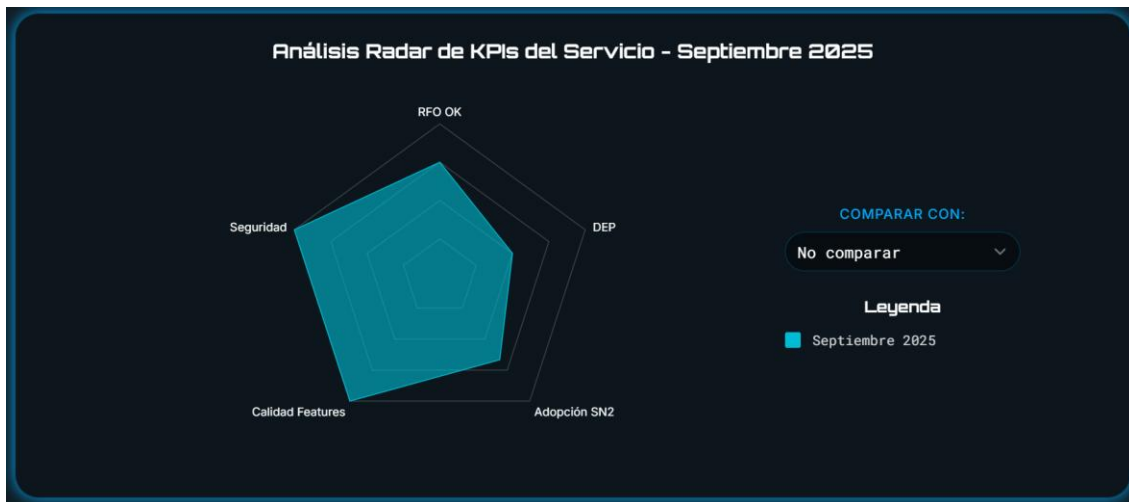
Fuente: *Dashboard BI BBVA.*

D) Análisis Radar de KPIs del Servicio

Visualiza todos los KPIs de un servicio en un gráfico tipo radar para proporcionar una vista holística de su rendimiento. Muestra por defecto los datos del mes más reciente, pero ofrece un selector para comparar con meses anteriores, superponiendo ambos períodos en el mismo gráfico. El backend maneja dos modos: sin parámetro de fecha devuelve el mes más reciente, con parámetro devuelve el mes específico solicitado. Por lo que esta gráfica cambia si seleccionamos al un mes diferente al actual en la lista desplegable.

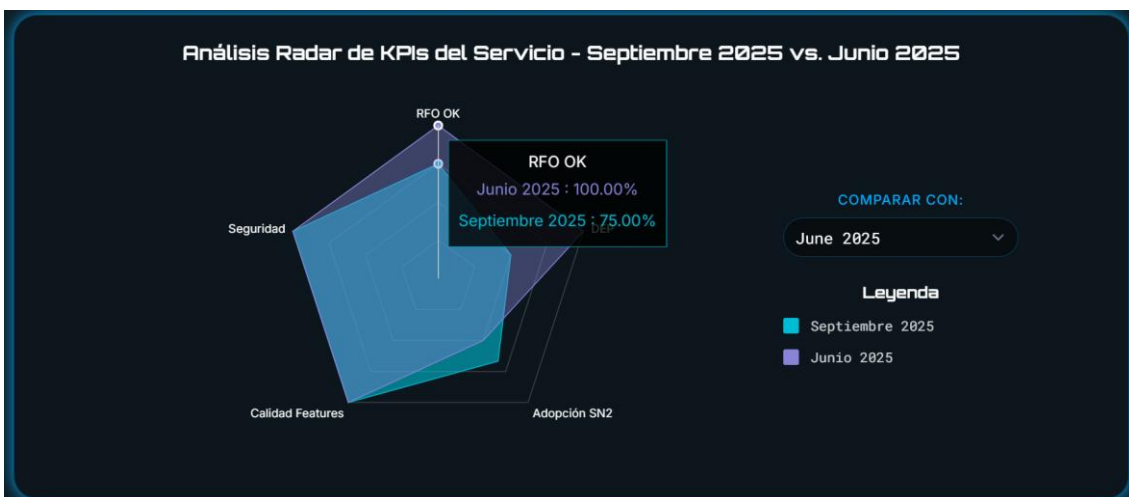
- Qué representa: Cada eje del radar corresponde a un KPI diferente (por ejemplo, RFO OK, DEP, ADOPCIÓN SN2, CALIDAD FEATURES, SEGURIDAD). La distancia desde el centro indica el valor del KPI.
- Comparación: Puedes seleccionar un mes pasado (por ejemplo, Julio 2025) para compararlo con el mes actual (Agosto 2025). El gráfico superpondrá ambos períodos, permitiéndote ver fácilmente si el servicio ha mejorado o empeorado en cada KPI.
- Leyenda: A la derecha, una leyenda explica qué color corresponde a cada período.

Figura 75. Gráfico radar de KPIs del servicio



Fuente: Dashboard BI BBVA.

Figura 76. Comparativo radar de KPIs

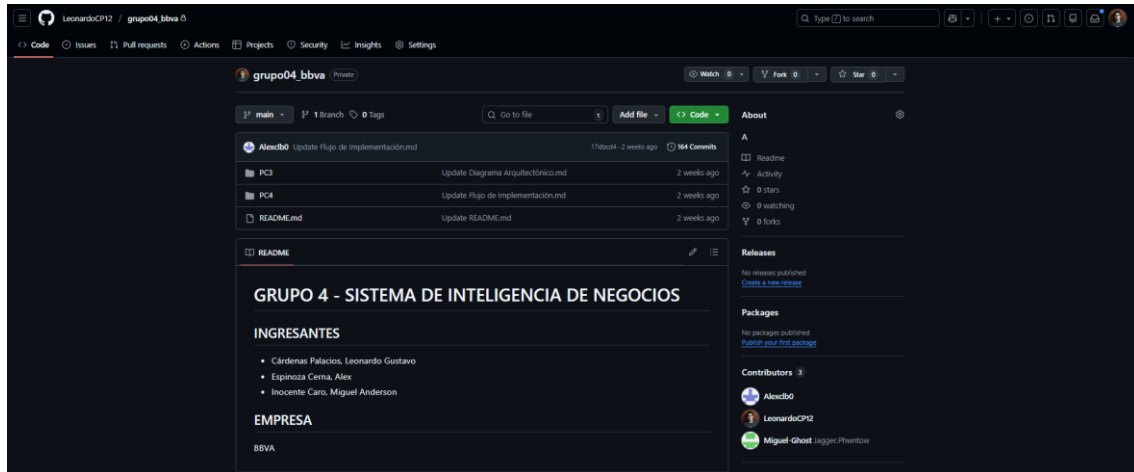


Fuente: Dashboard BI BBVA.

5. DISTRIBUCIÓN DE LOS ROLES GITHUB

Para facilitar el manejo del trabajo, se utilizó un repositorio privado para luego subir los archivos a la rama correspondiente en el github público: feature/grupo04-leonardo-pc4 y luego a feature/grupo04-alex-pc4

Figura 77. Github Privado del Grupo 4

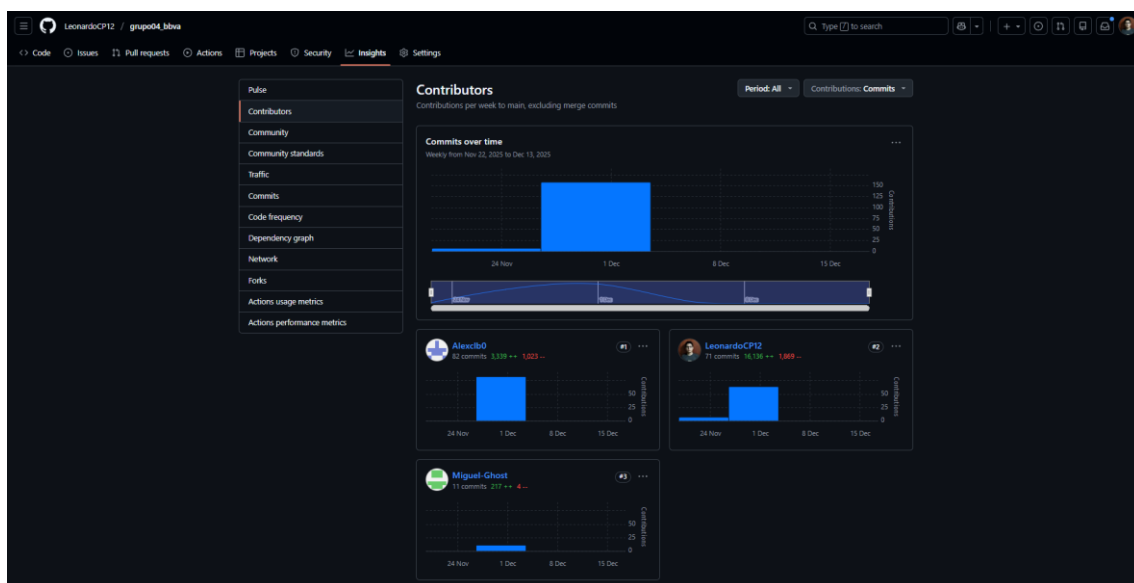


Fuente: *GitHub.*

A los miembros se les asignó un apartado para poder distribuir de mejor manera el trabajo.

- LeonardoCP: Líder del Proyecto y Administrador del Repositorio.
- Alexclb0: Desarrollador Backend y Responsable de Integración.
- Miguel-Ghost: Desarrollador Frontend y Documentación.

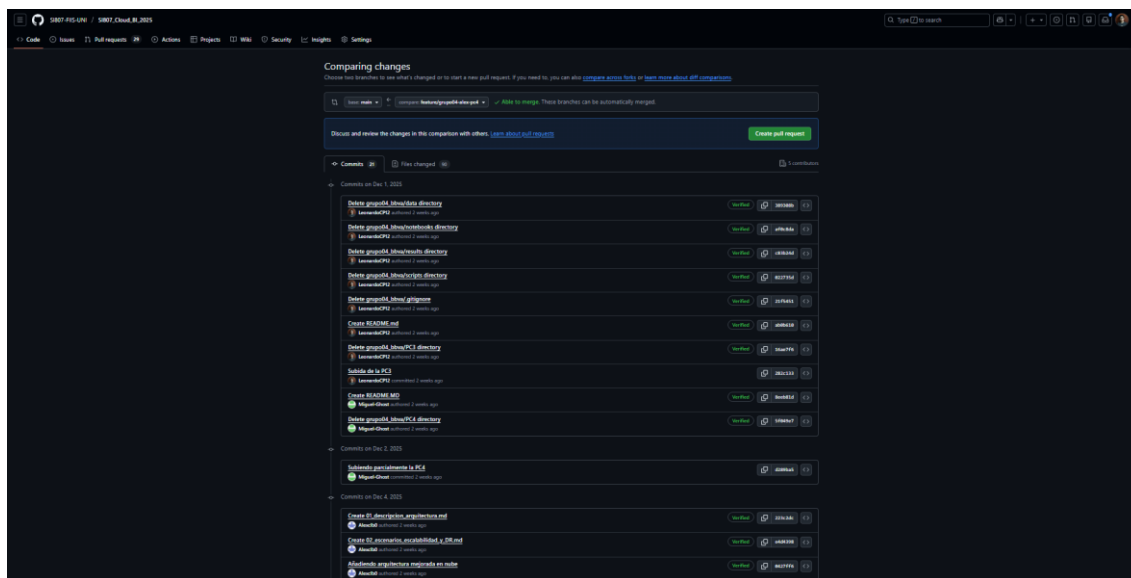
Figura 78. Contribuciones por Miembro



Fuente: *GitHub.*

Como se mencionó se subió el trabajo en la rama feature/grupo04-leonardo-pc4 y finalmente a la rama feature/grupo04-alex-pc4 del github público.

Figura 79. Merge por Miembro en la rama feature/grupo04-alex-pc4



Fuente: *Github.*

6. REFERENCIAS BIBLIOGRAFIA

- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. Proceedings of CIDR 2021. http://cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf
- Databricks. (2024). The Databricks Lakehouse Platform Architecture Guide. <https://www.databricks.com/product/data-lakehouse>
- Flexera. (2024). State of the Cloud Report: Cloud Pricing Analysis. Flexera Software LLC. https://info.flexera.com/CM-REPORT-State-of-the-Cloud?lead_source=Organic%20Search
- Gartner. (2024). Magic Quadrant for Cloud Infrastructure and Platform Services. Gartner, Inc. <https://cloud.google.com/blog/products/databases/2024-gartner-magic-quadrant-for-cloud-database-management-systems>
- Inmon, W. H., Linstedt, D., & Levins, M. (2019). Data architecture: A primer for the data scientist. Academic Press.
- Kimball, R., & Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3rd ed.). Wiley.
- Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media.
- Machado, I., Costa, C., & Santos, M. Y. (2022). Data mesh: Concepts and principles of a paradigm shift in data architectures. Procedia Computer Science, 196, 263-271. <https://doi.org/10.1016/j.procs.2021.12.013>
- Microsoft Azure. (2024). Azure Architecture Center: Reference architectures. Microsoft Corporation. <https://learn.microsoft.com/en-us/azure/architecture/>

- Microsoft Azure. (2024). Azure Cost Management + Billing documentation. Microsoft Corporation. <https://learn.microsoft.com/en-us/azure/cost-management-billing/>
- Microsoft Azure. (2024). Azure Migration and Modernization Program Guide. Microsoft Corporation. <https://azure.microsoft.com/en-us/solutions/migration/>
- Microsoft Azure. (2024). Azure Pricing Calculator. Microsoft Corporation. <https://azure.microsoft.com/en-us/pricing/calculator/>
- Microsoft Azure. (2024). Azure role-based access control (RBAC) documentation. Microsoft Corporation. <https://learn.microsoft.com/en-us/azure/role-based-access-control/>
- Microsoft Azure. (2024). Azure security documentation. Microsoft Corporation. <https://learn.microsoft.com/en-us/azure/security/>
- Microsoft Azure. (2024). Cloud Adoption Framework for Azure. Microsoft Corporation. <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/>
- Reis, J., & Housley, M. (2022). Fundamentals of data engineering: Plan and build robust data systems. O'Reilly Media.
- Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster computing with working sets. HotCloud, 10(10-10), 95. https://www.usenix.org/legacy/event/hotcloud10/tech/full_papers/Zaharia.pdf